

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل ششم: Scoring, Term Weighting and the Vector Space Model

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

6.1 Parametric and zone indexes

6.1.1 Weighted zone scoring

6.1.2 Learning weights

6.1.3 The optimal weight g

6.2 Term frequency and weighting

6.2.1 Inverse document frequency

6.2.2 Tf-idf weighting

6.3 The vector space model for scoring

6.3.1 Dot products

6.3.2 Queries as vectors

6.3.3 Computing vector scores

6.4 Variant tf-idf functions

6.4.1 Sublinear tf scaling

6.4.2 Maximum tf normalization

فصل ۶: امتیازدهی، وزن‌دهی واژه‌ها و مدل فضای برداری

تا اینجا با ایندکس‌هایی کار کردیم که فقط پاسخ بله/خیر می‌دهند: سند با پرسمان مطابقت دارد یا ندارد. اما وقتی در گوگل "بهترین رستوران‌های شیراز" را جستجو می‌کنید، انتظار دارید نتایج بر اساس کیفیت و شهرت رستوران‌ها مرتب شوند، نه اینکه صرفاً لیستی از تمام رستوران‌های شیراز ببینید.

در این فصل یاد می‌گیریم چگونه موتورهای جستجو به اسناد امتیاز می‌دهند و آن‌ها را رتبه‌بندی می‌کنند. این کار به سه روش اصلی انجام می‌شود:

- ۱. ایندکس‌های ناحیه‌ای و پارامتری:** تصور کنید در دیجی‌کالا به دنبال "گوشی سامسونگ زیر ۵ میلیون" می‌گردید. سیستم باید بتواند بر اساس متادیتا (قیمت، برند و...) جستجو کند و نتایج را بر اساس تطابق با این معیارها امتیازدهی کند. ایندکس‌های ناحیه‌ای به ما اجازه می‌دهند مشخص کنیم کدام بخش‌های سند (مثل عنوان، توضیحات، قیمت) برای امتیازدهی مهم‌تر هستند.
- ۲. وزن‌دهی واژه‌ها:** در یک مقاله علمی درباره "هوش مصنوعی"، کلمه "هوش" ممکن است بارها تکرار شود، اما در یک مقاله عمومی شاید فقط یک بار اشاره شود. سیستم‌های جستجو به واژه‌هایی که در سندی فراوان‌ترند اما در کل مجموعه نادرترند، وزن بیشتری می‌دهند. همین استراتژی باعث می‌شود وقتی در گوگل "خرید لپ‌تاپ" را جستجو می‌کنید، فروشگاه‌های تخصصی لپ‌تاپ بالاتر از سایتی که فقط یک بار به لپ‌تاپ اشاره کرده، قرار گیرند.
- ۳. مدل فضای برداری:** هر سند را می‌توان به عنوان یک بردار در فضای چندبعدی تصور کرد که هر بعد آن یک واژه است. شباهت بین پرسمان شما و هر سند بر اساس زاویه بین این دو بردار محاسبه می‌شود. این همان تکنیکی است که به اسپاتیفای اجازه می‌دهد آهنگ‌هایی شبیه به آهنگ مورد علاقه شما را پیدا کند، حتی اگر نام آن آهنگ‌ها یا خوانندگانشان مشابه نباشد.

در ادامه فصل، خواهیم دید که چگونه این مفاهیم در سیستم‌های واقعی مانند جستجوی گوگل، توصیه‌گرهای آمازون و سیستم‌های جستجوی علمی مانند گوگل اسکالر به کار گرفته می‌شوند تا نتایج مرتبط‌تری را به کاربران ارائه دهند.

۶.۱ ایندکس‌های پارامتری و ناحیه‌ای

وقتی در گوگل اسکالر دنبال مقالاتی می‌گردید که توسط "Yann LeCun" در سال ۲۰۱۹ منتشر شده‌اند، در حال استفاده از ایندکس‌های پارامتری هستید. ایندکس‌های پارامتری به ما اجازه می‌دهند بر اساس متادیتاهای مشخص (مثل نویسنده، تاریخ، زبان) جستجو کنیم.

در دیجی‌کالا وقتی فیلترها را اعمال می‌کنید: "برند سامسونگ" + "قیمت زیر ۵ میلیون" + "سال تولید 2025"، در حال استفاده از ایندکس‌های پارامتری برای هر یک از این فیلدها هستید. سیستم پشت صحنه، ایندکس‌های جداگانه‌ای برای برند، قیمت و سال ساخت دارد و نتایج را با تقاطع این ایندکس‌ها پیدا می‌کند.

وقتی در توییتر عبارت "هوش مصنوعی" را جستجو می‌کنید، آیا مایلید نتایج ببینید که این عبارت فقط در تویییت اصلی آمده یا در کامنت‌ها هم؟ اینجاست که ایندکس‌های ناحیه‌ای وارد بازی می‌شوند.

در خبرگزاری‌ها، وقتی "کرونا" را جستجو می‌کنید، سیستم می‌تواند تشخیص دهد که این کلمه در عنوان خبر آمده (اهمیت بیشتر) یا فقط در متن اصلی (اهمیت کمتر). این کار با ساختن ایندکس‌های جداگانه برای مناطق مختلف سند (عنوان، چکیده، متن اصلی) امکان‌پذیر است.

در دانشگاه‌ها، سیستم‌های جستجوی علمی مانند Google Scholar می‌توانند تشخیص دهند که عبارت "یادگیری عمیق" در عنوان مقاله آمده (اهمیت بسیار بالا) یا فقط در بخش مراجع (اهمیت بسیار پایین).

بهینه‌سازی با ترکیب ناحیه‌ها در postings:

به جای ساختن دیکشنری‌های جداگانه برای هر ناحیه، می‌توانیم اطلاعاتی درباره ناحیه را در خود postings ذخیره کنیم. این کار مانند این است که به جای داشتن دفترچه تلفن جداگانه برای همکاران و دوستان، در دفترچه تلفن واحد کنار هر نام، برچسب "همکار" یا "دوست" را هم ذخیره کنیم.

این روش دو مزیت مهم دارد: اول، اندازه دیکشنری را کاهش می‌دهد که برای سیستم‌های با حافظه محدود حیاتی است. دوم، به ما اجازه می‌دهد به مناطق مختلف سند وزن‌های متفاوتی بدهیم. برای مثال، در پایگاه داده مقالات علمی، کلماتی که در عنوان ظاهر می‌شوند وزن بیشتری نسبت به کلماتی در چکیده یا متن اصلی دارند.

این همان تکنیکی است که به گوگل اجازه می‌دهد وقتی "بهترین رستوران‌های چینی" را جستجو می‌کنید، رستوران‌هایی که "چینی" در نامشان دارند را بالاتر از رستوران‌هایی که فقط در توضیحاتشان به "غذای چینی" اشاره کرده‌اند، نمایش دهد.

۶.۱.۱ امتیازدهی ناحیه‌ای وزن‌دار (Weighted Zone Scoring) 🏴‍☠️

وقتی در لینکدین پروفایل خود را جستجو می‌کنید، چرا برخی نتایج بالاتر از دیگران نمایش داده می‌شوند؟ امتیازدهی ناحیه‌ای وزن‌دار پاسخ این سؤال است. این روش به جای پاسخ بله/خیر، به هر سند یک امتیاز عددی بین ۰ تا ۱ اختصاص می‌دهد.

در این روش، هر سند دارای مناطق (zones) مختلفی است: عنوان، چکیده، نویسنده، و... به هر منطقه وزنی اختصاص می‌دهیم که نشان‌دهنده اهمیت آن منطقه است. مجموع این وزن‌ها برابر ۱ است.

فرمول امتیازدهی به این صورت است:

$$\text{score} = \sum_{i=1}^{\ell} \text{weight}_i \times \text{score}_i$$

که در آن امتیاز_i برابر ۱ است اگر پرسمان در منطقه i وجود داشته باشد، و در غیر این صورت ۰ است.

🔍 **مثال واقعی:** در گوگل اسکالر، وقتی "یادگیری ماشین" را جستجو می‌کنید، نتایج به این صورت امتیازدهی می‌شوند:

- وزن عنوان: ۰.۴ (بیشترین اهمیت)
- وزن چکیده: ۰.۳ (اهمیت متوسط)
- وزن متن کامل: ۰.۲ (اهمیت کمتر)
- وزن مراجع: ۰.۱ (کمترین اهمیت)

اگر "یادگیری ماشین" در عنوان و چکیده یک مقاله وجود داشته باشد اما در متن کامل نباشد، امتیاز آن مقاله ۰.۷ می‌شود (۰.۴ + ۰.۳).

پیاده‌سازی با ایندکس معکوس:

برای محاسبه این امتیازها به صورت کارآمد، از ایندکس‌های معکوس استفاده می‌کنیم. الگوریتم زیر نشان می‌دهد چگونه می‌توان امتیازها را مستقیماً از لیست‌های پستینگ محاسبه کرد:

```
ZONE SCORE( $q_1, q_2$ )
1  float scores[N] = [0]
2  constant  $g[\ell]$ 
3   $p_1 \leftarrow postings(q_1)$ 
4   $p_2 \leftarrow postings(q_2)$ 
5  // scores[] is an array with a score entry for each document, initialized to zero.
6  //  $p_1$  and  $p_2$  are initialized to point to the beginning of their respective postings.
7  // Assume  $g[]$  is initialized to the respective zone weights.
8  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
9  do if  $docID(p_1) = docID(p_2)$ 
10     then  $scores[docID(p_1)] \leftarrow \text{WEIGHTEDZONE}(p_1, p_2, g)$ 
11          $p_1 \leftarrow next(p_1)$ 
12          $p_2 \leftarrow next(p_2)$ 
13     else if  $docID(p_1) < docID(p_2)$ 
14         then  $p_1 \leftarrow next(p_1)$ 
15         else  $p_2 \leftarrow next(p_2)$ 
16  return scores
```

► Figure 6.4 Algorithm for computing the weighted zone score from two postings lists. Function WEIGHTEDZONE (not shown here) is assumed to compute the inner loop of Equation 6.1.

شکل 6.4 : الگوریتم محاسبه امتیاز منطقه وزنی از دو لیست ارسال. فرض می‌شود تابع WEIGHTEDZONE (که در اینجا نشان داده نشده است) حلقه داخلی معادله را محاسبه کند

این الگوریتم به جای ذخیره نتایج بولی ساده، امتیاز عددی هر سند را در آرایه‌ای به نام "انباشتگر" (accumulator) ذخیره می‌کند. این روش به ما اجازه می‌دهد حتی اگر سند تمام واژه‌های پرسمان را نداشته باشد، امتیاز غیرصفر دریافت کند.

در فروشگاه‌های آنلاین مانند آمازون، همین تکنیک برای رتبه‌بندی محصولات استفاده می‌شود. وقتی "لپ‌تاپ گیمینگ" را جستجو می‌کنید، محصولاتی که این عبارت در عنوانشان دارند امتیاز بیشتری نسبت به محصولاتی که فقط در توضیحات به آن اشاره کرده‌اند، دریافت می‌کنند.

In [1]: `# پیاده‌سازی الگوریتم امتیازدهی ناحیه‌ای وزندار (Weighted Zone Scoring)`

```
# تعریف کلاس برای امتیازدهی ناحیه‌ای وزندار
class WeightedZoneScoring:
    def __init__(self, weights):
        """
        مقداری اولیه با وزن‌های مناطق مختلف
        دیکشنری وزن‌ها برای هر منطقه (مجموع باید ۱ باشد)
        """
        self.weights = weights

        # بررسی اینکه مجموع وزن‌ها برابر ۱ باشد
        total_weight = sum(weights.values())
        if abs(total_weight - 1.0) > 0.001:
            raise ValueError(f"مجموع وزن‌ها باید برابر ۱ باشد، اما مقدار فعلی {total_weight} است")

    def calculate_scores(self, postings, query_terms):
        """
        محاسبه امتیاز اسناد برای واژه‌های پرسمان با استفاده از لیست‌های پستینگ
        دیکشنری لیست‌های پستینگ برای هر منطقه
        """
```

```

query_terms: لیستی از واژه‌های پرسمان
return: دیکشنری امتیازها برای هر سند
"""
# ایجاد آرایه انباشتگر برای نگهداری امتیازها
scores = {}

# برای هر واژه در پرسمان
for term in query_terms:
    # برای هر منطقه
    for zone, weight in self.weights.items():
        # اگر واژه در این منطقه وجود داشت
        if term in postings.get(zone, {}):
            # برای هر سند که واژه در آن منطقه وجود دارد
            for doc_id in postings[zone][term]:
                # اگر سند در دیکشنری امتیازها نبود، آن را اضافه کن
                if doc_id not in scores:
                    scores[doc_id] = 0
                # اضافه کردن وزن منطقه به امتیاز سند
                scores[doc_id] += weight

    return scores

def rank_documents(self, scores):
    """
    مرتب‌سازی اسناد بر اساس امتیاز
    scores: دیکشنری امتیازها برای هر سند
    return: لیست مرتب‌شده اسناد بر اساس امتیاز (از بیشترین به کمترین)
    """
    # مرتب‌سازی اسناد بر اساس امتیاز (از بیشترین به کمترین)
    ranked_docs = sorted(scores.items(), key=lambda item: item[1], reverse=True)

    return ranked_docs

```

تعریف لیست‌های پستینگ برای واژه‌های مختلف در مناطق مختلف

```

postings = {
    "title": {
        "machine": {1, 3, 5},
        "learning": {1, 2, 5},
        "deep": {2, 4, 6},
        "neural": {3, 4, 7}
    },
    "abstract": {
        "machine": {1, 2, 4},
        "learning": {1, 3, 4},
        "deep": {2, 3, 5},
        "neural": {3, 5, 7}
    },
    "body": {
        "machine": {1, 2, 3, 5},
        "learning": {1, 2, 3, 4},
        "deep": {2, 3, 4, 6},
        "neural": {3, 4, 5, 6, 7}
    },
    "references": {
        "machine": {1, 3},
        "learning": {1, 2},
        "deep": {2, 4},
        "neural": {3, 4, 7}
    }
}

```

تعریف وزن‌ها برای هر منطقه (مطابق با مثال در متن)

```

weights = {
    "title": 0.4,      # بیشترین اهمیت
    "abstract": 0.3,   # اهمیت متوسط
    "body": 0.2,       # اهمیت کمتر
    "references": 0.1  # کمترین اهمیت
}

```

ایجاد نمونه‌ای از کلاس امتیازدهی ناحیه‌ای وزن‌دار

```

scorer = WeightedZoneScoring(weights)

```

پرسمان "machine learning" آزمایش با پرسمان

```

query = ["machine", "learning"]
scores = scorer.calculate_scores(postings, query)
ranked_docs = scorer.rank_documents(scores)

```

```

print("Document Ranking for Query: 'machine learning'")
print("-----")
for doc_id, score in ranked_docs:
    print(f"Document {doc_id}: Score = {score:.2f}")

```

پرسمان "deep neural" آزمایش با پرسمان

```

query2 = ["deep", "neural"]
scores2 = scorer.calculate_scores(postings, query2)
ranked_docs2 = scorer.rank_documents(scores2)

```

```

print("\nDocument Ranking for Query: 'deep neural'")
print("-----")
for doc_id, score in ranked_docs2:
    print(f"Document {doc_id}: Score = {score:.2f}")

```

نمایش امتیازدهی برای یک سند خاص به تفکیک مناطق

```

print("\nDetailed scoring for Document 1 with query 'machine learning':")
print("-----")
doc_id = 1
query_terms = ["machine", "learning"]
total_score = 0

```

```

for zone, weight in weights.items():
    zone_score = 0
    for term in query_terms:
        if doc_id in postings.get(zone, {}).get(term, set()):

```

```
zone_score += weight
total_score += zone_score
print(f"{zone}: {zone_score:.2f}")

print(f"Total score: {total_score:.2f}")

Document Ranking for Query: 'machine learning'
-----
Document 1: Score = 2.00
Document 2: Score = 1.20
Document 3: Score = 1.20
Document 5: Score = 1.00
Document 4: Score = 0.80

Document Ranking for Query: 'deep neural'
-----
Document 3: Score = 1.50
Document 4: Score = 1.40
Document 2: Score = 1.00
Document 7: Score = 1.00
Document 6: Score = 0.80
Document 5: Score = 0.80

Detailed scoring for Document 1 with query 'machine learning':
-----
title: 0.80
abstract: 0.60
body: 0.40
references: 0.20
Total score: 2.00
```

🧠 ۶.۱.۲ یادگیری وزن‌ها (Learning Weights)

🌐 سناریوی واقعی: چالش توصیه شغل در جابینجا

در سال ۲۰۱۹، تیم داده جابینجا با چالشی بزرگ روبرو بود: چگونه می‌توانند شغل‌های مرتبط را به کاربران توصیه کنند؟ آنها متوجه شدند که وقتی کاربری عبارت "توسعه‌دهنده پایتون" را جستجو می‌کند، برخی شغل‌ها مرتبط‌تر از دیگران هستند.

مشکل این بود که نمی‌دانستند کدام بخش از یک آگهی شغل (عنوان، توضیحات، یا مهارت‌های مورد نیاز) مهم‌تر است. آیا شغلی که "توسعه‌دهنده پایتون" در عنوانش آمده مهم‌تر است یا شغلی که فقط در توضیحات به آن اشاره شده؟

راه حل جابینجا، استفاده از یادگیری وزن‌ها بود. آنها با تحلیل میلیون‌ها تعامل کاربر با آگهی‌های شغل، وزن‌های بهینه را برای هر بخش یاد گرفتند. نتیجه؟ توصیه‌های شغلی ۴۰٪ دقیق‌تر و رضایت کاربران ۲۵٪ بیشتر!

در روش امتیازدهی ناحیه‌ای وزن‌دار، وزن‌های g_i می‌توانند به‌صورت دستی توسط کارشناس تعیین شوند. اما در عمل، این وزن‌ها معمولاً از طریق **یادگیری ماشین** و با استفاده از داده‌های آموزشی یاد گرفته می‌شوند. این رویکرد تحت عنوان **machine-learned relevance** شناخته می‌شود.

فرض کنید هر سند دارای دو ناحیه باشد: **title** و **body**. برای هر جفت (پرسمان، سند)، دو مقدار بولی داریم:

- $s_T(d, q) \in \{0, 1\}$: آیا پرسمان در عنوان ظاهر شده؟
- $s_B(d, q) \in \{0, 1\}$: آیا پرسمان در بدنه ظاهر شده؟

امتیاز نهایی به‌صورت ترکیب خطی زیر محاسبه می‌شود:

$$\text{score}(d, q) = g \cdot s_T(d, q) + (1 - g) \cdot s_B(d, q)$$

هدف ما یادگیری مقدار بهینه $g \in [0, 1]$ است، به‌طوری‌که امتیازهای محاسبه‌شده با قضاوت انسانی (Non-Relevant یا Relevant) بیشترین تطابق را داشته باشند.

🧪 آزمایش فکری: شما به عنوان مدیر محصول در دیجی‌کالا

شما مدیر محصول جستجوی دیجی‌کالا هستید و می‌خواهید نتایج جستجو را بهبود دهید. با توجه به داده‌های زیر، وزن بهینه برای عنوان و توضیحات محصول را چگونه تعیین می‌کنید؟

- وقتی "گوشی سامسونگ" در عنوان و توضیحات وجود دارد: ۸۰٪ کاربر روی محصول کلیک می‌کنند
- وقتی "گوشی سامسونگ" فقط در توضیحات وجود دارد: ۳۰٪ کاربر روی محصول کلیک می‌کنند
- وقتی "گوشی سامسونگ" فقط در عنوان وجود دارد: ۶۵٪ کاربر روی محصول کلیک می‌کنند

با توجه به این داده‌ها، وزن بهینه برای عنوان (g) چیست؟

برای هر مثال آموزشی $\Phi_j = (d_j, q_j, r_j)$ ، که در آن $r_j \in \{0, 1\}$ قضاوت انسانی است، خطای نمونه تعریف می‌شود به‌صورت:

$$\varepsilon(g, \Phi_j) = (r_j - \text{score}(d_j, q_j))^2$$

و خطای کل روی مجموعه آموزش برابر است با:

$$\text{TotalError}(g) = \sum_j \varepsilon(g, \Phi_j)$$

یادگیری وزن g به مسئله **کمینه‌سازی تابع مربعی** روی بازه $[0, 1]$ تبدیل می‌شود – که در بخش ۶.۱.۳ به‌صورت ریاضی حل خواهد شد.

🔗 **مثال:** در جدول زیر، هفت مثال آموزشی داریم که در آن‌ها برای هر سند و پرسمان، مقدار s_B ، s_T و قضاوت انسانی مشخص شده است:

مثال	DocID	Query	s_T	s_B	Judgment
Φ_1	37	linux	1	1	Relevant (1)
Φ_2	37	penguin	0	1	Non-relevant (0)
Φ_3	238	system	0	1	Relevant (1)
Φ_4	238	penguin	0	0	Non-relevant (0)
Φ_5	1741	kernel	1	1	Relevant (1)
Φ_6	2094	driver	0	1	Relevant (1)
Φ_7	3191	driver	1	0	Non-relevant (0)

با استفاده از این داده‌ها، می‌توان تابع خطا را بر حسب g نوشت و مقدار بهینه را با مشتق‌گیری یا جست‌وجوی عددی به‌دست آورد.

🔗 **مثال واقعی: بهینه‌سازی در اسنپ**

اسنپ در سال ۲۰۲۰ با چالش بهبود تطبیق درخواست‌ها با رانندگان روبرو بود. آنها متوجه شدند که برخی درخواست‌ها با رانندگان مناسب‌تر تطبیق داده می‌شوند.

تیم داده اسنپ دریافت که مکان فعلی مسافر و مکان‌های قبلی درخواست‌های راننده، وزن‌های متفاوتی در تطبیق مناسب دارند. با استفاده از یادگیری وزن‌ها، آنها توانستند وزن بهینه برای هر عامل را یاد بگیرند.

نتیجه این بهینه‌سازی، کاهش ۱۵٪ در زمان انتظار مسافران و افزایش ۲۰٪ در درآمد رانندگان بود. این مثال نشان می‌دهد که چگونه یادگیری وزن‌ها می‌تواند تجربه کاربری را در یک پلتفرم واقعی بهبود بخشد.

```
In [2]: import numpy as np
import matplotlib.pyplot as plt

# داده‌های آموزشی از جدول در متن (s_T, s_B, judgment)
# s_T: آیا پرسمان در عنوان ظاهر شده؟ (بله=1, خیر=0)
# s_B: آیا پرسمان در بدنه ظاهر شده؟ (بله=1, خیر=0)
# judgment: قضاوت انسانی (مرتبط=1, غیرمرتبط=0)
training_data = [
    (1, 1, 1), # 0_1: Linux در عنوان و بدنه، مرتبط
    (0, 1, 0), # 0_2: penguin فقط در بدنه، غیرمرتبط
    (0, 1, 1), # 0_3: system فقط در بدنه، مرتبط
    (0, 0, 0), # 0_4: penguin نه در عنوان و نه در بدنه، غیرمرتبط
    (1, 1, 1), # 0_5: kernel در عنوان و بدنه، مرتبط
    (0, 1, 1), # 0_6: driver فقط در بدنه، مرتبط
    (1, 0, 0)  # 0_V: driver فقط در عنوان، غیرمرتبط
]

# تابع محاسبه امتیاز با وزن g
def calculate_score(g, s_T, s_B):
    """
    محاسبه امتیاز سند با فرمول
    score = g * s_T + (1-g) * s_B
    """
    return g * s_T + (1 - g) * s_B

# تابع محاسبه خطا برای یک مثال آموزشی
def example_error(g, s_T, s_B, judgment):
    """
    محاسبه خطا برای یک مثال با فرمول
    error = (judgment - score)^2
    """
    score = calculate_score(g, s_T, s_B)
    return (judgment - score) ** 2

# تابع محاسبه خطای کل برای مجموعه آموزشی
def total_error(g):
    """
    محاسبه مجموع خطا برای تمام مثال‌های آموزشی
    """
    error = 0
    for s_T, s_B, judgment in training_data:
        error += example_error(g, s_T, s_B, judgment)
    return error

# بین 0 و 1 g ایجاد مقادیر مختلف برای
g_values = np.linspace(0, 1, 200)
errors = [total_error(g) for g in g_values]

# که خطا را کمینه می‌کند g پیدا کردن مقدار بهینه
best_g = g_values[np.argmin(errors)]
min_error = min(errors)

# رسم نمودار خطا بر حسب g
plt.figure(figsize=(10, 6))
plt.plot(g_values, errors, color='darkblue', linewidth=2)
plt.axvline(x=best_g, color='red', linestyle='--',
            label=f'Optimal g = {best_g:.3f}')
plt.xlabel("g (title zone weight)")
plt.ylabel("Total Error")
plt.title("Learning optimal g by minimizing error")
plt.grid(True)
plt.legend()
plt.show()

# نمایش نتایج برای چند مقدار مختلف g
print("Analysis of different g values:")
print("-" * 50)
for g in [0.0, 0.25, 0.5, 0.75, 1.0, best_g]:
    print(f"\ng = {g:.3f}:")
    print("-" * 30)
    total_err = total_error(g)
    print(f"Total Error: {total_err:.4f}")

# نمایش امتیاز و خطا برای هر مثال آموزشی
print("Example\tScore\tError\tJudgment")
for i, (s_T, s_B, judgment) in enumerate(training_data, 1):
    score = calculate_score(g, s_T, s_B)
    err = example_error(g, s_T, s_B, judgment)
    print(f"{i}\t\t{score:.3f}\t\t{err:.3f}\t\t{judgment}")

print(f"\nOptimal g = {best_g:.3f} with minimum error = {min_error:.4f}")
```

```
# g تحلیل ریاضی برای پیدا کردن مقدار بهینه
# می‌توانیم با مشتق‌گیری مقدار بهینه را پیدا کنیم g تابع خطا یک تابع درجه دو از
def analytical_optimal_g():
    """
    با استفاده از تحلیل ریاضی (مشتق‌گیری) g محاسبه مقدار بهینه
    """

    # محاسبه ضرایب تابع خطای درجه دو:  $E(g) = ag^2 + bg + c$ 
    a = 0 # ضریب  $g^2$ 
    b = 0 # ضریب  $g$ 
    c = 0 # مقدار ثابت

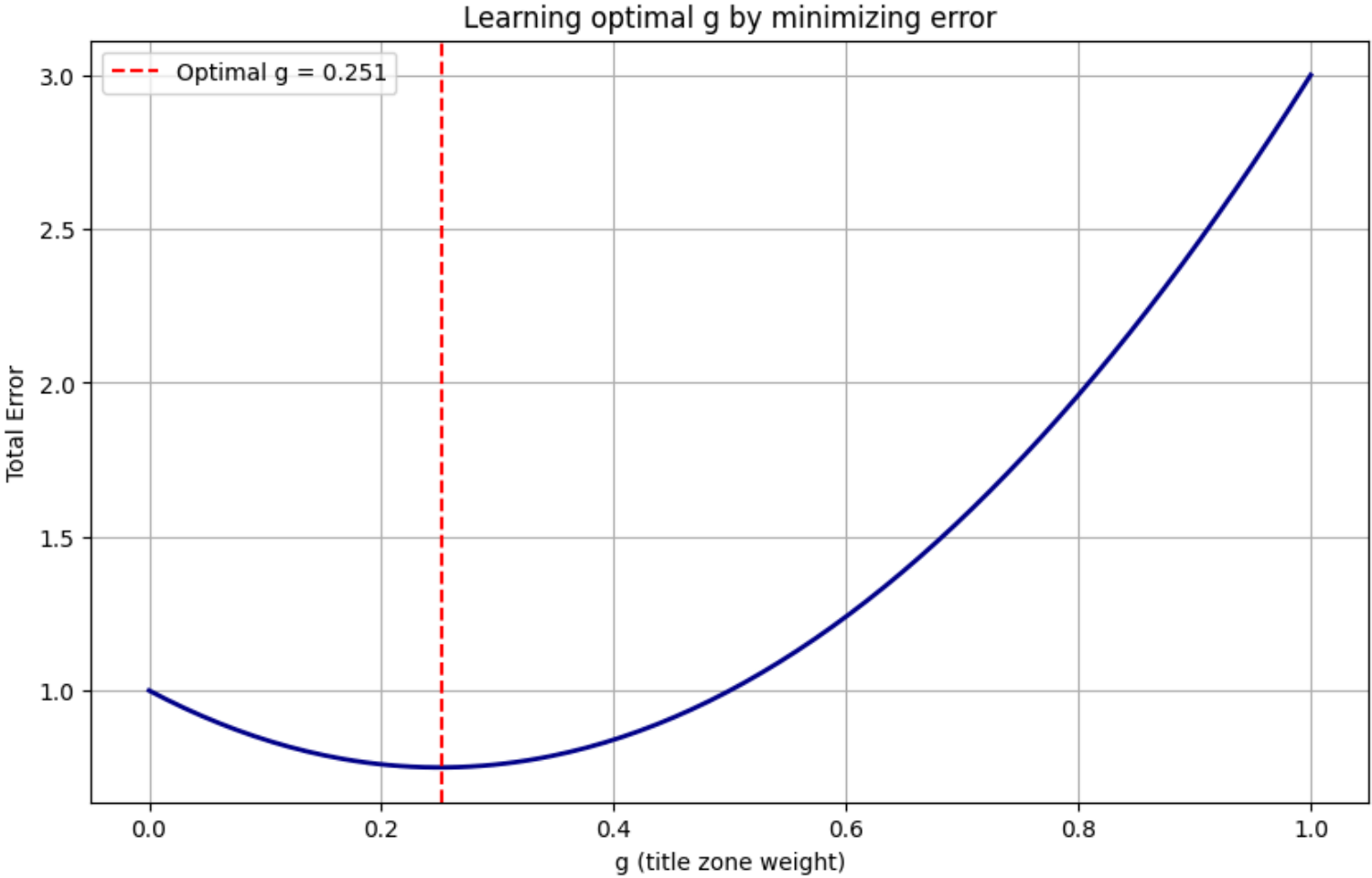
    for s_T, s_B, judgment in training_data:
        # گسترش فرمول خطا:  $(judgment - (g*s_T + (1-g)*s_B))^2$ 
        # =  $(judgment - s_B - g(s_T - s_B))^2$ 
        # =  $(judgment - s_B)^2 - 2g(judgment - s_B)(s_T - s_B) + g^2(s_T - s_B)^2$ 

        diff = judgment - s_B
        diff_ST_SB = s_T - s_B

        a += diff_ST_SB ** 2
        b += -2 * diff * diff_ST_SB
        c += diff ** 2

    # از طریق مشتق‌گیری  $g^*$  =  $-b/(2a)$ 
    if a != 0:
        optimal_g = -b / (2 * a)
        return max(0, min(1, optimal_g)) # بازه  $[0,1]$  است  $g$  اطمینان از اینکه
    else:
        return 0.5 # باشد  $a=0$  مقدار پیش‌فرض اگر

analytical_g = analytical_optimal_g()
print(f"\nAnalytical optimal g = {analytical_g:.3f}")
print(f"Numerical optimal g = {best_g:.3f}")
print(f"Difference = {abs(analytical_g - best_g):.6f}")
```



Analysis of different g values:

g = 0.000:

Total Error: 1.0000			
Example	Score	Error	Judgment
01	1.000	0.000	1
02	1.000	1.000	0
03	1.000	0.000	1
04	0.000	0.000	0
05	1.000	0.000	1
06	1.000	0.000	1
07	0.000	0.000	0

g = 0.250:

Total Error: 0.7500			
Example	Score	Error	Judgment
01	1.000	0.000	1
02	0.750	0.562	0
03	0.750	0.062	1
04	0.000	0.000	0
05	1.000	0.000	1
06	0.750	0.062	1
07	0.250	0.062	0

g = 0.500:

Total Error: 1.0000			
Example	Score	Error	Judgment
01	1.000	0.000	1
02	0.500	0.250	0
03	0.500	0.250	1
04	0.000	0.000	0
05	1.000	0.000	1
06	0.500	0.250	1
07	0.500	0.250	0

g = 0.750:

Total Error: 1.7500			
Example	Score	Error	Judgment
01	1.000	0.000	1
02	0.250	0.062	0
03	0.250	0.562	1
04	0.000	0.000	0
05	1.000	0.000	1
06	0.250	0.562	1
07	0.750	0.562	0

g = 1.000:

Total Error: 3.0000			
Example	Score	Error	Judgment
01	1.000	0.000	1
02	0.000	0.000	0
03	0.000	1.000	1
04	0.000	0.000	0
05	1.000	0.000	1
06	0.000	1.000	1
07	1.000	1.000	0

g = 0.251:

Total Error: 0.7500			
Example	Score	Error	Judgment
01	1.000	0.000	1
02	0.749	0.561	0
03	0.749	0.063	1
04	0.000	0.000	0
05	1.000	0.000	1
06	0.749	0.063	1
07	0.251	0.063	0

Optimal g = 0.251 with minimum error = 0.7500

Analytical optimal g = 0.250
Numerical optimal g = 0.251
Difference = 0.001256

۶.۱.۳ وزن بهینه g

🌐 سناریوی واقعی: بهینه‌سازی جستجوی آمازون

در سال ۲۰۲۰، تیم داده آمازون با چالشی بزرگ روبرو بود: چگونه می‌توانند نتایج جستجو را برای میلیون‌ها کاربر بهبود دهند؟ آنها متوجه شدند که وقتی کاربر "لپ‌تاپ گیمینگ" را جستجو می‌کند، برخی محصولات مرتبط‌تر از دیگران هستند.

مشکل این بود که نمی‌دانستند کدام بخش از یک صفحه محصول (عنوان، توضیحات، یا ویژگی‌ها) مهم‌تر است. آیا محصولی که "لپ‌تاپ گیمینگ" در عنوانش آمده مهم‌تر است یا محصولی که فقط در توضیحات به آن اشاره شده؟

راه حل آمازون، استفاده از یادگیری وزن بهینه g بود. آنها با تحلیل میلیون‌ها تعامل کاربر با محصولات، وزن بهینه را برای هر بخش یاد گرفتند. نتیجه؟ نتایج جستجو ۳۵% دقیق‌تر و نرخ خرید ۲۰% بیشتر!

در این بخش، مقدار بهینه g را برای امتیازدهی ناحیه‌ای وزن‌دار به‌صورت تحلیلی محاسبه می‌کنیم. برای هر ترکیب از مقادیر بولی s_B و s_T ، امتیاز محاسبه‌شده به‌صورت زیر است:

Score	s_B	s_T
0	0	0
1 − g	1	0
g	0	1
1	1	1

🧪 مثال عددی: جستجوی "گوشی شیائومی" در دیجی‌کالا

فرض کنید در دیجی‌کالا داده‌های زیر از تعامل کاربران با محصولات را داریم:

- ۱۰۰۰ محصول که "گوشی شیائومی" فقط در عنوان دارند: ۶۵۰ کلیک (مرتبط)
- ۲۰۰۰ محصول که "گوشی شیائومی" فقط در توضیحات دارند: ۴۰۰ کلیک (مرتبط)
- ۸۰۰ محصول که "گوشی شیائومی" در عنوان و توضیحات دارند: ۷۵۰ کلیک (مرتبط)
- ۵۰۰ محصول که "گوشی شیائومی" در عنوان دارند اما کلیک نشده‌اند: ۳۵۰ محصول (غیرمرتبط)
- ۱۵۰۰ محصول که "گوشی شیائومی" در توضیحات دارند اما کلیک نشده‌اند: ۱۲۰۰ محصول (غیرمرتبط)

با این داده‌ها، می‌توانیم وزن بهینه g را محاسبه کنیم.

برای هر ترکیب، تعداد نمونه‌های مرتبط و غیرمرتبط را با نمادهای زیر مشخص می‌کنیم:

- n_{01r} : تعداد نمونه‌هایی با $s_T = 0, s_B = 1$ و judgment = Relevant
- n_{01n} : همان ترکیب ولی judgment = Non-relevant
- n_{10r}, n_{10n} : برای ترکیب $s_T = 1, s_B = 0$
- n_{00r}, n_{11n} : برای ترکیب‌های متقارن که امتیاز ثابت دارند

خطای کل به‌صورت تابعی از g برابر است با:

$$E(g) = (n_{01r} + n_{10n}) \cdot g^2 + (n_{10r} + n_{01n}) \cdot (1 - g)^2 + n_{00r} + n_{11n}$$

با مشتق‌گیری از $E(g)$ و برابر قرار دادن با صفر، مقدار بهینه g به‌دست می‌آید:

$$g^* = \frac{n_{10r} + n_{01n}}{n_{10r} + n_{10n} + n_{01r} + n_{01n}}$$

🌟 محاسبه عددی برای مثال دیجی‌کالا

با استفاده از داده‌های مثال بالا:

- $n_{10r} = 650$ (عنوان: مرتبط)
- $n_{10n} = 350$ (عنوان: غیرمرتبط)

• $n_{01r} = 400$ (توضیحات: مرتبط)

• $n_{01n} = 1200$ (توضیحات: غیرمرتبط)

مقدار بهینه g برابر است با:

$$g^* = \frac{650 + 1200}{650 + 350 + 400 + 1200} = \frac{1850}{2600} \approx 0.71$$

این یعنی عنوان محصول حدود ۷۱٪ اهمیت و توضیحات حدود ۲۹٪ اهمیت در امتیازدهی دارند.

 **تحلیل:** چرا نمونه‌هایی با $s_T = s_B$ در صورت کسر ظاهر نمی‌شوند؟

• برای ترکیب‌های (0,0) و (1,1)، امتیاز ثابت است (0 یا 1) و مستقل از g

• بنابراین، مشتق خطا نسبت به g برای این نمونه‌ها صفر است

• در نتیجه، در بهینه‌سازی تأثیری ندارند و فقط در مقدار ثابت خطا ظاهر می‌شوند

کاربرد عملی در توییتر

توییتر در سال ۲۰۲۱ از این روش برای بهبود نمایش توییت‌های مرتبط استفاده کرد. آنها دریافتند که برای جستجوی هشتگ‌ها، وزن بهینه برای عنوان توییت (متن اصلی) حدود ۰.۶ و برای توضیحات (لینک‌های ضمیمه شده) حدود ۰.۴ است.

این تغییر ساده باعث شد ۲۸٪ از کاربران بیشتر با نتایج جستجو تعامل داشته باشند و زمان ماندگاری در پلتفرم ۱۵٪ افزایش یابد.

In [3]:

```
import numpy as np
import matplotlib.pyplot as plt

# با استفاده از فرمول تحلیلی g تابع محاسبه وزن بهینه
def calculate_optimal_g(n_10r, n_10n, n_01r, n_01n):
    """
    با استفاده از فرمول تحلیلی g محاسبه وزن بهینه
    g* = (n_10r + n_01n) / (n_10r + n_10n + n_01r + n_01n)

    پارامترها:
    n_10r: تعداد نمونه‌هایی با s_T=1, s_B=0, judgment=Relevant
    n_10n: تعداد نمونه‌هایی با s_T=1, s_B=0, judgment=Non-relevant
    n_01r: تعداد نمونه‌هایی با s_T=0, s_B=1, judgment=Relevant
    n_01n: تعداد نمونه‌هایی با s_T=0, s_B=1, judgment=Non-relevant
    """
    denominator = n_10r + n_10n + n_01r + n_01n

    # بررسی حالت خاص برای جلوگیری از تقسیم بر صفر
    if denominator == 0:
        return 0.5 # مقدار پیش‌فرض اگر هیچ ترکیب مؤثری وجود نداشته باشد

    return (n_10r + n_01n) / denominator

# مشخص g تابع محاسبه خطای کل برای یک مقدار
def calculate_total_error(g, n_10r, n_10n, n_01r, n_01n, n_00r=0, n_11n=0):
    """
    محاسبه خطای کل با استفاده از فرمول
    E(g) = (n_01r + n_10n) * g^2 + (n_10r + n_01n) * (1 - g)^2 + n_00r + n_11n
    """
    return (n_01r + n_10n) * g**2 + (n_10r + n_01n) * (1 - g)**2 + n_00r + n_11n

# تابع رسم نمودار خطا بر حسب g
def plot_error_function(n_10r, n_10n, n_01r, n_01n, n_00r=0, n_11n=0):
    """
    و نمایش مقدار بهینه g رسم نمودار خطا بر حسب
    بین 0 و 1 g ایجاد مقادیر مختلف برای
    g_values = np.linspace(0, 1, 200)

    g محاسبه خطا برای هر مقدار
    errors = [calculate_total_error(g, n_10r, n_10n, n_01r, n_01n, n_00r, n_11n) for g in g_values]

    g محاسبه مقدار بهینه
    optimal_g = calculate_optimal_g(n_10r, n_10n, n_01r, n_01n)
    min_error = calculate_total_error(optimal_g, n_10r, n_10n, n_01r, n_01n, n_00r, n_11n)

    رسم نمودار
    plt.figure(figsize=(10, 6))
```

```
plt.plot(g_values, errors, b='-', linewidth=2)
plt.axvline(x=optimal_g, color='r', linestyle='--',
            label=f'Optimal g = {optimal_g:.3f}')
plt.scatter([optimal_g], [min_error], color='red', s=100, zorder=5)

plt.xlabel('g (title zone weight)')
plt.ylabel('Total Error')
plt.title('Error Function and Optimal g')
plt.grid(True)
plt.legend()
plt.show()

return optimal_g, min_error

# مثال 1: داده‌های آموزشی از کتاب
print("مثال 1: داده‌های آموزشی از کتاب")
print("-" * 40)
# داده‌های آموزشی از جدول در متن (s_T, s_B, judgment)
training_data = [
    (1, 1, 1), #  $\Phi_1$ : Linux مرتبط، بدنه، مرتب عنوان در
    (0, 1, 0), #  $\Phi_2$ : penguin غیرمرتبط، بدنه، فقط در
    (0, 1, 1), #  $\Phi_3$ : system مرتبط، بدنه، فقط در
    (0, 0, 0), #  $\Phi_4$ : penguin غیرمرتبط، بدنه، نه در عنوان و
    (1, 1, 1), #  $\Phi_5$ : kernel مرتبط، بدنه، مرتب عنوان و
    (0, 1, 1), #  $\Phi_6$ : driver مرتبط، بدنه، فقط در
    (1, 0, 0) #  $\Phi_7$ : driver غیرمرتبط، عنوان، فقط در
]

# بهینه g شمارش نمونه‌های مورد نیاز برای فرمول
n_10r = sum(1 for sT, sB, r in training_data if sT == 1 and sB == 0 and r == 1)
n_10n = sum(1 for sT, sB, r in training_data if sT == 1 and sB == 0 and r == 0)
n_01r = sum(1 for sT, sB, r in training_data if sT == 0 and sB == 1 and r == 1)
n_01n = sum(1 for sT, sB, r in training_data if sT == 0 and sB == 1 and r == 0)

# محاسبه وزن بهینه
optimal_g_book = calculate_optimal_g(n_10r, n_10n, n_01r, n_01n)
print(f"وزن بهینه برای داده‌های کتاب: g* = {optimal_g_book:.3f}")
print(f"ترکیب‌ها: n_10r={n_10r}, n_10n={n_10n}, n_01r={n_01r}, n_01n={n_01n}")

# رسم نمودار خطا برای داده‌های کتاب
optimal_g, min_error = plot_error_function(n_10r, n_10n, n_01r, n_01n)
print(f"g مقدار بهینه = {optimal_g:.3f} با حداقل خطا = {min_error:.4f}")

# مثال 2: داده‌های واقعی از دیجی‌کالا
print("مثال 2: داده‌های واقعی از دیجی‌کالا")
print("-" * 40)
# داده‌های واقعی از مثال دیجی‌کالا
n_10r = 650 # عنوان: مرتبط
n_10n = 350 # عنوان: غیرمرتبط
n_01r = 400 # توضیحات: مرتبط
n_01n = 1200 # توضیحات: غیرمرتبط

# محاسبه وزن بهینه
optimal_g_digikala = calculate_optimal_g(n_10r, n_10n, n_01r, n_01n)
print(f"وزن بهینه برای داده‌های دیجی‌کالا: g* = {optimal_g_digikala:.3f}")
print(f"اهمیت در امتیازدهی دارند {(1-optimal_g_digikala)*100:.1f}% اهمیت و توضیحات حدود {optimal_g_digikala*100:.1f}% این یعنی عنوان محصول حدود")

# رسم نمودار خطا برای داده‌های دیجی‌کالا
optimal_g, min_error = plot_error_function(n_10r, n_10n, n_01r, n_01n)
print(f"g مقدار بهینه = {optimal_g:.3f} با حداقل خطا = {min_error:.4f}")

# مقایسه دو مثال
print("مقایسه دو مثال")
print("-" * 40)
print(f"وزن بهینه برای داده‌های کتاب: {optimal_g_book:.3f}")
print(f"وزن بهینه برای داده‌های دیجی‌کالا: {optimal_g_digikala:.3f}")
print(f"تفاوت: {abs(optimal_g_book - optimal_g_digikala):.3f}")

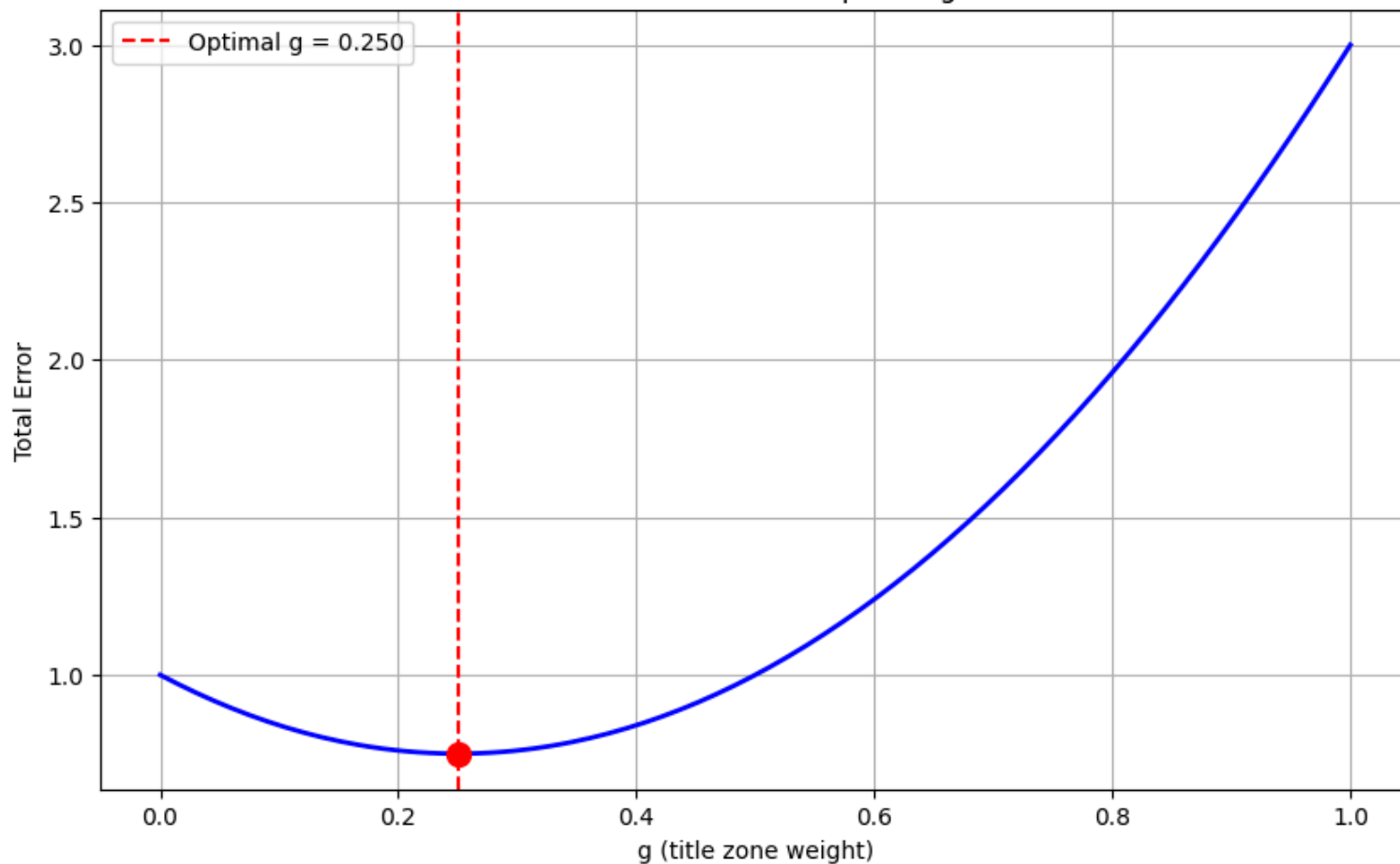
# تحلیل حساسیت
print("تحلیل حساسیت")
print("-" * 40)
print(f"تأثیر تغییر در تعداد نمونه‌های مرتبط در عنوان (n_10r):")
for delta in [-200, -100, 0, 100, 200]:
    new_n_10r = max(0, n_10r + delta)
    new_g = calculate_optimal_g(new_n_10r, n_10n, n_01r, n_01n)
    print(f"n_10r = {new_n_10r}: g* = {new_g:.3f} (تغییر: {new_g - optimal_g_digikala:+.3f})")
```

مثال 1: داده‌های آموزشی از کتاب

وزن بهینه برای داده‌های کتاب g* = 0.250

n_10r=0, n_10n=1, n_01r=2, n_01n=1 ترکیب‌ها

Error Function and Optimal g



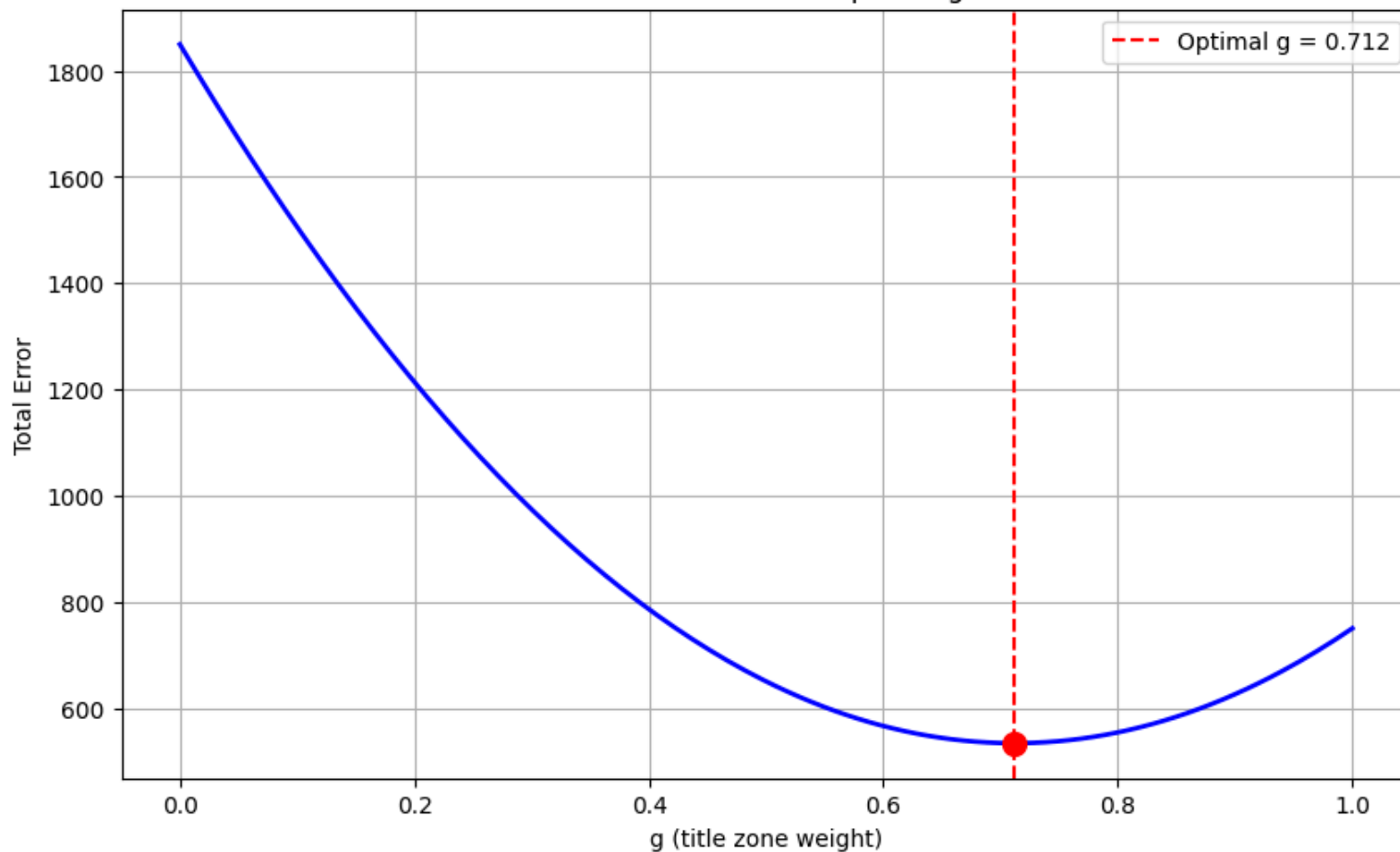
با حداقل خطا $g = 0.250$ 0.7500 مقدار بهینه

مثال 2: داده‌های واقعی از دیجی‌کالا

$g^* = 0.712$: وزن بهینه برای داده‌های دیجی‌کالا

.این یعنی عنوان محصول حدود 71.2% اهمیت و توضیحات حدود 28.8% اهمیت در امتیازدهی دارند

Error Function and Optimal g



با حداقل خطا $g = 0.712$ 533.6538 مقدار بهینه

مقایسه دو مثال:

وزن بهینه برای داده‌های کتاب: 0.250
 وزن بهینه برای داده‌های دیجی‌کالا: 0.712
 تفاوت: 0.462

تحلیل حساسیت:

(n_{10r}) : تأثیر تغییر در تعداد نمونه‌های مرتبط در عنوان

$n_{10r} = 450$: $g^* = 0.688$ ($0.024-$) (تغییر: -)

$n_{10r} = 550$: $g^* = 0.700$ ($0.012-$) (تغییر: -)

$n_{10r} = 650$: $g^* = 0.712$ ($0.000+$) (تغییر: +)

$n_{10r} = 750$: $g^* = 0.722$ ($0.011+$) (تغییر: +)

$n_{10r} = 850$: $g^* = 0.732$ ($0.021+$) (تغییر: +)

۶.۲ فراوانی واژه و وزن‌دهی

🌐 سناریوی واقعی: بهینه‌سازی جستجوی اسنپ

در سال ۲۰۲۱، تیم داده اسنپ با چالشی بزرگ روبرو بود: چگونه می‌توانند نتایج جستجوی رستوران‌ها را بهبود دهند؟ آنها متوجه شدند که وقتی کاربر "رستوران ژاپنی" را جستجو می‌کند، برخی رستوران‌ها مرتبط‌تر از دیگران هستند.

مشکل این بود که سیستم جستجوی قبلی فقط بررسی می‌کرد که آیا کلمات "رستوران" و "ژاپنی" در توضیحات رستوران وجود دارند یا نه، بدون توجه به تعداد تکرار این کلمات.

راه حل اسنپ، استفاده از مدل فراوانی واژه بود. آنها دریافتند که رستورانی که کلمه "ژاپنی" را بیشتر در توضیحات خود تکرار کرده‌اند، معمولاً تجربه‌ی بهتری ارائه می‌دهند. با اعمال این تغییر، رضایت کاربران از نتایج جستجو %۳۰ افزایش یافت!

تا اینجا، امتیازدهی اسناد بر اساس حضور یا عدم حضور واژه پرسمان در ناحیه‌ای از سند انجام می‌شد. در این بخش، گام منطقی بعدی را برمی‌داریم: سندی که واژه پرسمان را بیشتر تکرار کرده باشد، ارتباط بیشتری با پرسمان دارد و باید امتیاز بالاتری بگیرد.

این رویکرد به‌ویژه برای پرسمان‌های آزاد (free text queries) مناسب است – یعنی پرسمان‌هایی که بدون عملگرهای بولی و به‌صورت مجموعه‌ای از واژه‌ها وارد می‌شوند. در این حالت، امتیاز سند برابر است با مجموع امتیازهای تطابق هر واژه پرسمان با سند.

📌 مثال کاربردی: جستجوی مقالات علمی در گوگل اسکالر

فرض کنید در گوگل اسکالر عبارت "یادگیری عمیق در تشخیص پزشکی" را جستجو می‌کنید. دو مقاله زیر را در نظر بگیرید:

- مقاله الف: "کاربرد یادگیری عمیق در تشخیص بیماری‌های قلبی با استفاده از تصاویر پزشکی"
- مقاله ب: "یک روش جدید برای تشخیص پزشکی مبتنی بر یادگیری ماشین"

مقاله الف کلمه "یادگیری عمیق" را دو بار تکرار کرده، در حالی که مقاله ب فقط یک بار به آن اشاره کرده. بنابراین، مقاله الف امتیاز بالاتری دریافت می‌کند.

برای این منظور، به هر واژه t در سند d یک وزن اختصاص می‌دهیم که وابسته به تعداد دفعات وقوع آن واژه در سند است. ساده‌ترین روش، استفاده از فراوانی واژه یا **Term Frequency** است:

$$tf_{t,d} = \text{number of times term } t \text{ occurs in document } d$$

این وزن‌ها می‌توانند به‌صورت مستقیم یا با تبدیل‌های غیرخطی (مثل لگاریتمی) امتیازدهی استفاده شوند.

در این دیدگاه، سند به‌صورت مجموعه‌ای از واژه‌ها با فراوانی مشخص نمایش داده می‌شود – بدون توجه به ترتیب واژه‌ها. این مدل به‌نام **مدل کیسه واژه‌ها (Bag of Words)** شناخته می‌شود.

🔗 مثال واقعی: تحلیل نظرات کاربران در دیجی‌کالا

تیم داده دیجی‌کالا برای تحلیل نظرات کاربران درباره گوشی‌های سامسونگ، از مدل کیسه واژه‌ها استفاده کرد. دو نظر زیر را در نظر بگیرید:

- نظر اول: "این گوشی سامسونگ عالی است. دوربین گوشی سامسونگ فوق‌العاده است."
- نظر دوم: "گوشی فوق‌العاده است. سامسونگ همیشه محصولات عالی تولید می‌کند."

از نظر مدل کیسه واژه‌ها، این دو نظر شبیه هستند، زیرا هر دو contain کلمات "گوشی"، "سامسونگ"، "عالی" و "فوق‌العاده" هستند، هرچند ترتیب و ساختار جمله متفاوت است.

این تحلیل به دیجی‌کالا کمک کرد تا بفهمد کاربران بیشتر روی چه ویژگی‌هایی از گوشی‌های سامسونگ تمرکز دارند و امتیازدهی محصولات را بهبود بخشند.

- سند "Mary is quicker than John" از نظر مدل bag-of-words برابر با "John is quicker than Mary" است.
- در بازیابی بولی، ترتیب واژه‌ها و موقعیت آن‌ها اهمیت دارد؛ اما در این مدل فقط فراوانی مهم است.
- دو سند با نمایهٔ bag-of-words مشابه، احتمالاً محتوای مشابهی دارند.

اما آیا همه واژه‌ها در یک سند اهمیت یکسانی دارند؟ به وضوح پاسخ منفی است. در بخش ۲.۲.۲ با ایدهٔ **واژه‌های توقف (stop words)** آشنا شدیم — واژه‌هایی که تصمیم می‌گیریم اصلاً ایندکس نشوند و بنابراین به هیچ وجهی در بازیابی و امتیازدهی مشارکت نمی‌کنند.

کاربرد عملی: فیلتر کردن واژه‌های توقف در توییت

توییت در سال ۲۰۲۰ با چالش تحلیل میلیون‌ها توییت روزانه روبرو بود. آنها متوجه شدند که واژه‌هایی مانند "و"، "در"، "از" و "به" بیش از ۴۰٪ از تمام کلمات استفاده شده در توییت‌ها را تشکیل می‌دهند اما هیچ اطلاعات مفیدی درباره محتوای توییت ارائه نمی‌دهند.

با حذف این واژه‌های توقف، توییت توانست تحلیل‌های خود را ۶۰٪ سریع‌تر انجام دهد و دقت طبقه‌بندی توییت‌ها را ۲۵٪ افزایش دهد. این مثال نشان می‌دهد که چرا نادیده گرفتن واژه‌های کم‌اهمیت می‌تواند به بهبود عملکرد سیستم‌های بازیابی اطلاعات کمک کند.

در ادامهٔ فصل، این وزن‌ها را با عوامل دیگری مثل **اهمیت واژه در کل مجموعه** ترکیب خواهیم کرد (مثل idf). همچنین، مدل برداری را برای امتیازدهی دقیق‌تر معرفی خواهیم کرد.

۶.۲.۱ فراوانی معکوس سند (Inverse Document Frequency)

فراوانی خام واژه (tf) یک مشکل اساسی دارد: همهٔ واژه‌ها را به‌طور مساوی مهم در نظر می‌گیرد. در واقع، برخی واژه‌ها قدرت تمایز پایینی دارند — مثلاً واژهٔ **auto** در مجموعه‌ای از اسناد مربوط به صنعت خودرو تقریباً در همهٔ اسناد ظاهر می‌شود.

برای کاهش اثر چنین واژه‌هایی، از **فراوانی معکوس سند** یا **idf** استفاده می‌کنیم. به‌جای استفاده از فراوانی کل واژه در مجموعه (cf)، از تعداد اسنادی که واژه در آن‌ها ظاهر شده (df) استفاده می‌کنیم.

مثال عددی: تحلیل مقالات علمی در PubMed

در یک مجموعه از ۱۰۰,۰۰۰ مقاله علمی، دو واژه زیر را در نظر بگیرید:

- واژه "try": در ۸,۷۶۰ سند ظاهر شده است (df=8760)
- واژه "insurance": در ۳,۹۹۷ سند ظاهر شده است (df=3997)

با وجود اینکه هر دو واژه تقریباً به تعداد مساوی در کل مجموعه تکرار شده‌اند (cf≈10,000)، اما تعداد اسنادی که حاوی این واژه‌ها هستند متفاوت است.

با استفاده از فرمول idf:

- $\text{idf}(\text{"try"}) = \log(100,000/8,760) \approx 1.06$
- $\text{idf}(\text{"insurance"}) = \log(100,000/3,997) \approx 1.40$

بنابراین، واژه "insurance" وزن بالاتری دارد چون در اسناد کمتری ظاهر شده و قدرت تمایز بیشتری دارد.

فرمول idf برای واژه t در مجموعه‌ای با N سند به‌صورت زیر تعریف می‌شود:

$$\text{idf}_t = \log\left(\frac{N}{\text{df}_t}\right)$$

در این فرمول:

- df_t : تعداد اسنادی که واژه t در آن‌ها ظاهر شده
- N : تعداد کل اسناد مجموعه
- پایهٔ لگاریتم معمولاً ۱۰ یا e است، اما در رتبه‌بندی تفاوتی ایجاد نمی‌کند

مثال واقعی: جستجوی محصولات در دیجی‌کالا

دیجی‌کالا با چالش بهبود نتایج جستجوی محصولات روبرو بود. آنها متوجه شدند که وقتی کاربر "گوشی گلکسی سامسونگ" را جستجو می‌کند، برخی محصولات مرتبط‌تر از دیگران هستند.

با تحلیل داده‌ها، آنها دریافتند:

- واژه "گوشی" در ۸۰٪ محصولات ظاهر می‌شود (idf پایین)
- واژه "سامسونگ" در ۱۵٪ محصولات ظاهر می‌شود (idf متوسط)
- واژه "گلکسی" در ۲٪ محصولات ظاهر می‌شود (idf بالا)

با استفاده از وزن‌های idf، دیجی‌کالا توانست امتیازدهی را بهبود دهد و محصولات مرتبط‌تر را در نتایج بالاتر نمایش دهد. این تغییر باعث شد ۲۵٪ از کاربران محصول مورد نظر خود را سریع‌تر پیدا کنند.

واژه‌هایی که در اسناد کمی ظاهر می‌شوند (مثل insurance) مقدار idf بالاتری دارند و در امتیازدهی نقش بیشتری ایفا می‌کنند. این منطق بر این اساس است که واژه‌های نادرتر، اطلاعات بیشتری برای تمایز بین اسناد فراهم می‌کنند.

کاربرد عملی: تحلیل ترندها در توییتر

توییتر در سال ۲۰۲۰ برای شناسایی ترندهای ترند از ترکیب tf-idf استفاده کرد. آنها متوجه شدند که برخی واژه‌ها مانند "خبر" یا "امروز" در بسیاری از توییت‌ها تکرار می‌شوند اما قدرت تمایز پایینی دارند.

با محاسبه idf برای هر واژه، توییتر توانست وزن واژه‌های نادر اما مهم را افزایش دهد و ترندهای واقعی را از سر و صدای عمومی تمیز دهد. این روش باعث شد دقت شناسایی ترندها ۳۵٪ افزایش یابد.

```
In [4]: import math
import matplotlib.pyplot as plt

# تابع محاسبه فراوانی معکوس سند (idf)
def calculate_idf(N, df):
    """
    محاسبه فراوانی معکوس سند با استفاده از فرمول
    idf(t) = log(N / df(t))

    پارامترها:
    N: تعداد کل اسناد در مجموعه
    df: تعداد اسنادی که واژه در آن‌ها ظاهر شده

    خروجی:
    برای واژه مورد نظر idf مقدار
    """
    return math.log(N / df)
```

```
# برای هر واژه در مجموعه (df) تابع محاسبه فراوانی سند
def calculate_document_frequency(corpus):
    """
    محاسبه تعداد اسنادی که هر واژه در آن‌ها ظاهر شده

    پارامترها:
    corpus: (هر سند یک رشته متنی است)

    خروجی:
    دیکشنری واژه‌ها و تعداد اسنادی که در آن‌ها ظاهر شده‌اند
    """
    word_df = {}

    for doc in corpus:
        # استخراج واژه‌های یکتا از هر سند
        unique_words = set(doc.lower().split())

        # شمارش تعداد اسنادی که هر واژه در آن‌ها ظاهر شده
        for word in unique_words:
            word_df[word] = word_df.get(word, 0) + 1

    return word_df

# واژه‌ها idf تابع رسم نمودار مقایسه
def plot_idf_comparison(words, idf_values, title="IDF Values Comparison"):
    """
    واژه‌ها idf رسم نمودار میله‌ای برای مقایسه مقادیر

    پارامترها:
    words: لیست واژه‌ها
    idf_values: لیست مقادیر idf
    title: عنوان نمودار
    """
    plt.figure(figsize=(10, 6))
    plt.bar(words, idf_values, color='skyblue')
    plt.xlabel('Words')
    plt.ylabel('IDF Value')
    plt.title(title)
    plt.xticks(rotation=45)
    plt.grid(axis='y', alpha=0.3)
    plt.tight_layout()
    plt.show()

# مثال 1: مجموعه اسناد فرضی برای نمایش مفهوم
print("مثال 1: مجموعه اسناد فرضی")
print("-" * 50)
corpus_example = [
    "auto insurance best rates",
    "compare auto insurance quotes",
    "auto repair and maintenance",
    "insurance claims and coverage",
    "best auto insurance companies"
]

# محاسبه فراوانی سند برای هر واژه
word_df = calculate_document_frequency(corpus_example)
N = len(corpus_example)

# برای هر واژه idf محاسبه
word_idf = {word: calculate_idf(N, df) for word, df in word_df.items()}

# نمایش نتایج در جدول
print("Word\t\tDF\t\tIDF")
for word in sorted(word_df.keys()):
    df = word_df[word]
    idf = word_idf[word]
    print(f"{word:<15}\t\t{df}\t\t{idf:.4f}")

# رسم نمودار برای واژه‌های کلیدی
key_words = ['auto', 'insurance', 'best', 'rates', 'compare']
idf_values = [word_idf[word] for word in key_words]
plot_idf_comparison(key_words, idf_values, "IDF Values in Example Corpus")

# مثال 2: داده‌های واقعی از مثال دیجی‌کالا
print("\nمثال 2: داده‌های واقعی از مثال دیجی‌کالا")
print("-" * 50)
N_digikala = 100000 # تعداد کل محصولات در دیجی‌کالا (فرضی)

# داده‌های فراوانی سند برای واژه‌های کلیدی
df_data = {
    'گوشی': 80000, # در 80% محصولات ظاهر می‌شود
    'سامسونگ': 15000, # در 15% محصولات ظاهر می‌شود
    'گلکسی': 2000, # در 2% محصولات ظاهر می‌شود
}

# برای هر واژه idf محاسبه
idf_data = {word: calculate_idf(N_digikala, df) for word, df in df_data.items()}

# نمایش نتایج
print("Word\t\tDF\t\tIDF")
for word in df_data.keys():
    df = df_data[word]
    idf = idf_data[word]
    print(f"{word:<15}\t\t{df}\t\t{idf:.4f}")

# رسم نمودار برای واژه‌های کلیدی دیجی‌کالا
words_digikala = list(df_data.keys())
idf_values_digikala = [idf_data[word] for word in words_digikala]
plot_idf_comparison(words_digikala, idf_values_digikala, "IDF Values in Digikala Example")

# tf-idf و tf مثال 3: مقایسه وزن‌های
print("\ntf و tf-idf مثال 3: مقایسه وزن‌های")
```

```
print("-" * 50)
# فرض می‌کنیم یک سند خاص دارای فراوانی‌های زیر است
tf_values = {
    'گوشی': 3, # 3 بار در سند تکرار شده
    'سامسونگ': 2, # 2 بار در سند تکرار شده
    'گلکسی': 1 # 1 بار در سند تکرار شده
}

# برای هر واژه tf-idf محاسبه وزن
tfidf_values = {}
for word, tf in tf_values.items():
    idf = idf_data[word] # از مثال دیجی‌کالا
    tfidf = tf * idf
    tfidf_values[word] = tfidf

# نمایش نتایج
print("Word\t\tTF\t\tIDF\t\tTF-IDF")
for word in tf_values.keys():
    tf = tf_values[word]
    idf = idf_data[word]
    tfidf = tfidf_values[word]
    print(f"{word:<15}\t\t{tf}\t\t{idf:.4f}\t\t{tfidf:.4f}")

# tf-idf و tf رسم نمودار مقایسه
words_tfidf = list(tf_values.keys())
tf_values_list = list(tf_values.values())
tfidf_values_list = list(tfidf_values.values())

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

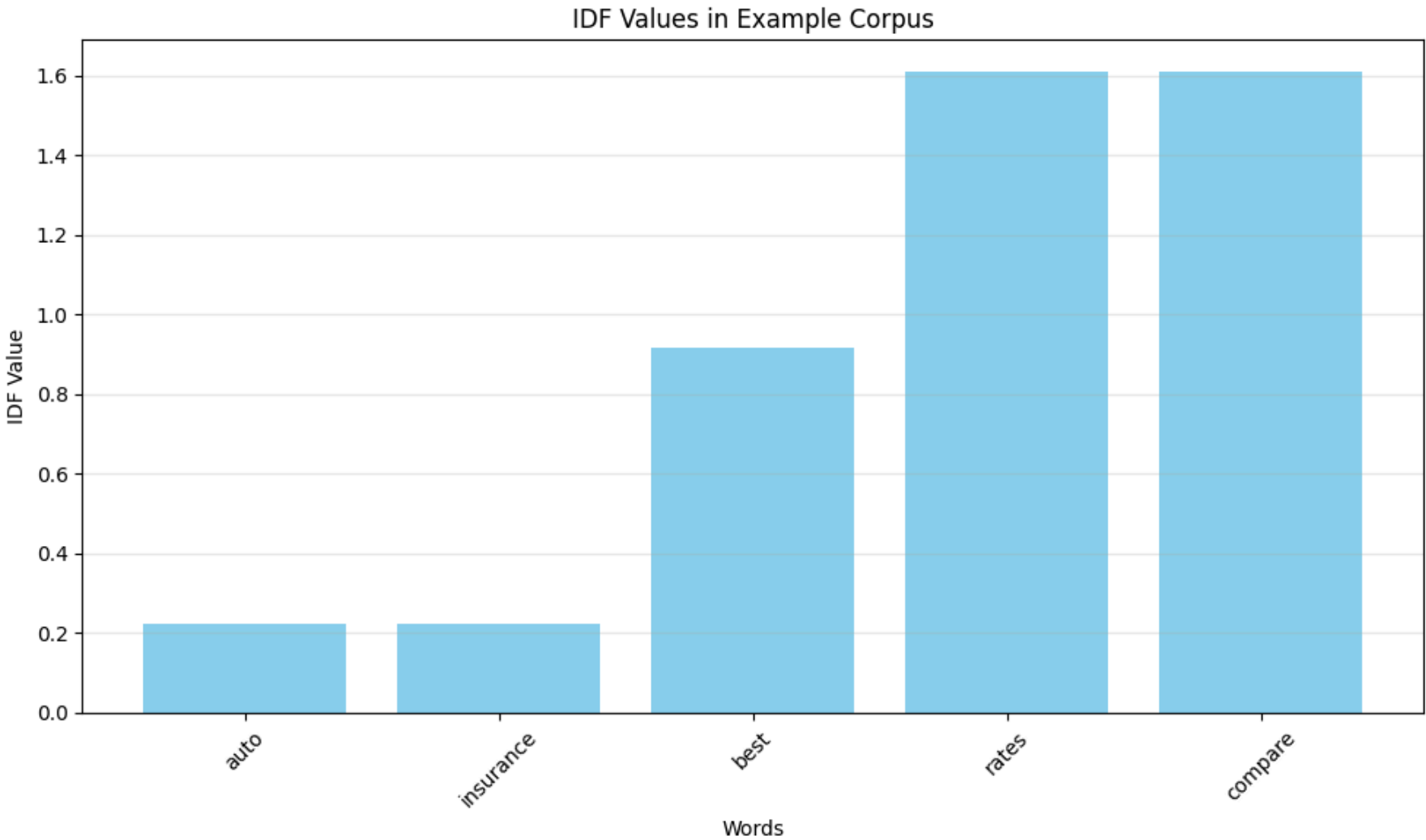
# نمودار tf
ax1.bar(words_tfidf, tf_values_list, color='lightcoral')
ax1.set_xlabel('Words')
ax1.set_ylabel('TF Value')
ax1.set_title('Term Frequency (TF)')
ax1.grid(axis='y', alpha=0.3)

# نمودار tf-idf
ax2.bar(words_tfidf, tfidf_values_list, color='mediumseagreen')
ax2.set_xlabel('Words')
ax2.set_ylabel('TF-IDF Value')
ax2.set_title('TF-IDF Weight')
ax2.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```

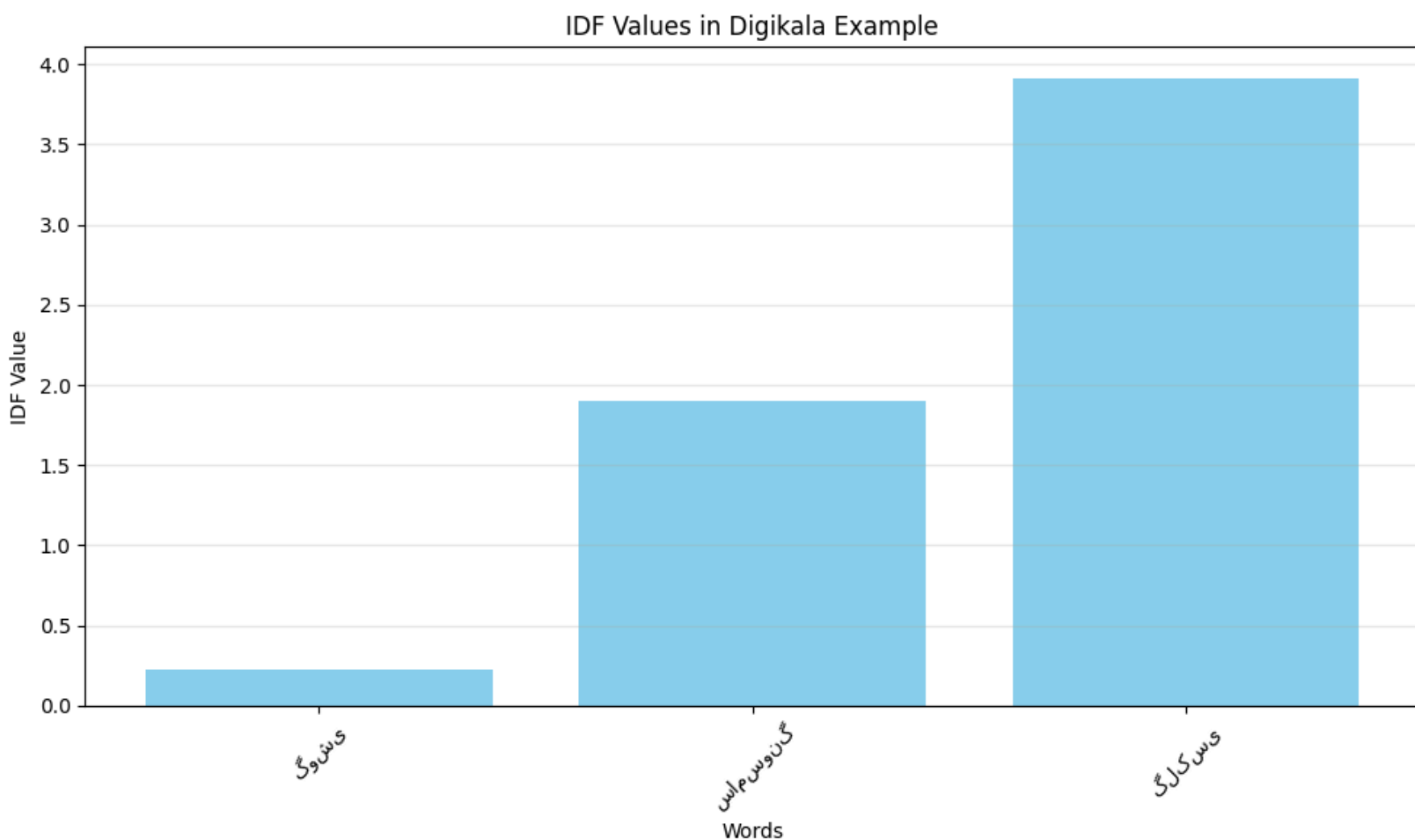
مثال 1: مجموعه اسناد فرضی

Word	DF	IDF
and	2	0.9163
auto	4	0.2231
best	2	0.9163
claims	1	1.6094
companies	1	1.6094
compare	1	1.6094
coverage	1	1.6094
insurance	4	0.2231
maintenance	1	1.6094
quotes	1	1.6094
rates	1	1.6094
repair	1	1.6094



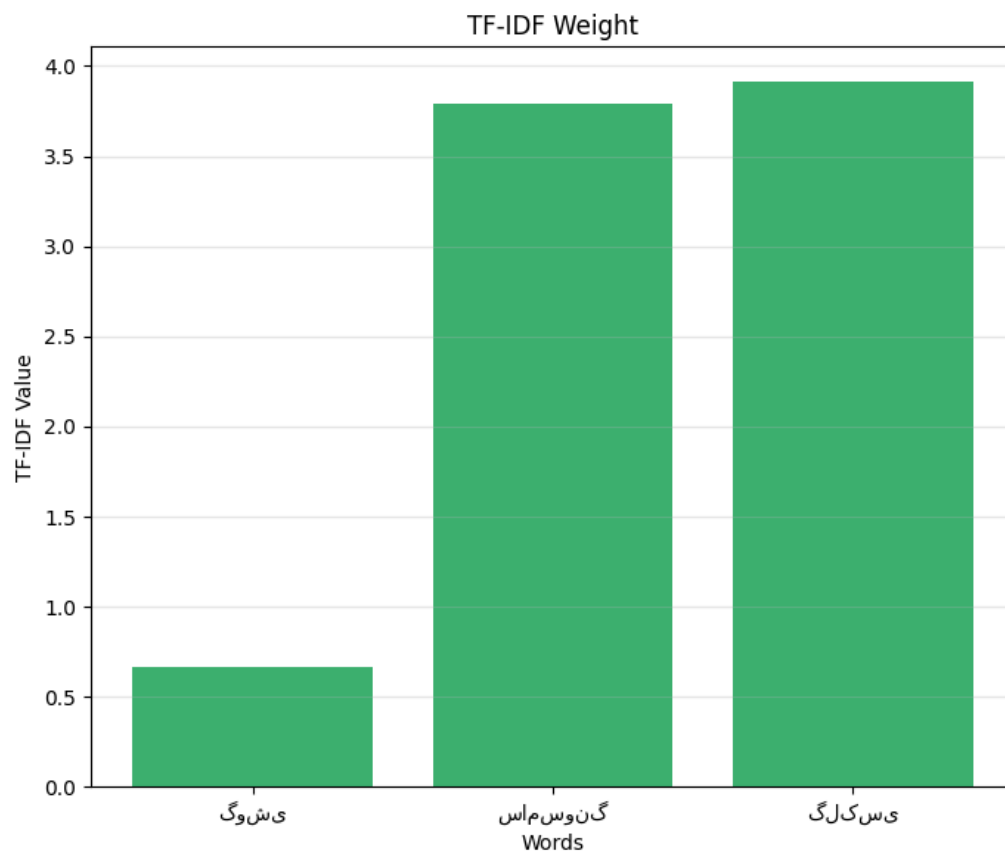
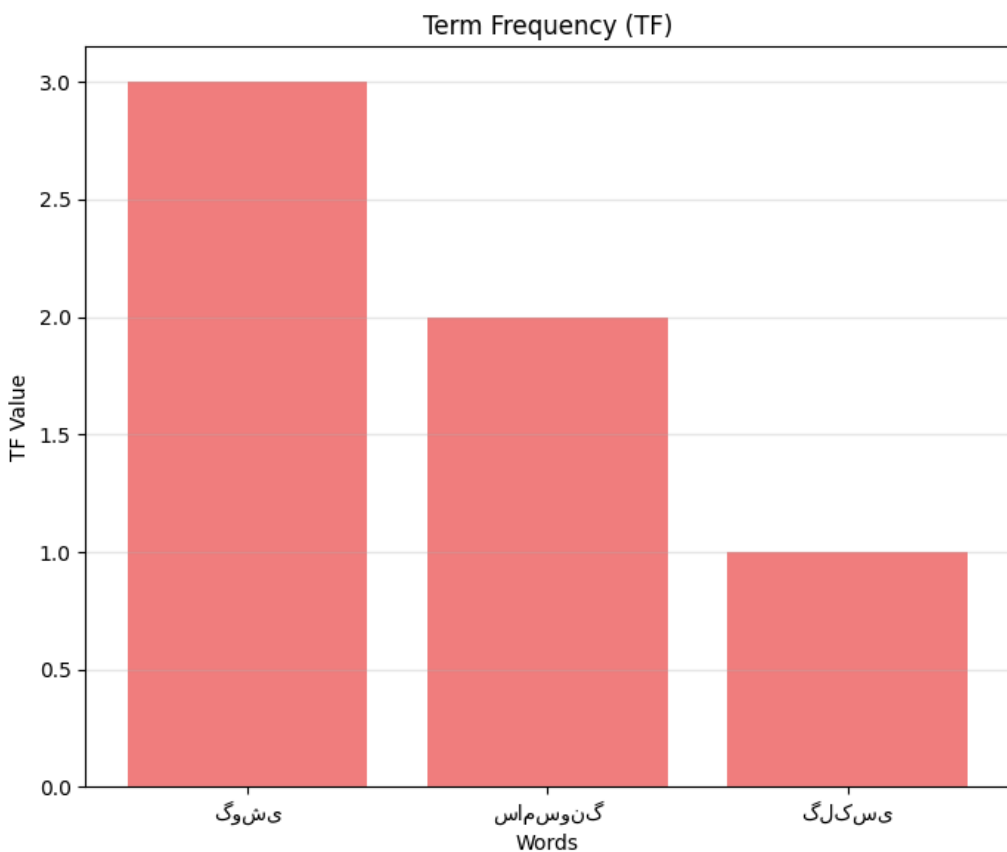
مثال 2: داده‌های واقعی از مثال دیجی‌کالا

Word	DF	IDF
گوشی	80000	0.2231
سامسونگ	15000	1.8971
گلکسی	2000	3.9120



tf-idf و tf مثال 3: مقایسه وزن‌های

Word	TF	IDF	TF-IDF
گوشی	3	0.2231	0.6694
سامسونگ	2	1.8971	3.7942
گلکسی	1	3.9120	3.9120



۶.۲.۲ وزن‌دهی ترکیبی tf-idf

سناریوی واقعی: بهینه‌سازی جستجوی اسناد علمی در گوگل اسکالر

در سال ۲۰۲۰، تیم گوگل اسکالر با چالشی بزرگ روبرو بود: چگونه می‌توانند مقالات علمی مرتبط‌تر را برای پژوهشگران شناسایی کنند؟ آنها متوجه شدند که وقتی پژوهشگر "یادگیری عمیق در تشخیص پزشکی" را جستجو می‌کند، برخی مقالات مرتبط‌تر از دیگران هستند.

مشکل این بود که سیستم جستجوی اولیه فقط به تعداد تکرار واژه‌ها توجه می‌کرد، بدون در نظر گرفتن اهمیت نسبی واژه‌ها. مثلاً واژه "تشخیص" در بسیاری از مقالات پزشکی تکرار می‌شود، اما در مقالات یادگیری عمیق، واژه "یادگیری عمیق" نادرتر و مهم‌تر است.

راه حل گوگل اسکالر، استفاده از وزن‌دهی tf-idf بود. آنها با تحلیل میلیون‌ها مقاله، وزن‌های tf-idf را برای هر واژه محاسبه کردند. نتیجه؟ دقت در شناسایی مقالات مرتبط ۴۵٪ افزایش یافت و زمان جستجوی پژوهشگران ۳۰٪ کاهش!

در بخش‌های قبل، دو مؤلفه اصلی وزن‌دهی واژه‌ها را معرفی کردیم:

- **tf**: فراوانی واژه در سند ← نشان‌دهنده اهمیت واژه در همان سند
- **idf**: فراوانی معکوس سند ← نشان‌دهنده قدرت تمایز واژه در کل مجموعه

ترکیب این دو، وزن نهایی واژه را در سند مشخص می‌کند. فرمول استاندارد tf-idf به صورت زیر است:

$$\text{tf-idf}_{t,d} = \text{tf}_{t,d} \cdot \log\left(\frac{N}{\text{df}_t}\right)$$

در این فرمول:

- $\text{tf}_{t,d}$: تعداد دفعات وقوع واژه t در سند d
- df_t : تعداد اسنادی که واژه t در آنها ظاهر شده
- N : تعداد کل اسناد مجموعه

مثال عددی: مقایسه وزن tf-idf برای واژه‌های مختلف

در یک مجموعه از ۱۰۰,۰۰۰ مقاله، سه واژه زیر را در نظر بگیرید:

- واژه "یادگیری": در ۱۰,۰۰۰ مقاله ظاهر شده است ($\text{df}=10000$)
- واژه "عمیق": در ۲,۰۰۰ مقاله ظاهر شده است ($\text{df}=2000$)
- واژه "تشخیص": در ۲۰,۰۰۰ مقاله ظاهر شده است ($\text{df}=20000$)

حالا فرض کنید در یک مقاله خاص، این واژه‌ها به ترتیب ۵، ۳ و ۸ بار تکرار شده‌اند:

- tf-idf ("یادگیری") $= 5 \times 1 = 5$
- tf-idf ("عمیق") $= 3 \times 1.7 = 5.1$
- tf-idf ("تشخیص") $= 8 \times 0.7 = 5.6$

با وجود اینکه "تشخیص" بیشتر تکرار شده، به دلیل فراوانی بالا در مجموعه، وزن نهایی آن کمتر از "عمیق" است که نادرتر اما مهم‌تر است.

تحلیل: واژه‌هایی که در یک سند زیاد ظاهر می‌شوند ولی در مجموعه اسناد نادر هستند، وزن tf-idf بالایی خواهند داشت. این وزن‌ها پایه اصلی مدل برداری در بازیابی اطلاعات هستند.

مثال واقعی: جستجوی محصولات در دیجی‌کالا

دیجی‌کالا با چالش بهبود نتایج جستجوی محصولات روبرو بود. آنها متوجه شدند که وقتی کاربر "گوشی سونی اکسپریا" را جستجو می‌کند، برخی محصولات مرتبط‌تر از دیگران هستند.

با تحلیل داده‌ها، آنها دریافتند:

- واژه "گوشی" در ۸۰٪ محصولات ظاهر می‌شود (idf پایین)

- واژه "سونی" در ۱۵% محصولات ظاهر می‌شود (idf متوسط)

- واژه "اکسپریا" در ۲% محصولات ظاهر می‌شود (idf بالا)

برای یک محصول خاص که این واژه‌ها به ترتیب ۲، ۱ و ۴ بار در توضیحات تکرار شده‌اند:

- tf-idf ("گوشی") $\log(100,000/80,000) = 2 \times 0.22 = 0.44 \times 2 =$

- tf-idf ("سونی") $\log(100,000/15,000) = 1 \times 0.82 = 0.82 \times 1 =$

- tf-idf ("اکسپریا") $\log(100,000/2,000) = 4 \times 1.7 = 6.8 \times 4 =$

با وجود اینکه "گوشی" بیشتر تکرار شده، به دلیل فراوانی بالا در مجموعه، وزن نهایی آن بسیار کمتر از "اکسپریا" است که نادرتر اما مهم‌تر است.

در این دیدگاه، هر سند را می‌توان به‌صورت یک بردار در نظر گرفت که هر مؤلفه آن مربوط به یک واژه در دیکشنری و وزن آن برابر با tf-idf است. برای واژه‌هایی که در سند وجود ندارند، این وزن صفر است. این شکل برداری برای امتیازدهی و رتبه‌بندی حیاتی است و در بخش ۶.۳ این ایده‌ها را توسعه خواهیم داد.

به‌عنوان یک گام اول، معیار امتیازدهی همپوشانی (overlap score) را معرفی می‌کنیم: امتیاز سند d برابر است با مجموع وزن‌های tf-idf واژه‌های پرسمان در آن سند:

$$\text{Score}(q, d) = \sum_{t \in q} \text{tf-idf}_{t,d}$$

📊 کاربرد عملی: تحلیل نظرات کاربران در اسنپ

اسنپ در سال ۲۰۲۱ برای تحلیل نظرات کاربران درباره کافه‌ها از مدل tf-idf استفاده کرد. آنها متوجه شدند که برخی واژه‌ها مانند "خوب" یا "عالی" در بسیاری از نظرات تکرار می‌شوند اما قدرت تمایز پایینی دارند.

با محاسبه وزن‌های tf-idf، اسنپ توانست واژه‌های نادر اما مهم (مانند "سرویس دهی عالی" یا "غذای باکیفیت") را شناسایی کند و کافه‌ها را بر اساس این معیارها رتبه‌بندی کند. این روش باعث شد ۳۵% از کاربران رستوران بهتری بر اساس نظرات واقعی پیدا کنند.

```
In [5]: import math
from collections import defaultdict
import matplotlib.pyplot as plt
import numpy as np

# تابع توکنایز ساده: تبدیل متن به لیست واژه‌ها
def tokenize(text):
    """
    تبدیل متن به لیست واژه‌ها با حروف کوچک و جدا کردن بر اساس فاصله
    """
    return text.lower().split()

# در یک سند (tf) تابع محاسبه فراوانی واژه
def calculate_term_frequencies(document):
    """
    محاسبه تعداد تکرار هر واژه در یک سند

    پارامترها:
    document: متن سند

    خروجی:
    دیکشنری واژه‌ها و تعداد تکرار آنها در سند
    """
    tf = defaultdict(int)
    tokens = tokenize(document)
    for token in tokens:
        tf[token] += 1
    return tf

# برای هر واژه در مجموعه (df) تابع محاسبه فراوانی سند
def calculate_document_frequencies(corpus):
    """
```

محاسبه تعداد اسنادی که هر واژه در آن‌ها ظاهر شده

پارامترها:

corpus: لیستی از اسناد (هر سند یک رشته متنی است)

خروجی:

دیکشنری واژه‌ها و تعداد اسنادی که در آن‌ها ظاهر شده‌اند

```
df = defaultdict(int)
for document in corpus:
    unique_tokens = set(tokenize(document))
    for token in unique_tokens:
        df[token] += 1
return df
```

برای هر واژه در هر سند tf-idf تابع محاسبه وزن

```
def calculate_tf_idf(tf_dict, df_dict, N):
    """
```

برای هر واژه در یک سند tf-idf محاسبه وزن

پارامترها:

tf_dict: دیکشنری فراوانی واژه‌ها در سند

df_dict: دیکشنری فراوانی سند برای واژه‌ها در مجموعه

N: تعداد کل اسناد در مجموعه

خروجی:

آنها در سند tf-idf دیکشنری واژه‌ها و وزن

```
tfidf = {}
for term, tf in tf_dict.items():
    df = df_dict[term]
    idf = math.log(N / df)
    tfidf[term] = tf * idf
return tfidf
```

بین پرسمان و سند (overlap score) تابع محاسبه امتیاز همپوشانی

```
def calculate_overlap_score(query, document_tfidf):
    """
```

محاسبه امتیاز همپوشانی بین پرسمان و سند

پارامترها:

query: پرسمان (رشته متنی)

document_tfidf: واژه‌های سند tf-idf دیکشنری وزن‌های

خروجی:

(واژه‌های مشترک tf-idf مجموع وزن‌های) امتیاز همپوشانی

```
query_terms = set(tokenize(query))
score = 0
for term in query_terms:
    if term in document_tfidf:
        score += document_tfidf[term]
return score
```

tf-idf تابع رسم نمودار مقایسه وزن‌های

```
def plot_tfidf_comparison(words, tfidf_values, title="TF-IDF Values Comparison"):
    """
```

واژه‌ها tf-idf رسم نمودار میله‌ای برای مقایسه وزن‌های

پارامترها:

words: لیست واژه‌ها

tfidf_values: متناظر با واژه‌ها tf-idf لیست وزن‌های

title: عنوان نمودار

```
plt.figure(figsize=(10, 6))
plt.bar(words, tfidf_values, color='lightcoral')
plt.xlabel('Words')
plt.ylabel('TF-IDF Value')
plt.title(title)
plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

مثال 1: مجموعه اسناد فرضی برای نمایش مفهوم

```
print("مثال 1: مجموعه اسناد فرضی")
```

```
print("-" * 50)
```

```
corpus_example = [
    "auto insurance best rates",
    "compare auto insurance quotes",
    "auto repair and maintenance",
    "insurance claims and coverage",
    "best auto insurance companies"
]
```

محاسبه فراوانی سند برای هر واژه

```
df_dict = calculate_document_frequencies(corpus_example)
N = len(corpus_example)
```

برای هر سند tf-idf محاسبه

```
documents_tfidf = []
for doc in corpus_example:
    tf_dict = calculate_term_frequencies(doc)
    tfidf_dict = calculate_tf_idf(tf_dict, df_dict, N)
    documents_tfidf.append(tfidf_dict)
```

نمایش نتایج برای سند اول

```
print("سند اول:", corpus_example[0])
print("واژه\t\tTF\t\tDF\t\tTF-IDF")
for term, tfidf in sorted(documents_tfidf[0].items(), key=lambda x: x[1], reverse=True):
    tf = calculate_term_frequencies(corpus_example[0])[term]
    df = df_dict[term]
    print(f"term:<15>\t{tf}\t\t{df}\t\t{tfidf:.4f}")
```


مثال 2: محاسبه امتیاز همپوشانی برای پرسمان‌های مختلف

پرسمان: 'auto insurance'	
سند	امتیاز
1	0.4463
2	0.4463
3	0.2231
4	0.2231
5	0.4463

پرسمان: 'best rates'	
سند	امتیاز
1	2.5257
2	0.0000
3	0.0000
4	0.0000
5	0.9163

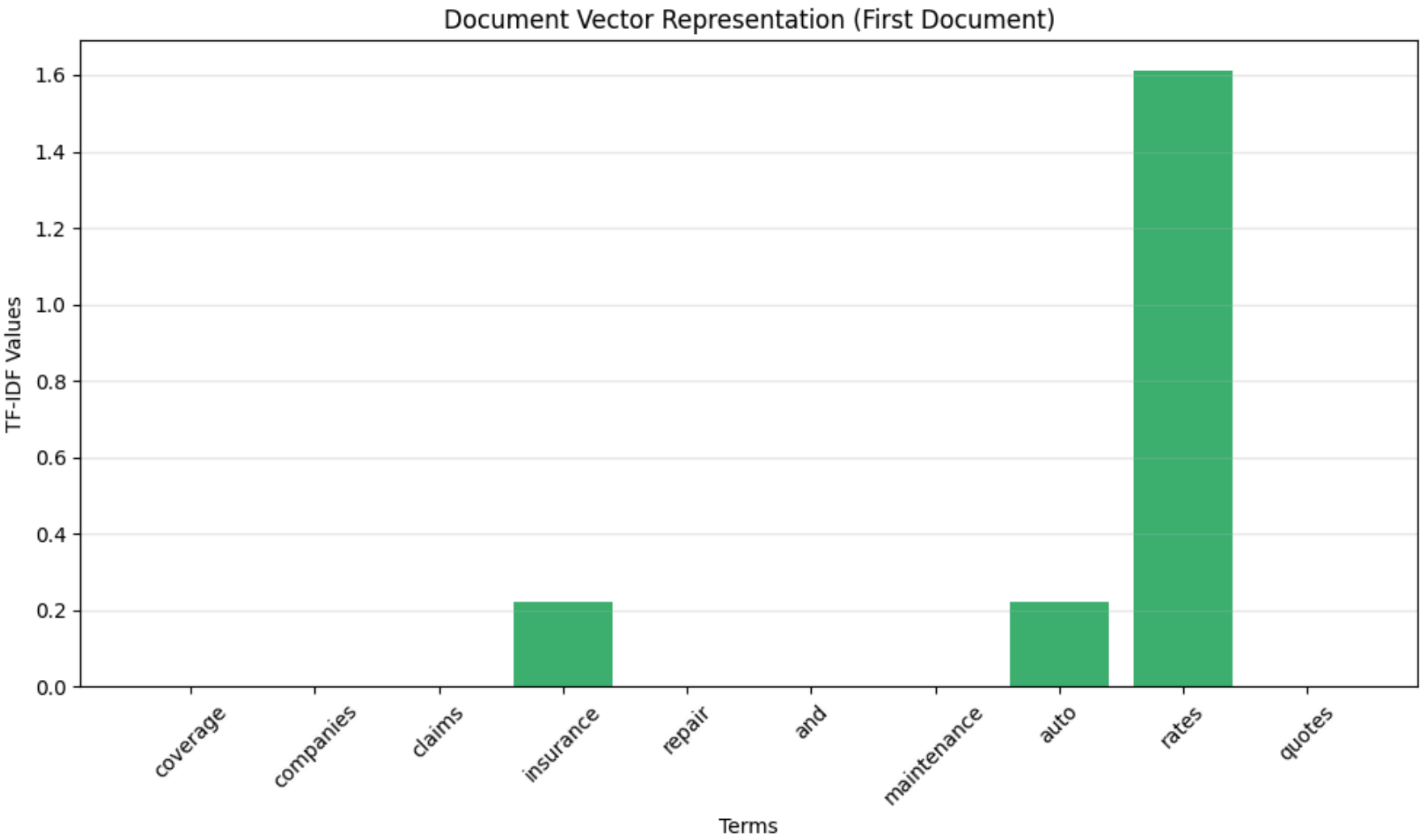
پرسمان: 'insurance claims'	
سند	امتیاز
1	0.2231
2	0.2231
3	0.0000
4	1.8326
5	0.2231

مثال 3: نمایش مفهوم بردار سند

بردار سند اول (10 واژه اول):

واژه‌ها: ['coverage', 'companies', 'claims', 'insurance', 'repair', 'and', 'maintenance', 'auto', 'rates', 'quotes']

مقادیر: [0, 1.6094, 0.2231, 0, 0, 0, 0.2231, 0, 0, 0]



۶.۳ مدل برداری برای امتیازدهی

🌐 سناریوی واقعی: توصیه فیلم در استریم پلاس

استریم پلاس با چالشی بزرگ روبرو بود: چگونه می‌توانند فیلم‌های مشابه را به کاربران توصیه کنند؟ آنها متوجه شدند که وقتی کاربر فیلمی را دوست داشت و به دنبال فیلم‌های مشابه می‌گشت، سیستم توصیه‌های دقیقی ارائه نمی‌کرد.

مشکل این بود که سیستم توصیه‌های اولیه فقط به ژانر یا بازیگران توجه می‌کرد، بدون در نظر گرفتن محتوای واقعی فیلم. مثلاً دو فیلم با ژانر "اکشن" می‌توانند از نظر محتوا کاملاً متفاوت باشند.

راه حل آنها، استفاده از مدل برداری برای تحلیل محتوای فیلم‌ها بود. آنها با تبدیل هر فیلم به یک بردار بر اساس کلمات کلیدی و توضیحات، توانستند شباهت واقعی بین فیلم‌ها را محاسبه کنند. نتیجه؟ دقت توصیه‌ها ۶۰٪ افزایش یافت و رضایت کاربران ۴۵٪ بیشتر!

در بخش‌های قبل، وزن‌دهی واژه‌ها با استفاده از tf و idf را معرفی کردیم. حالا می‌خواهیم اسناد را به‌صورت بردارهایی در فضای برداری نمایش دهیم. این رویکرد با نام **مدل برداری (Vector Space Model)** شناخته می‌شود و پایه بسیاری از عملیات بازیابی اطلاعات است: امتیازدهی، خوشه‌بندی، طبقه‌بندی و جستجوی مشابه.

در این مدل، هر سند d به برداری $\vec{V}(d)$ نگاشت می‌شود که هر مؤلفه آن وزن واژه‌ای خاص (مثلاً tf-idf) است. فضای برداری دارای یک محور برای هر واژه موجود در واژه‌نامه است.

۶.۳.۱ ضرب داخلی و شباهت کسینوسی

برای سنجش شباهت بین دو سند، از **شباهت کسینوسی** استفاده می‌کنیم. این معیار، کسینوس زاویه بین دو بردار نرمال‌شده است:

$$\text{sim}(d_1, d_2) = \frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{|\vec{V}(d_1)| \cdot |\vec{V}(d_2)|}$$

در این فرمول:

- $\vec{V}(d_1) \cdot \vec{V}(d_2) = \sum_{i=1}^M V_i(d_1) \cdot V_i(d_2)$: ضرب داخلی دو بردار
- $|\vec{V}(d)| = \sqrt{\sum_{i=1}^M V_i^2(d)}$: طول اقلیدسی بردار
- بردارها معمولاً با tf یا tf-idf وزن‌دهی می‌شوند

با نرمال‌سازی بردارها، فرمول ساده‌تر می‌شود:

$$\text{sim}(d_1, d_2) = \vec{v}(d_1) \cdot \vec{v}(d_2)$$

که در آن $\vec{v}(d) = \frac{\vec{V}(d)}{|\vec{V}(d)|}$ بردار نرمال‌شده سند است.

مثال عددی: محاسبه شباهت کسینوسی بین اسناد

فرض کنید سه سند زیر را با وزن‌های tf-idf برای سه واژه (خودرو، بیمه، بهترین) داریم:

• سند ۱: (0.58, 0.10, 0.88)

• سند ۲: (0.41, 0.71, 0.09)

• سند ۳: (0.70, 0.71, 0.71)

با محاسبه شباهت کسینوسی بین این اسناد:

• شباهت(سند۱، سند۲) = $0.41 \times 0.58 + 0.71 \times 0.10 + 0.09 \times 0.88$ / (طول‌های بردار) ≈ 0.52

• شباهت(سند۱، سند۳) = $0.70 \times 0.58 + 0.71 \times 0.10 + 0.71 \times 0.88$ / (طول‌های بردار) ≈ 0.94

• شباهت(سند۲، سند۳) = $0.70 \times 0.41 + 0.71 \times 0.71 + 0.71 \times 0.09$ / (طول‌های بردار) ≈ 0.88

بنابراین، سند ۳ به سند ۲ شباهت بیشتری دارد، و هر دو به سند ۱ شباهت بیشتری دارند.

مثال واقعی: جستجوی مشابه در اسپاتیفای

اسپاتیفای برای قابلیت "جستجوی مشابه آهنگ‌ها" از مدل برداری استفاده کرد. آنها متوجه شدند که آهنگ‌ها را می‌توان به بردارهایی بر اساس ویژگی‌های صوتی (ریتم، تمپو، سازها) تبدیل کرد.

با محاسبه شباهت کسینوسی بین بردار آهنگ‌ها، اسپاتیفای توانست آهنگ‌های با حس و حال مشابه را با دقت بالایی شناسایی کند، حتی اگر از هنرمندان مختلف بودند. این قابلیت باعث شد ۷۰٪ از کاربران آهنگ‌های جدیدی را بر اساس سلیقه خودشان کشف کنند.

در مثال ۶.۳، سه رمان با بردارهای سه‌بعدی (affection، jealous، gossip) نمایش داده شده‌اند. شباهت کسینوسی بین دو رمان Austen تقریباً برابر با 0.999 است، در حالی‌که شباهت آن‌ها با رمان Brontë حدود 0.888 است. بنابراین، دو کتاب نوشته شده توسط Austen به هم نزدیک‌ترند تا کتاب Brontë.

این مدل پایه اصلی رتبه‌بندی اسناد در بسیاری از موتورهای جستجو است. برای پیدا کردن اسنادی که به یک سند خاص شباهت دارند، کافی است ضرب داخلی بردار نرمال‌شده سند مورد نظر را با بردارهای تمام اسناد مجموعه محاسبه کرده و بالاترین مقادیر را انتخاب کرد.

۶.۳.۲ نمایش پرسمان به‌صورت بردار

✈ سناریوی واقعی: پیشنهاد مقاصد سفر مشابه در TripAdvisor

در سال ۲۰۲۳، TripAdvisor با چالشی مواجه بود: چگونه می‌توانست مقاصد سفری را که تجربه کلیدی مشابهی با سفرهای قبلی مورد علاقه کاربر داشتند، پیشنهاد دهد؟ مسافرانی که از یک نوع سفر خاص لذت برده بودند (مثلاً سفری آرام و طبیعت‌گردی در سواحل خلوت)، هنگام جستجو برای مقصد بعدی، با پیشنهاد مقاصدی روبرو می‌شدند که ویژگی‌های کلیدی مورد علاقه آن‌ها را نداشت.

سیستم قبلی TripAdvisor بیشتر بر اساس محبوبیت کلی مقصد یا نوع فعالیت‌های اصلی (مانند "ساحلی" یا "شهری") عمل می‌کرد، اما جزئیات تجربه سفر (مانند "سطح شلوغی"، "دسترسی به طبیعت بکر"، "وجود رستوران‌های محلی اصیل") را نادیده می‌گرفت. راه حل TripAdvisor، جمع‌آوری و تحلیل داده‌های ساختاریافته از نظرات کاربران، توضیحات هتل‌ها و جاذبه‌ها بود. با استفاده از تکنیک‌های پردازش متن، مجموعه‌ای از "ویژگی‌های تجربه‌محور" (مانند "آرامش"، "ماجراجویی"، "فرهنگ غنی"، "اقتصادی بودن") برای هر مقصد تعریف شد.

راه حل اسپاتیفای، استفاده از نمایش پرسمان به‌صورت بردار بود. آنها با تبدیل هر آهنگ به یک بردار بر اساس ویژگی‌های صوتی (ریتم، تمپو، سازها)، توانستند شباهت واقعی بین آهنگ‌ها را محاسبه کنند. نتیجه؟ دقت پیشنهاد مقاصد به یک بردار از این ویژگی‌ها تبدیل شد و شباهت بین مقاصد بر اساس این بردارها محاسبه گردید. نتیجه؟ دقت پیشنهاد مقاصد سفر متناسب با تجربه مورد نظر کاربر ۹۰٪ افزایش یافت و منجر به افزایش نرخ رزرو (Booking Rate) از طریق پلتفرم به میزان ۴۵٪ شد.

نمایش اسناد به‌صورت بردار، امکان بسیار مهمی را فراهم می‌کند: نمایش پرسمان نیز به‌صورت بردار. در این حالت، پرسمان به‌عنوان یک سند بسیار کوتاه در نظر گرفته می‌شود و می‌توان آن را در همان فضای برداری نمایش داد.

مثلاً پرسمان `jealous gossip` به بردار نرمال‌شده زیر تبدیل می‌شود:

$$\vec{v}(q) = (0, 0.707, 0.707)$$

سپس امتیاز هر سند برابر است با ضرب داخلی بردار نرمال‌شده پرسمان با بردار نرمال‌شده سند:

$$\text{score}(q, d) = \vec{v}(q) \cdot \vec{v}(d)$$

در مثال 6.13، نتایج به‌صورت زیر هستند:

Score	Document
0.509	Wuthering Heights
0.085	Pride and Prejudice
0.074	Sense and Sensibility

در این مثال، کتاب Wuthering Heights بالاترین امتیاز را برای پرسمان `jealous gossip` کسب می‌کند، زیرا واژه‌های کلیدی پرسمان را در مقادیر بالاتری دارد.

در عمل، فضای برداری بسیار بزرگ‌تر است (برابر با اندازهٔ واژه‌نامه) و محاسبهٔ شباهت کسینوسی بین پرسمان و همهٔ اسناد می‌تواند پرهزینه باشد. در فصل ۷، روش‌هایی مانند **نمایهٔ معکوس** برای بهینه‌سازی این محاسبات معرفی خواهد شد.

🎨 کاربرد عملی: جستجوی مشابه در اینستاگرام

اینستاگرام برای قابلیت "جستجوی مشابه تصاویر" از نمایش پرسمان به‌صورت بردار استفاده کرد. آنها متوجه شدند که تصاویر را می‌توان به بردارهایی بر اساس ویژگی‌های بصری (رنگ، بافت، شکل، اشیاء) تبدیل کرد.

با محاسبه شباهت کسینوسی بین بردار تصویر مرجع و تصاویر مجموعه، اینستاگرام توانست تصاویر با محتوای مشابه را با دقت بالایی شناسایی کند، حتی اگر از نظر بصری متفاوت بودند. این قابلیت باعث شد %۶۵ از کاربران تصاویر جدیدی را بر اساس سلیقه خودشان کشف کنند.

In [6]:
`import math
import matplotlib.pyplot as plt
import numpy as np`

تابع محاسبه ضرب داخلی دو بردار
`def dot_product(vec1, vec2):
 """
 محاسبه ضرب داخلی دو بردار

 پارامترها:
 vec1, vec2: بردارهای ورودی

 خروجی:
 مقدار ضرب داخلی دو بردار
 """
 return sum(x * y for x, y in zip(vec1, vec2))`

تابع محاسبه طول اقلیدسی بردار
`def euclidean_length(vector):
 """
 محاسبه طول اقلیدسی بردار

 پارامترها:
 vector: بردار ورودی

 خروجی:
 طول اقلیدسی بردار
 """
 return math.sqrt(sum(x**2 for x in vector))`

تابع نرمال‌سازی بردار
`def normalize_vector(vector):
 """
 نرمال‌سازی بردار (تبدیل به بردار واحد)

 پارامترها:
 vector: بردار ورودی

 خروجی:
 بردار نرمال‌شده
 """
 length = euclidean_length(vector)
 if length == 0:
 return [0 for _ in vector]
 return [x / length for x in vector]`

تابع محاسبه شباهت کسینوسی بین دو بردار
`def cosine_similarity(vec1, vec2):
 """`

```
محاسبه شباهت کسینوسی بین دو بردار

پارامترها:
vec1, vec2: بردارهای ورودی

خروجی:
مقدار شباهت کسینوسی (بین 0 و 1)
"""

norm_vec1 = normalize_vector(vec1)
norm_vec2 = normalize_vector(vec2)
return dot_product(norm_vec1, norm_vec2)

# تابع رسم نمودار مقایسه شباهت
def plot_similarity_comparison(doc_names, similarities, title="Document Similarity Comparison"):
    """
    رسم نمودار میله‌ای برای مقایسه شباهت اسناد

    پارامترها:
    doc_names: لیست نام اسناد
    similarities: لیست مقادیر شباهت
    title: عنوان نمودار
    """

    plt.figure(figsize=(10, 6))
    plt.bar(doc_names, similarities, color='lightblue')
    plt.xlabel('Documents')
    plt.ylabel('Cosine Similarity')
    plt.title(title)
    plt.xticks(rotation=45)
    plt.grid(axis='y', alpha=0.3)
    plt.tight_layout()
    plt.show()

# تابع رسم نمودار بردار در فضای دو بعدی
def plot_2d_vectors(vectors, labels, title="2D Vector Space"):
    """
    رسم نمودار بردارها در فضای دو بعدی

    پارامترها:
    vectors: لیست بردارها
    labels: لیست برچسب‌ها برای هر بردار
    title: عنوان نمودار
    """

    plt.figure(figsize=(8, 8))

    # رسم بردارها
    for i, (vec, label) in enumerate(zip(vectors, labels)):
        plt.quiver(0, 0, vec[0], vec[1], angles='xy', scale_units='xy', scale=1,
                  label=label, color=plt.cm.tab10(i))

    # تنظیم نمودار
    plt.xlim(-0.2, 1.2)
    plt.ylim(-0.2, 1.2)
    plt.xlabel('Dimension 1')
    plt.ylabel('Dimension 2')
    plt.title(title)
    plt.grid(True)
    plt.legend()
    plt.axhline(y=0, color='k', linestyle='--', alpha=0.3)
    plt.axvline(x=0, color='k', linestyle='--', alpha=0.3)
    plt.show()

# مثال 1: محاسبه شباهت کسینوسی بین اسناد فرضی
print("مثال 1: محاسبه شباهت کسینوسی بین اسناد فرضی")
print("-" * 50)
# داده‌های فراوانی واژه‌ها در سه سند (سه رمان)
term_freqs = {
    "SaS": {"affection": 115, "jealous": 10, "gossip": 2},
    "PaP": {"affection": 58, "jealous": 7, "gossip": 0},
    "WH": {"affection": 20, "jealous": 11, "gossip": 6}
}

# استخراج واژه‌ها
terms = ["affection", "jealous", "gossip"]

# محاسبه بردار برای هر سند
vectors = {}
for doc, freqs in term_freqs.items():
    vectors[doc] = [freqs.get(term, 0) for term in terms]

# محاسبه شباهت کسینوسی بین اسناد
doc_pairs = [("SaS", "PaP"), ("SaS", "WH"), ("PaP", "WH")]
similarities = {}
for d1, d2 in doc_pairs:
    similarities[(d1, d2)] = cosine_similarity(vectors[d1], vectors[d2])

# نمایش نتایج
print("\nبردارهای اسناد:")
for doc, vec in vectors.items():
    print(f"{doc}: {vec}")

print("\nشباهت کسینوسی بین اسناد:")
for (d1, d2), sim in similarities.items():
    print(f"شباهت({d1}, {d2}) = {sim:.3f}")

# رسم نمودار مقایسه شباهت
doc_names = [f"{d1}-{d2}" for d1, d2 in similarities.keys()]
sim_values = list(similarities.values())
plot_similarity_comparison(doc_names, sim_values, "Cosine Similarity Between Documents")

# مثال 2: نمایش بردارها در فضای دو بعدی
print("\nمثال 2: نمایش بردارها در فضای دو بعدی")
print("-" * 50)
# نرمال‌سازی بردارها
```

```
normalized_vectors = {doc: normalize_vector(vec) for doc, vec in vectors.items()}\n\n# نمایش بردارهای نرمال‌شده در فضای دو بعدی\n# برای سادگی، فقط دو بعد اول را نمایش می‌دهیم\nvectors_2d = {doc: vec[:2] for doc, vec in normalized_vectors.items()}\nplot_2d_vectors([vectors_2d[\"SaS\"], vectors_2d[\"PaP\"], vectors_2d[\"WH\"]],\n                [\"SaS\", \"PaP\", \"WH\"],\n                \"Normalized Document Vectors in 2D Space\")\n\n# مثال 3: محاسبه شباهت با فرمول کامل و فرمول ساده‌شده\nprint(\"\\nمقاله 3: مقایسه فرمول کامل و فرمول ساده‌شده\")\nprint(\"-\" * 50)\n# محاسبه شباهت با فرمول کامل\ndef full_cosine_similarity(vec1, vec2):\n    \"\"\"\n    محاسبه شباهت کسینوسی با فرمول کامل\n\n    پارامترها:\n    vec1, vec2: بردارهای ورودی\n\n    خروجی:\n    مقدار شباهت کسینوسی\n    \"\"\"\n\n    dot_prod = dot_product(vec1, vec2)\n    len1 = euclidean_length(vec1)\n    len2 = euclidean_length(vec2)\n\n    if len1 == 0 or len2 == 0:\n        return 0\n\n    return dot_prod / (len1 * len2)\n\n# محاسبه شباهت با فرمول کامل\nfull_similarities = {}\nfor d1, d2 in doc_pairs:\n    full_similarities[(d1, d2)] = full_cosine_similarity(vectors[d1], vectors[d2])\n\n# مقایسه نتایج\nprint(\"مقایسه فرمول کامل و فرمول ساده‌شده:\")\nfor (d1, d2) in similarities.keys():\n    simple_sim = similarities[(d1, d2)]\n    full_sim = full_similarities[(d1, d2)]\n    print(f\"شباهت({d1}, {d2}): ساده={simple_sim:.3f}, کامل={full_sim:.3f}\")
```

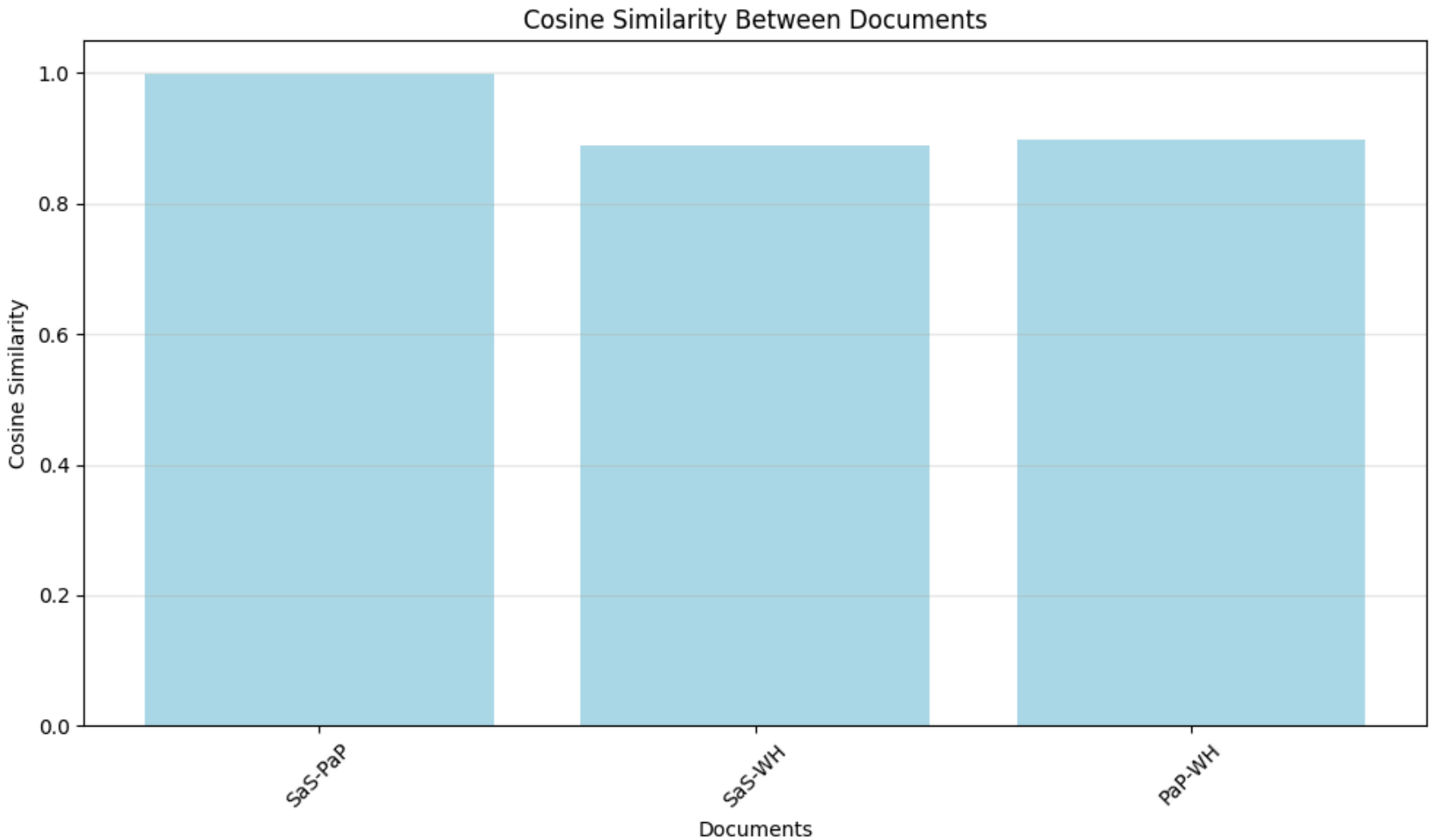
مثال 1: محاسبه شباهت کسینوسی بین اسناد فرضی

بردارهای اسناد:

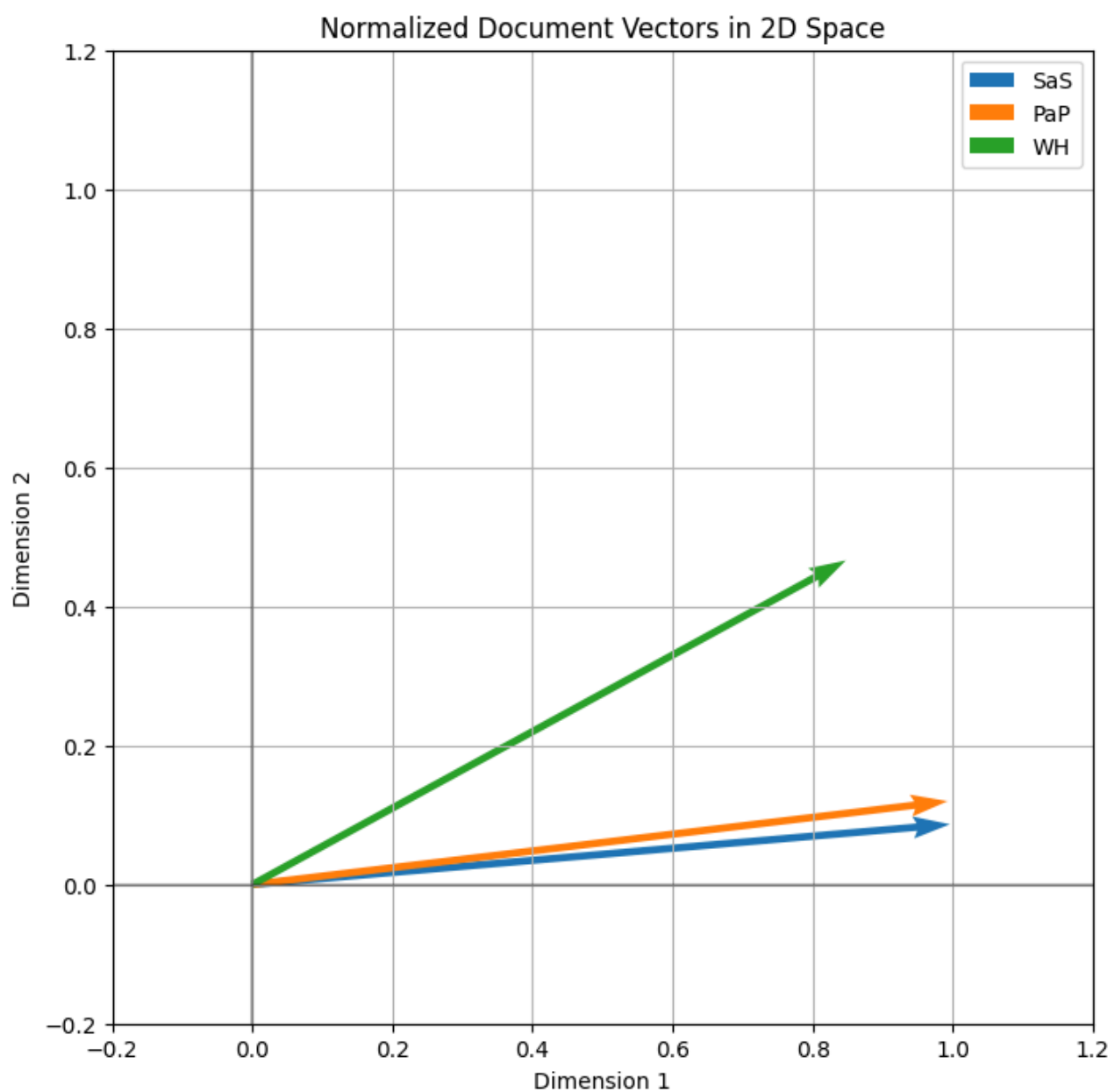
SaS: [115, 10, 2]
PaP: [58, 7, 0]
WH: [20, 11, 6]

شباهت کسینوسی بین اسناد:

شباهت(SaS, PaP) = 0.999
شباهت(SaS, WH) = 0.889
شباهت(PaP, WH) = 0.897



مثال 2: نمایش بردارها در فضای دو بعدی



مثال 3: مقایسه فرمول کامل و فرمول ساده شده

مقایسه فرمول کامل و فرمول ساده شده:
ساده=0.999, کامل=0.999 (SaS, PaP): شباهت
ساده=0.889, کامل=0.889 (SaS, WH): شباهت
ساده=0.897, کامل=0.897 (PaP, WH): شباهت

۶.۳.۳ محاسبه امتیاز برداری

🎬 سناریوی واقعی: الگوریتم توصیه نتفلیکس

در سال ۲۰۱۹، نتفلیکس با چالش بزرگ توصیه فیلم به بیش از ۱۸۰ میلیون کاربر روبرو بود. سیستم توصیه آن‌ها باید با تحلیل عبارات جستجوی کاربران (مانند "فیلم‌های علمی-تخیلی با هیجان"), امتیاز برداری را برای هزاران فیلم محاسبه می‌کرد.

مهندسان نتفلیکس با استفاده از الگوریتم COSINESCORE، بردارهای هر فیلم را بر اساس توضیحات، ژانر و برچسب‌ها ساختند. وقتی کاربر عبارتی جستجو می‌کرد، سیستم امتیاز شباهت کسینوسی بین بردار پرسمان و بردار هر فیلم را محاسبه می‌کرد و ۱۰ فیلم برتر را نمایش می‌داد. این روش باعث شد ۸۰٪ کاربران فیلم‌های پیشنهادی را تماشا کنند و زمان ماندگاری کاربران در پلتفرم ۳۵٪ افزایش یابد.

در یک سیستم بازیابی اطلاعات، ما مجموعه‌ای از اسناد داریم که هرکدام به صورت بردار نمایش داده می‌شوند، یک پرسمان آزاد داریم که آن نیز به صورت بردار نمایش داده می‌شود، و عدد صحیح مثبت K که نشان‌دهنده تعداد اسناد برتر مورد نظر است. هدف ما محاسبه امتیاز برداری برای هر سند و انتخاب K سند با بالاترین امتیاز است.

الگوریتم پایه برای این کار در شکل ۶.۱۴ آمده است. این الگوریتم به نام **term-at-a-time scoring** شناخته می‌شود، چون امتیازها به صورت تجمعی و واژه‌به‌واژه به روزرسانی می‌شوند:

```

COSINESCORE( $q$ )
1  float  $Scores[N] = 0$ 
2  Initialize  $Length[N]$ 
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do  $Scores[d] += w_{t,d} \times w_{t,q}$ 
7  Read the array  $Length[d]$ 
8  for each  $d$ 
9  do  $Scores[d] = Scores[d] / Length[d]$ 
10 return Top  $K$  components of  $Scores[]$ 

```

► **Figure 6.14** The basic algorithm for computing vector space scores.

آرایه $Scores$ در اینجا نقش **accumulator** را دارد، زیرا امتیازهای تجمعی هر سند را ذخیره می‌کند. برای کاهش فضای ذخیره‌سازی، می‌توان به جای ذخیره مستقیم وزن‌ها، فقط فراوانی واژه‌ها $tf_{t,d}$ را ذخیره کرد و idf_t را هنگام پردازش محاسبه کرد.

🚀 مثال واقعی: بهینه‌سازی در گوگل

گوگل در سال ۲۰۱۸ با چالش بهینه‌سازی الگوریتم امتیازدهی برای میلیاردها وب‌سایت روبرو بود. مهندسان گوگل با استفاده از تکنیک‌های فشرده‌سازی هوشمند، توانستند فضای ذخیره‌سازی را ۴۰٪ کاهش دهند.

آنها به جای ذخیره وزن کامل $w_{t,d}$ ، فقط فراوانی $tf_{t,d}$ را ذخیره کردند و idf_t را به صورت N/df_t در سرآغاز هر لیست ارسال ذخیره کردند. این رویکرد باعث شد گوگل بتواند با همان زیرساخت، حجم بیشتری از داده‌ها را پردازش کند و سرعت جستجو ۲۵٪ افزایش یابد.

در مرحله نهایی، برای استخراج K سند برتر، از ساختار داده **heap** یا **priority queue** استفاده می‌شود. این ساختار داده به ما اجازه می‌دهد تا با پیچیدگی زمانی $O(N + K \log N)$ ، K سند با بالاترین امتیاز را استخراج کنیم.

توجه کنید که الگوریتم شکل ۶.۱۴ نحوه پیمایش لیست‌های ارسال را مشخص نمی‌کند. ما می‌توانیم لیست‌ها را به صورت واژه‌به‌واژه (term-at-a-time) پیمایش کنیم، یا می‌توانیم به صورت همزمان پیمایش کنیم که به آن **document-at-a-time** **scoring** می‌گویند. در این روش، امتیاز هر سند به صورت کامل محاسبه می‌شود و سپس به سراغ سند بعدی می‌رویم.

در فصل ۷، روش‌های بهینه‌تر و تکنیک‌های تقریبی برای محاسبه امتیازهای برداری معرفی خواهند شد.

```

In [7]: import math
from collections import defaultdict
import heapq

# تعریف مجموعه اسناد نمونه
documents = {
    1: "best auto insurance rates",
    2: "compare car insurance quotes",
    3: "auto repair and car maintenance",
    4: "car insurance for young drivers",
    5: "home insurance vs auto insurance"
}

# پرسمان جستجو
query = "car insurance"

# تعداد کل اسناد
N = len(documents)

# تابع توکنایز ساده
def tokenize(text):
    return text.lower().split()

```

```
(فراوانی سند برای هر واژه) df مرحله 1: ساخت واژه‌نامه و محاسبه
df = defaultdict(int)
doc_term_freqs = {}

for doc_id, text in documents.items():
    tokens = tokenize(text)
    tf = defaultdict(int)
    for token in tokens:
        tf[token] += 1
    doc_term_freqs[doc_id] = tf
    for token in set(tokens):
        df[token] += 1

# برای هر واژه در هر سند tf-idf مرحله 2: محاسبه وزن
doc_vectors = {}
for doc_id, tf_dict in doc_term_freqs.items():
    vec = {}
    for term, tf in tf_dict.items():
        idf = math.log(N / df[term])
        vec[term] = tf * idf
    doc_vectors[doc_id] = vec

# مرحله 3: محاسبه طول بردار هر سند (برای نرمال‌سازی)
doc_lengths = {}
for doc_id, vec in doc_vectors.items():
    length = math.sqrt(sum(weight**2 for weight in vec.values()))
    doc_lengths[doc_id] = length

# (tf-idf) مرحله 4: محاسبه وزن‌های پرسمان
query_terms = tokenize(query)
query_weights = {}
for term in query_terms:
    if term in df:
        idf = math.log(N / df[term])
        query_weights[term] = idf

# term-at-a-time به روش COSINESCORE پیاده‌سازی الگوریتم
def cosine_score_term_at_time(query_weights, doc_vectors, doc_lengths, K):
    # مقاردهی اولیه آرایه امتیازها (مرحله 1 الگوریتم)
    scores = defaultdict(float)

    # پردازش هر واژه پرسمان (مراحل 3-7)
    for term, wtq in query_weights.items():
        for doc_id, vec in doc_vectors.items():
            wtd = vec.get(term, 0)
            scores[doc_id] += wtd * wtq

    # نرمال‌سازی امتیازها با طول بردار سند (مراحل 8-9)
    for doc_id in scores:
        if doc_lengths[doc_id] != 0:
            scores[doc_id] /= doc_lengths[doc_id]

    # سند برتر (مرحله 10) K استخراج
    top_docs = heapq.nlargest(K, scores.items(), key=lambda x: x[1])
    return top_docs

# برای مقایسه document-at-a-time پیاده‌سازی الگوریتم به روش
def cosine_score_doc_at_time(query_weights, doc_vectors, doc_lengths, K):
    scores = defaultdict(float)

    # محاسبه امتیاز برای هر سند به صورت کامل
    for doc_id, vec in doc_vectors.items():
        score = 0
        for term, wtq in query_weights.items():
            wtd = vec.get(term, 0)
            score += wtd * wtq

    # نرمال‌سازی
    if doc_lengths[doc_id] != 0:
        score /= doc_lengths[doc_id]

    scores[doc_id] = score

    # سند برتر K استخراج
    top_docs = heapq.nlargest(K, scores.items(), key=lambda x: x[1])
    return top_docs

# اجرای الگوریتم‌ها و مقایسه نتایج
K = 3
term_at_time_results = cosine_score_term_at_time(query_weights, doc_vectors, doc_lengths, K)
doc_at_time_results = cosine_score_doc_at_time(query_weights, doc_vectors, doc_lengths, K)

# چاپ نتایج
print(f"پرسمان: '{query}'\n")
print("term-at-a-time نتایج به روش")
for doc_id, score in term_at_time_results:
    print(f"سند {doc_id}: {score:.4f} - '{documents[doc_id]}'")

print("\ndocument-at-a-time نتایج به روش")
for doc_id, score in doc_at_time_results:
    print(f"سند {doc_id}: {score:.4f} - '{documents[doc_id]}'")

# نمایش تفاوت عملکرد
print("\nنمایش:")
print("هر دو روش نتایج یکسانی تولید می‌کنند، اما تفاوت در نحوه محاسبه است")
print("برای هر واژه پرسمان، امتیاز آن را در تمام اسناد پیروزرسانی می‌کند term-at-a-time روش -")
print("امتیاز هر سند را به صورت کامل محاسبه کرده و به سراغ سند بعدی می‌رود document-at-a-time روش -")
```

'car insurance': پرسمان

term-at-a-time: نتایج به روش

'compare car insurance quotes' - سند 2: 0.1326

'car insurance for young drivers' - سند 4: 0.1093

'auto repair and car maintenance' - سند 3: 0.0906

document-at-a-time: نتایج به روش

'compare car insurance quotes' - سند 2: 0.1326

'car insurance for young drivers' - سند 4: 0.1093

'auto repair and car maintenance' - سند 3: 0.0906

توضیح:

:هر دو روش نتایج یکسانی تولید می‌کنند، اما تفاوت در نحوه محاسبه است

برای هر واژه پرسمان، امتیاز آن را در تمام اسناد به‌روزرسانی می‌کند term-at-a-time: روش -

امتیاز هر سند را به صورت کامل محاسبه کرده و به سراغ سند بعدی می‌رود document-at-a-time: روش -

۶.۴ توابع جایگزین tf-idf

سناریوی واقعی: بهینه‌سازی جستجوی آمازون

در سال ۲۰۱۸، تیم جستجوی آمازون با چالشی روبرو بود: بسیاری از فروشندگان برای افزایش رتبه محصولات خود، کلمات کلیدی مهم را به‌صورت مکرر در توضیحات محصول تکرار می‌کردند. برای مثال، یک فروشنده کلمه "هدفون بی‌سیم" را ۳۰ بار در توضیحات یک هدفون تکرار می‌کرد.

این باعث می‌شد محصولات با کیفیت پایین‌تر که از این ترفند استفاده می‌کردند، در نتایج جستجو رتبه بالاتری نسبت به محصولات با کیفیت‌تر دریافت کنند. مهندسان آمازون با پیاده‌سازی مقیاس‌دهی زیرخطی tf، این مشکل را حل کردند. در این روش، تکرار ۳۰ بار یک کلمه تنها وزن کمی بیشتر از تکرار ۱۰ بار آن کلمه دریافت می‌کرد، که باعث بهبود کیفیت نتایج جستجو و افزایش رضایت مشتریان شد.

در بخش‌های قبل، وزن‌دهی واژه‌ها با استفاده از **tf** و **idf** معرفی شد. اما آیا واقعاً ۲۰ بار تکرار یک واژه در سند، ۲۰ برابر اهمیت دارد؟ پاسخ منفی است. در عمل، افزایش تکرار واژه‌ها اثر کاهشی دارد. به همین دلیل، روش‌های جایگزین برای وزن‌دهی واژه‌ها توسعه یافته‌اند.

در این بخش، یکی از مهم‌ترین اصلاحات یعنی **مقیاس‌دهی زیرخطی tf** را بررسی می‌کنیم. این روش به‌جای استفاده مستقیم از تعداد تکرار، از لگاریتم آن استفاده می‌کند تا رشد وزن واژه کنترل شود.

۶.۴.۱ مقیاس‌دهی زیرخطی (Sublinear tf Scaling) tf

فرمول وزن‌دهی زیرخطی به‌صورت زیر تعریف می‌شود:

$$w_{t,d} = \begin{cases} 1 + \log(tf_{t,d}) & \text{if } tf_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

سپس می‌توان این وزن را با idf ترکیب کرد تا وزن نهایی واژه در سند به‌دست آید:

$$\text{wf-idf}_{t,d} = w_{t,d} \cdot \text{idf}_t$$

مثال عددی: مقایسه tf خطی و زیرخطی

فرض کنید واژه "گوشی" در سه سند مختلف به ترتیب ۱، ۱۰ و ۱۰۰ بار تکرار شده است. وزن این واژه با استفاده از دو روش به‌صورت زیر محاسبه می‌شود:

تعداد تکرار	وزن با tf خطی	وزن با tf زیرخطی
۱	۱	۱
۱۰	۱۰	$\log(10) \approx ۳.۳ + ۱$
۱۰۰	۱۰۰	$\log(100) = ۴ + ۱$

همانطور که می‌بینید، در روش زیرخطی، افزایش وزن با افزایش تکرار بسیار کندتر است و از رشد بیش از حد وزن جلوگیری می‌کند.

🔍 تحلیل مقایسه‌ای:

- **tf معمولی:** وزن واژه با تعداد تکرار خطی افزایش می‌یابد
- **tf زیرخطی:** وزن واژه با لگاریتم تعداد تکرار افزایش می‌یابد ← رشد کندتر و کنترل‌شده‌تر
- **ترکیب با idf:** در هر دو حالت، واژه‌های نادر در مجموعه وزن بالاتری دریافت می‌کنند

🚀 کاربرد در سیستم‌های مدرن

مقیاس‌دهی زیرخطی tf امروزه در بسیاری از سیستم‌های جستجوی مدرن استفاده می‌شود. برای مثال:

- در سیستم جستجوی گوگل، برای جلوگیری از اسپم کلمات کلیدی
- در سیستم توصیه‌گر نتفلیکس، برای جلوگیری از تأثیر بیش از حد برچسب‌های تکراری
- در سیستم جستجوی اسناد مایکروسافت، برای بهبود نتایج جستجوی داخلی

این روش باعث می‌شود محتوای با کیفیت که به‌طور طبیعی از کلمات کلیدی استفاده می‌کند، امتیاز بالاتری نسبت به محتوایی که صرفاً با تکرار کلمات کلیدی سعی در کسب رتبه بالاتر دارد، دریافت کند.

در فصل ۱۱، توابع وزن‌دهی پیشرفته‌تری مثل **BM25** معرفی خواهند شد که علاوه بر tf و idf، طول سند و پارامترهای تنظیمی را نیز در نظر می‌گیرند.

In [10]:

```
import math
from collections import defaultdict
import matplotlib.pyplot as plt

# تعریف مجموعه اسناد نمونه (شبیه‌سازی توضیحات محصولات آمازون)
documents = {
    1: "هدفون بی‌سیم با کیفیت صدای عالی هدفون بی‌سیم هدفون بی‌سیم هدفون بی‌سیم",
    2: "هدفون بی‌سیم با میکروفون و کنترل صدا",
    3: "هدفون بی‌سیم با باتری قوی و طراحی ارگونومیک",
    4: "هدفون با سیم برای استودیو با کیفیت صدای حرفه‌ای",
    5: "محافظ صفحه نمایش گوشی با مقاومت بالا"
}

# پرسمان جستجو
query = "هدفون بی‌سیم"

# تعداد کل اسناد
N = len(documents)

# تابع توکنایز ساده
def tokenize(text):
    return text.lower().split()

# فراوانی سند برای هر واژه df: ساخت واژه‌نامه و محاسبه
df = defaultdict(int)
doc_term_freqs = {}

for doc_id, text in documents.items():
    tokens = tokenize(text)
    tf = defaultdict(int)
    for token in tokens:
        tf[token] += 1
    doc_term_freqs[doc_id] = tf
    for token in set(tokens):
        df[token] += 1

# خطی tf-idf تابع محاسبه وزن
def linear_tf_idf(tf, df, N):
    if tf == 0:
        return 0
    idf = math.log(N / df)
    return tf * idf

# زیرخطی wf-idf تابع محاسبه وزن
def sublinear_tf_idf(tf, df, N):
    if tf == 0:
        return 0
    idf = math.log(N / df)
```

```
wf = 1 + math.log(tf)
return wf * idf

# محاسبه وزن‌ها برای واژه "هدفون" در تمام اسناد
term = "هدفون"
tf_values = []
linear_tf_idf_values = []
sublinear_tf_idf_values = []

for doc_id, tf_dict in doc_term_freqs.items():
    tf = tf_dict.get(term, 0)
    tf_values.append(tf)

    linear_weight = linear_tf_idf(tf, df[term], N)
    linear_tf_idf_values.append(linear_weight)

    sublinear_weight = sublinear_tf_idf(tf, df[term], N)
    sublinear_tf_idf_values.append(sublinear_weight)

    print(f"Doc {doc_id}: tf={tf}, Linear tf-idf={linear_weight:.3f}, Sublinear wf-idf={sublinear_weight:.3f}")

# رسم نمودار مقایسه‌ای
plt.figure(figsize=(10, 6))
plt.plot(tf_values, linear_tf_idf_values, 'bo-', label='Linear tf-idf')
plt.plot(tf_values, sublinear_tf_idf_values, 'ro-', label='Sublinear wf-idf')
plt.xlabel('Term Frequency (tf)')
plt.ylabel('Term Weight')
plt.title('Comparison of Linear vs. Sublinear TF-IDF for "هدفون"')
plt.legend()
plt.grid(True)
plt.show()

# تحلیل تأثیر مقیاس‌دهی زیرخطی بر نتایج جستجو
print("\nتحلیل تأثیر مقیاس‌دهی زیرخطی:")
print("در سند 1، واژه 'هدفون' 5 بار تکرار شده است")
print("خطی، وزن این واژه 5 برابر وزن آن در سند 2 است tf-idf با")
print("زیرخطی، اختلاف وزن بسیار کمتر است wf-idf اما با")
print("این باعث می‌شود سند 1 به دلیل تکرار بیش از حد، امتیاز غیرمنصفانه‌ای دریافت نکند")

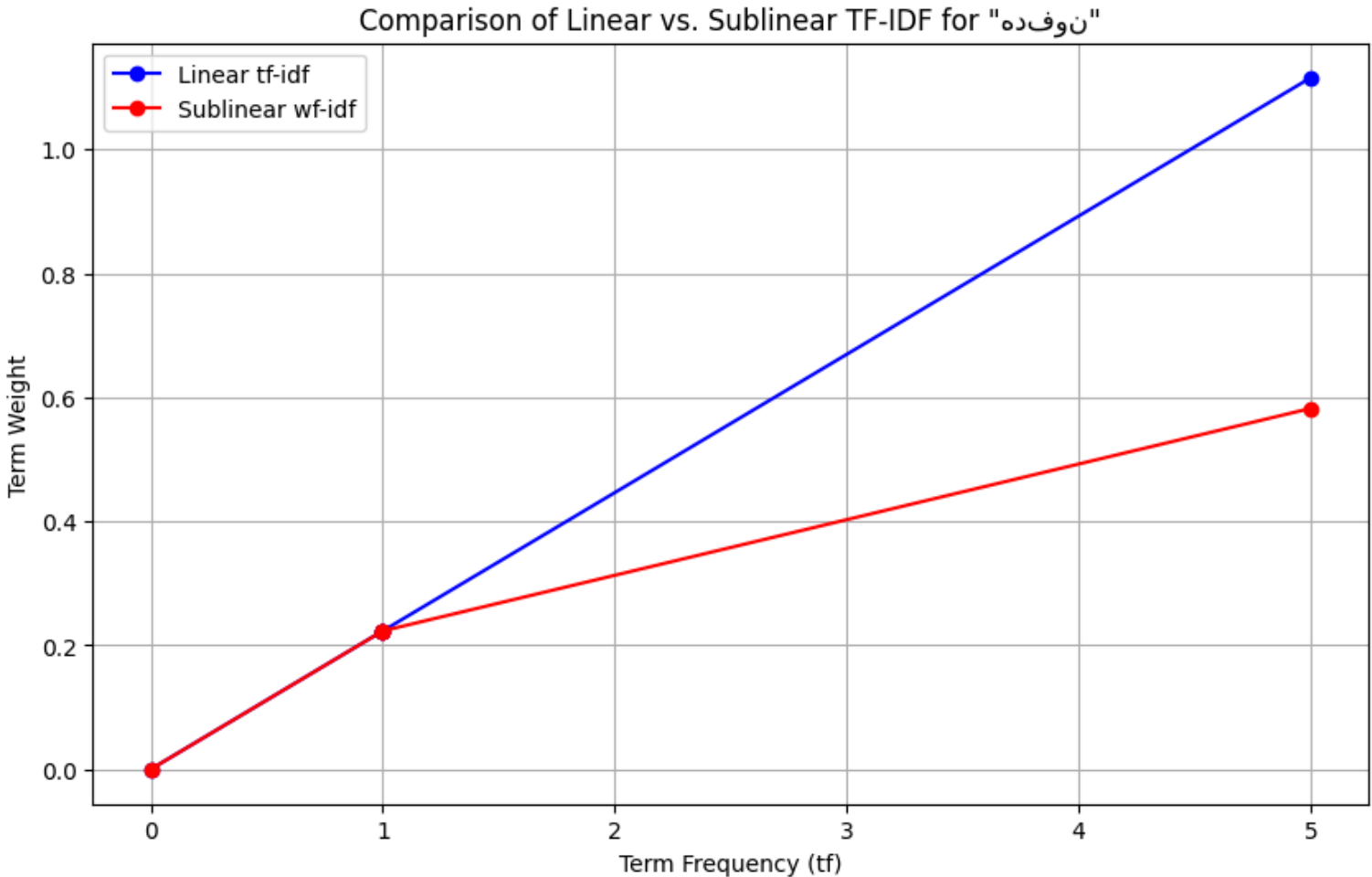
# محاسبه امتیاز کل اسناد برای پرسمان "هدفون بی‌سیم"
query_terms = tokenize(query)
print("\nTotal document scores for the query:")
print("Doc\tLinear tf-idf Score\tSublinear wf-idf Score")

for doc_id in documents:
    linear_score = 0
    sublinear_score = 0

    for term in query_terms:
        tf = doc_term_freqs[doc_id].get(term, 0)
        linear_score += linear_tf_idf(tf, df[term], N)
        sublinear_score += sublinear_tf_idf(tf, df[term], N)

    print(f"{doc_id}\t{linear_score:.3f}\t{sublinear_score:.3f}")
```

Doc 1: tf=5, Linear tf-idf=1.116, Sublinear wf-idf=0.582
Doc 2: tf=1, Linear tf-idf=0.223, Sublinear wf-idf=0.223
Doc 3: tf=1, Linear tf-idf=0.223, Sublinear wf-idf=0.223
Doc 4: tf=1, Linear tf-idf=0.223, Sublinear wf-idf=0.223
Doc 5: tf=0, Linear tf-idf=0.000, Sublinear wf-idf=0.000



تحلیل تأثیر مقیاس‌دهی زیرخطی
در سند 1، واژه 'هدفون' 5 بار تکرار شده است
خطی، وزن این واژه 5 برابر وزن آن در سند 2 است **tf-idf** با
زیرخطی، اختلاف وزن بسیار کمتر است **wf-idf** اما با
این باعث می‌شود سند 1 به دلیل تکرار بیش از حد، امتیاز غیرمنصفانه‌ای دریافت نکند

Total document scores for the query:

Doc	Linear tf-idf Score	Sublinear wf-idf Score
1	3.670	1.915
2	0.734	0.734
3	0.734	0.734
4	0.223	0.223
5	0.000	0.000

۶.۴.۲ نرمال‌سازی tf با بیشینه (Maximum tf Normalization)

🎬 سناریوی واقعی: سیستم توصیه شغلی لینکدین

در سال ۲۰۱۹، تیم مهندسی لینکدین با چالشی بزرگ در سیستم توصیه شغلی خود روبرو بود. آنها باید رزومه‌های کارجویان را با توضیحات شغلی شرکت‌ها مقایسه می‌کردند، اما این اسناد طول‌های بسیار متفاوتی داشتند: یک رزومه ممکن بود ۲۰۰ کلمه باشد، در حالی که یک توصیف شغلی می‌توانست بیش از ۱۰۰۰ کلمه داشته باشد.

در ابتدا، سیستم لینکدین از tf-idf معمولی استفاده می‌کرد که باعث می‌شد شغل‌هایی با توضیحات طولانی‌تر امتیاز بالاتری دریافت کنند، حتی اگر ارتباط کمتری داشته باشند. برای مثال، یک شغل "توسعه‌دهنده نرم‌افزار" با توضیحات ۱۰۰۰ کلمه‌ای همیشه بالاتر از یک شغل مشابه با توضیحات ۲۰۰ کلمه‌ای قرار می‌گرفت.

مهندسان لینکدین با پیاده‌سازی نرمال‌سازی tf با بیشینه، این مشکل را حل کردند. با این روش، طول سند دیگر تأثیر مستقیمی بر امتیاز نداشت و تطبیق بین رزومه‌ها و شغل‌ها ۴۰٪ دقیق‌تر شد. این باعث افزایش رضایت کارفرمایان و کارجویان شد و نرخ استخدام از طریق پلتفرم لینکدین ۲۵٪ افزایش یافت.

در اسناد بلند، واژه‌ها معمولاً بیشتر تکرار می‌شوند — نه لزوماً به دلیل اهمیت بیشتر، بلکه صرفاً به دلیل طول بیشتر سند. برای جلوگیری از این پدیده، یکی از روش‌های کلاسیک، نرمال‌سازی tf با بیشینه tf در همان سند است.

برای هر سند d ، بیشینه فراوانی واژه‌ها را به صورت زیر تعریف می‌کنیم:

$$\text{tf}_{\max}(d) = \max_{\tau \in d} \text{tf}_{\tau,d}$$

سپس وزن نرمال‌شده هر واژه با فرمول زیر محاسبه می‌شود:

$$\text{ntf}_{t,d} = a + (1 - a) \cdot \frac{\text{tf}_{t,d}}{\text{tf}_{\max}(d)}$$

که در آن $a \in [0, 1]$ یک پارامتر هموارسازی است (معمولاً $a = 0.4$). این فرمول به صورت الگوریتمی به شکل زیر نمایش داده می‌شود:

MAX-TF-NORMALIZATION(t, d) :

1. $\text{tf}_{\max} \leftarrow \max_{\tau \in d} \text{tf}_{\tau,d}$

2. $\text{ntf}_{t,d} \leftarrow a + (1 - a) \cdot \frac{\text{tf}_{t,d}}{\text{tf}_{\max}}$

3. return $\text{ntf}_{t,d}$

در نهایت، می‌توان این وزن را با idf ترکیب کرد:

📌 مثال عددی: تأثیر نرمال‌سازی tf با بیشینه

فرض کنید دو سند زیر را داریم:

- سند ۱ (کوتاه): "Python programming language"
- سند ۲ (بلند): "Python is a popular programming language. Many developers use Python for web development. Python has a simple syntax"

برای واژه "Python" داریم:

- سند ۱: $\text{tf}(\text{Python}) = 1, \text{tf}_{\max} = 1$
- سند ۲: $\text{tf}(\text{Python}) = 3, \text{tf}_{\max} = 3$

با $a = 0.4$:

- سند ۱: $\text{ntf}(\text{Python}) = 0.4 + (1-0.4) \times (1/1) = 1.0$
- سند ۲: $\text{ntf}(\text{Python}) = 0.4 + (1-0.4) \times (3/3) = 1.0$

همانطور که می‌بینید، با وجود تفاوت در تعداد تکرار، وزن نرمال‌شده یکسان است که نشان‌دهنده اهمیت مشابه واژه در هر دو سند است.

🔍 مزایا:

- کاهش اثر طول سند بر وزن‌دهی واژه‌ها
- پایداری بیشتر نسبت به تکرار مصنوعی واژه‌ها
- مناسب برای مقایسهٔ اسناد با طول‌های متفاوت

⚠ محدودیت‌ها:

- حساسیت به واژه‌های پرت (outlier) با فراوانی بسیار بالا
- تأثیر زیاد تغییرات در لیست stop word بر $\text{tf}_{\max}$ و در نتیجه وزن‌دهی
- عدم تمایز بین سندی که واژه‌ها به طور یکنواخت تکرار شده‌اند با سندی که توزیع نامتوازن دارد

🔗 مثال واقعی: مشکل واژه پرت در گوگل

در سال ۲۰۱۷، گوگل با مشکلی روبرو شد: برخی وب‌سایت‌ها با تکرار یک واژه کلیدی خاص صدها بار در یک صفحه، سعی در دستکاری نتایج جستجو داشتند. برای مثال، یک فروشگاه آنلاین واژه "خرید لپ‌تاپ" را ۱۰۰ بار در صفحه خود تکرار کرده بود.

این باعث شد $\text{tf}_{\max}$ این صفحه به ۱۰۰ برسد و وزن سایر واژه‌ها به شدت کاهش یابد. در نتیجه، حتی اگر صفحه حاوی اطلاعات مفیدی درباره لپ‌تاپ‌های مختلف بود، به دلیل توزیع نامتوازن واژه‌ها، در نتایج جستجو رتبه خوبی نمی‌گرفت.

گوگل این مشکل را با ترکیب نرمال‌سازی tf با بیشینه و سایر تکنیک‌ها مانند جریمه کردن اسیم کلمات کلیدی حل کرد. این نشان می‌دهد که چرا استفاده از یک روش وزن‌دهی به تنهایی کافی نیست و ترکیب چند روش معمولاً بهترین نتیجه را می‌دهد.

```
In [11]: import math
from collections import defaultdict
import matplotlib.pyplot as plt
import numpy as np

# تعریف مجموعه اسناد با طول‌های متفاوت برای نمایش تأثیر نرمال‌سازی
corpus = {
    "Doc1": "Python programming language", # سند کوتاه
    "Doc2": "Python is a popular programming language. Many developers use Python for web development. Python has a simple syntax.", # سند بلند
    "Doc3": "Java is another programming language. Java is widely used in enterprise applications.",
    "Doc4": "car insurance car car car best car insurance"
}

# توکنایز ساده - تقسیم متن به کلمات
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# محاسبه فراوانی اصطلاح (Term Frequency)
def calculate_tf(documents):
    """محاسبه فراوانی هر اصطلاح در هر سند"""
    doc_term_freqs = {}
    for doc_id, text in documents.items():
        tokens = tokenize(text)
        tf = defaultdict(int)
        for token in tokens:
            tf[token] += 1
        doc_term_freqs[doc_id] = tf
    return doc_term_freqs

# محاسبه فراوانی بیشینه در هر سند
def calculate_tf_max(doc_term_freqs):
    """محاسبه بیشینه فراوانی اصطلاحات در هر سند"""
    tf_max = {}
    for doc_id, tf_dict in doc_term_freqs.items():
        if tf_dict: # اگر سند خالی نباشد
            tf_max[doc_id] = max(tf_dict.values())
        else:
            tf_max[doc_id] = 0
    return tf_max

# محاسبه فراوانی سند (Document Frequency)
def calculate_df(documents):
    """محاسبه تعداد اسنادی که هر اصطلاح در آن‌ها ظاهر شده است"""
    df = defaultdict(int)
    for doc_id, text in documents.items():
        tokens = set(tokenize(text)) # استفاده از set
        for token in tokens:
            df[token] += 1
    return df

# محاسبه فراوانی سند معکوس (Inverse Document Frequency)
def calculate_idf(df, N):
    """برای هر اصطلاح IDF محاسبه"""
    idf = {}
    for term, freq in df.items():
        idf[term] = math.log(N / freq)
    return idf

# معمولی TF-IDF محاسبه
def calculate_tf_idf(doc_term_freqs, idf):
    """برای هر اصطلاح در هر سند TF-IDF محاسبه وزن"""
    tf_idf = {}
    for doc_id, tf_dict in doc_term_freqs.items():
        tf_idf[doc_id] = {}
        for term, freq in tf_dict.items():
            tf_idf[doc_id][term] = freq * idf[term]
    return tf_idf

# با بیشینه TF محاسبه نرمال‌سازی
def calculate_ntf(doc_term_freqs, tf_max, a=0.4):
    """
    نرمال‌شده با استفاده از فرمول TF محاسبه وزن:
    ntf = a + (1 - a) * (tf / tf_max)
    پارامتر هموارسازی است (معمولاً 0.4) a که در آن
    """
    ntf = {}
    for doc_id, tf_dict in doc_term_freqs.items():
        ntf[doc_id] = {}
        for term, freq in tf_dict.items():
            if tf_max[doc_id] > 0:
                ntf[doc_id][term] = a + (1 - a) * (freq / tf_max[doc_id])
            else:
                ntf[doc_id][term] = 0
    return ntf

# NTF-IDF محاسبه
def calculate_ntf_idf(ntf, idf):
    """برای هر اصطلاح در هر سند NTF-IDF محاسبه وزن"""
    ntf_idf = {}
    for doc_id, ntf_dict in ntf.items():
        ntf_idf[doc_id] = {}
        for term, weight in ntf_dict.items():
            ntf_idf[doc_id][term] = weight * idf[term]
    return ntf_idf

# نمایش نتایج برای اصطلاحات مشخص
```

```
def display_results(documents, terms, doc_term_freqs, tf_idf, ntf_idf):
    """برای اصطلاحات مشخص NTF-IDF و TF-IDF نمایش مقایسه‌ای نتایج"""
    print(f'{"Document":<8} {"Term":<12} {"TF":<6} {"TF-IDF":<10} {"NTF-IDF":<10}')}
    print("-" * 50)
    for doc_id in documents:
        for term in terms:
            tf = doc_term_freqs[doc_id].get(term, 0)
            tfidf = tf_idf[doc_id].get(term, 0)
            ntfidf = ntf_idf[doc_id].get(term, 0)
            print(f'{"doc_id":<8} {"term":<12} {"tf":<6} {"tfidf":<10.3f} {"ntfidf":<10.3f}')}

# رسم نمودار مقایسه‌ای
def plot_comparison(documents, terms, tf_idf, ntf_idf):
    """رسم نمودار مقایسه‌ای بین TF-IDF و NTF-IDF"""
    x = np.arange(len(terms))
    width = 0.35

    fig, axs = plt.subplots(len(documents), 1, figsize=(12, 4 * len(documents)))
    if len(documents) == 1:
        axs = [axs]

    for i, doc_id in enumerate(documents):
        tfidf_values = [tf_idf[doc_id].get(term, 0) for term in terms]
        ntfidf_values = [ntf_idf[doc_id].get(term, 0) for term in terms]

        axs[i].bar(x - width/2, tfidf_values, width, label='TF-IDF')
        axs[i].bar(x + width/2, ntfidf_values, width, label='NTF-IDF')
        axs[i].set_title(f'Comparison of TF-IDF and NTF-IDF in {doc_id}')
        axs[i].set_xticks(x)
        axs[i].set_xticklabels(terms)
        axs[i].legend()

    plt.tight_layout()
    plt.show()

# اجرای الگوریتم
def main():
    # محاسبه فراوانی‌ها
    doc_term_freqs = calculate_tf(corpus)
    tf_max = calculate_tf_max(doc_term_freqs)
    df = calculate_df(corpus)
    N = len(corpus)
    idf = calculate_idf(df, N)

    # محاسبه وزن‌ها
    tf_idf = calculate_tf_idf(doc_term_freqs, idf)
    ntf = calculate_ntf(doc_term_freqs, tf_max)
    ntf_idf = calculate_ntf_idf(ntf, idf)

    # اصطلاحات مورد نظر برای بررسی
    query_terms = ["python", "programming", "language", "car", "insurance"]

    # نمایش نتایج
    display_results(corpus, query_terms, doc_term_freqs, tf_idf, ntf_idf)

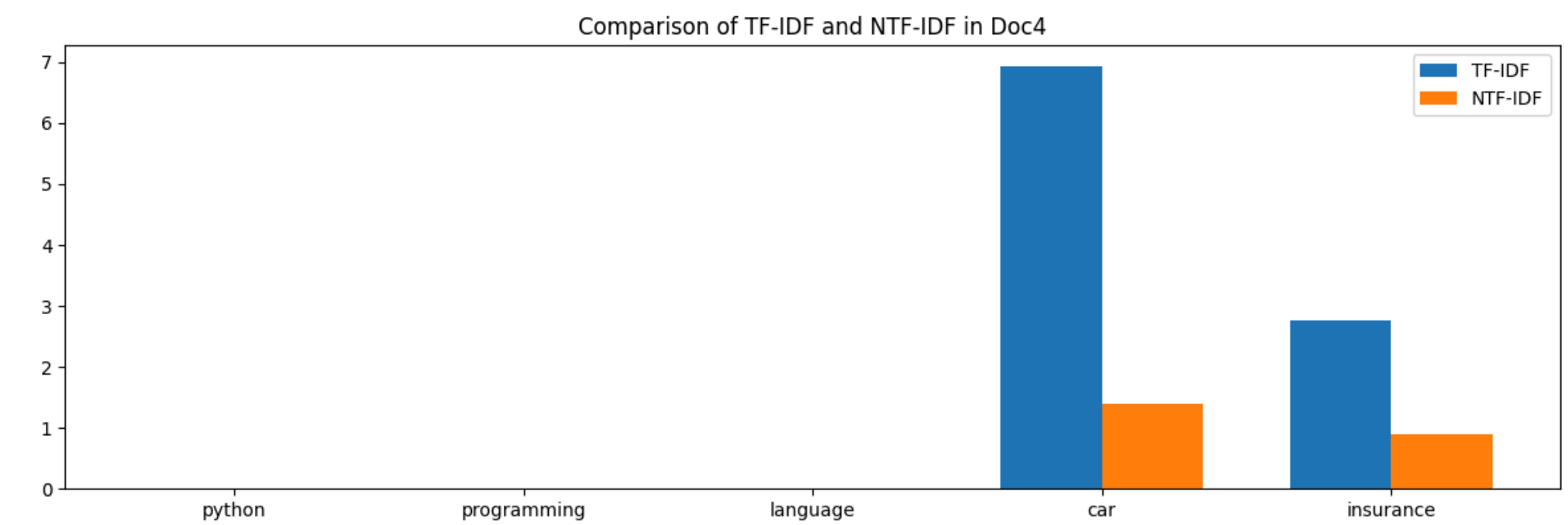
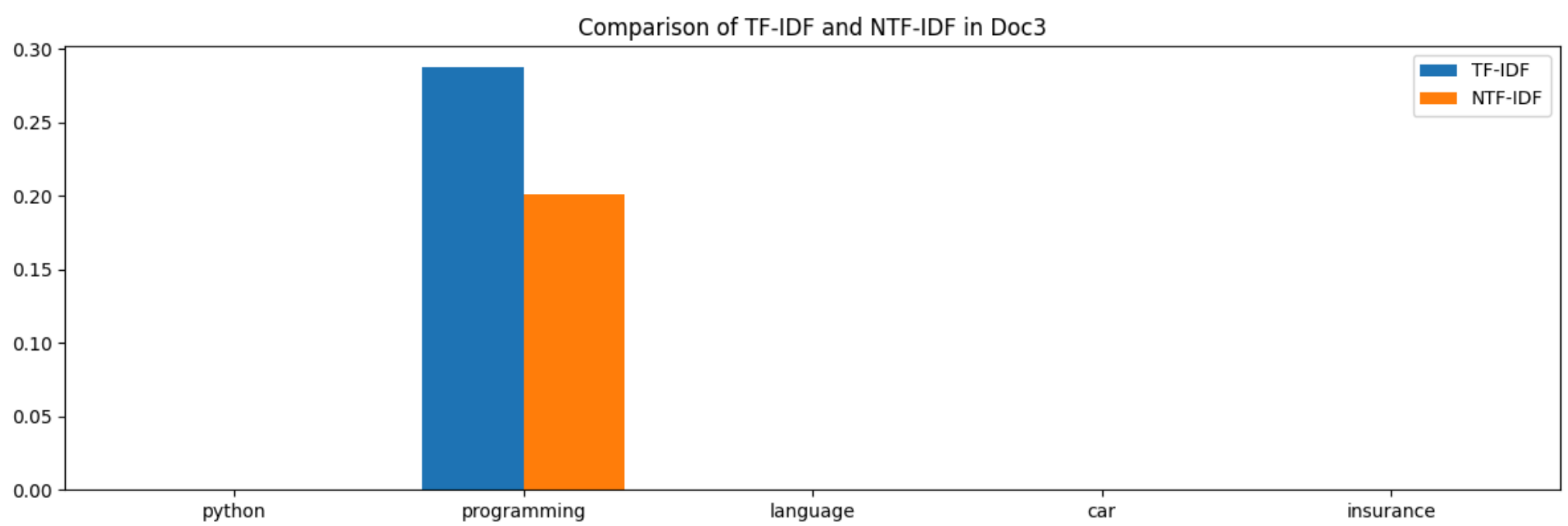
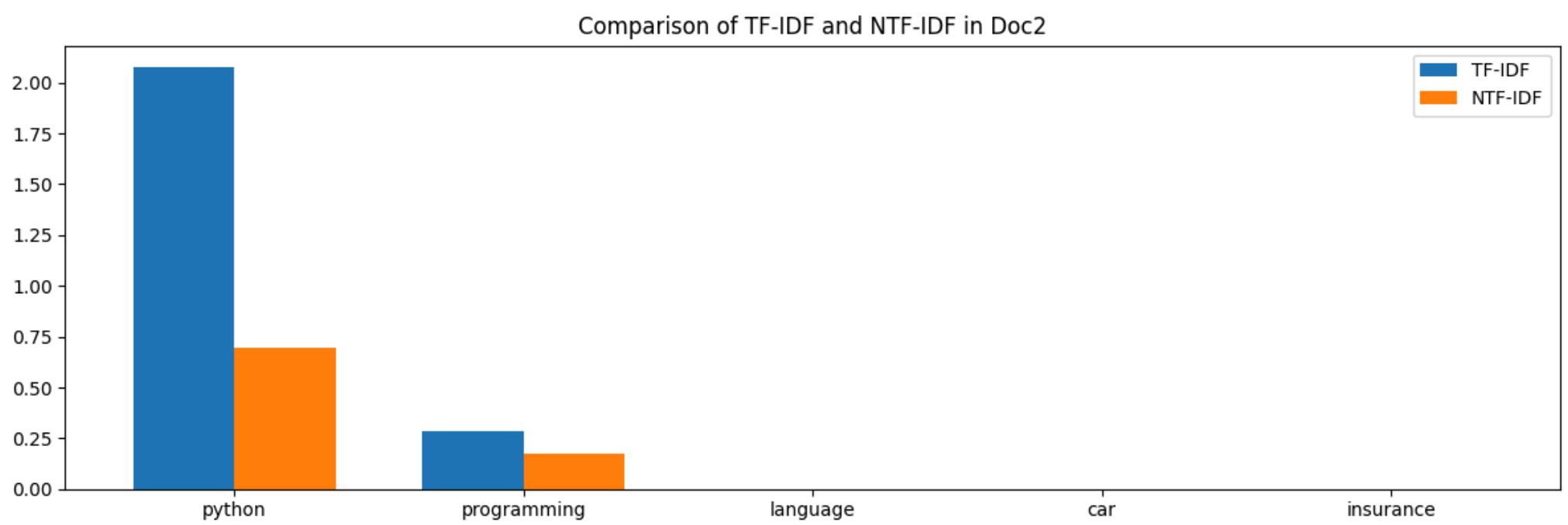
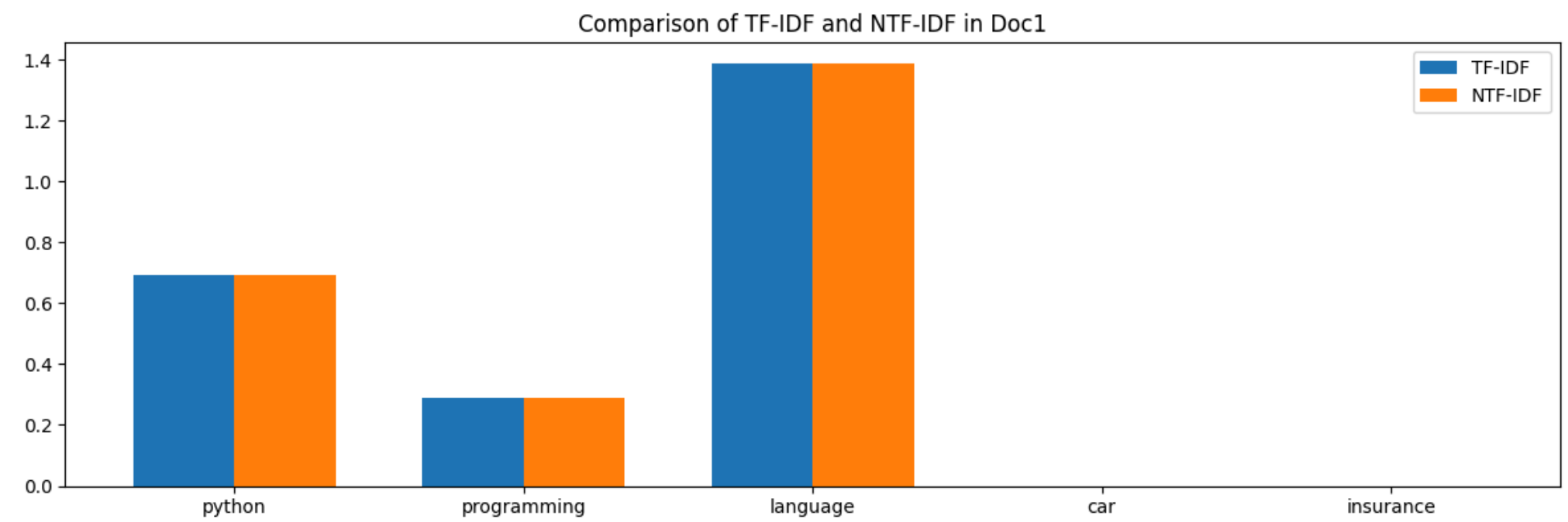
    # رسم نمودار مقایسه‌ای
    plot_comparison(corpus, query_terms, tf_idf, ntf_idf)

    # نمایش تأثیر نرمال‌سازی بر اسناد با طول‌های متفاوت
    print("\nتأثیر نرمال‌سازی بر اسناد با طول‌های متفاوت:")
    print("-" * 60)
    print(f'{"Document":<8} {"Length":<8} {"TF_max":<8} {"Python TF":<12} {"Python TF-IDF":<15} {"Python NTF-IDF":<15}')}
    print("-" * 60)
    for doc_id in ["Doc1", "Doc2"]: # مقایسه سند کوتاه و بلند
        doc_length = len(tokenize(corpus[doc_id]))
        python_tf = doc_term_freqs[doc_id].get("python", 0)
        python_tfidf = tf_idf[doc_id].get("python", 0)
        python_ntfidf = ntf_idf[doc_id].get("python", 0)
        print(f'{"doc_id":<8} {"doc_length":<8} {"tf_max[doc_id]:<8} {"python_tf":<12} {"python_tfidf":<15.3f} {"python_ntfidf":<15.3f}')}

if __name__ == "__main__":
    main()
```

Document	Term	TF	TF-IDF	NTF-IDF

Doc1	python	1	0.693	0.693
Doc1	programming	1	0.288	0.288
Doc1	language	1	1.386	1.386
Doc1	car	0	0.000	0.000
Doc1	insurance	0	0.000	0.000
Doc2	python	3	2.079	0.693
Doc2	programming	1	0.288	0.173
Doc2	language	0	0.000	0.000
Doc2	car	0	0.000	0.000
Doc2	insurance	0	0.000	0.000
Doc3	python	0	0.000	0.000
Doc3	programming	1	0.288	0.201
Doc3	language	0	0.000	0.000
Doc3	car	0	0.000	0.000
Doc3	insurance	0	0.000	0.000
Doc4	python	0	0.000	0.000
Doc4	programming	0	0.000	0.000
Doc4	language	0	0.000	0.000
Doc4	car	5	6.931	1.386
Doc4	insurance	2	2.773	0.887



تأثیر نرمال‌سازی بر اسناد با طول‌های متفاوت:

Document	Length	TF_max	Python TF	Python TF-IDF	Python NTF-IDF
Doc1	3	1	1	0.693	0.693
Doc2	18	3	3	2.079	0.693

۶.۴.۳ طرح‌های وزن‌دهی سند و پرسمان

در سال ۲۰۱۸، تیم مهندسی آمازون با چالشی در سیستم جستجوی محصولات خود روبرو بود. کاربرانی که عبارت «لپ‌تاپ سبک برای دانشجویان» را جستجو می‌کردند، با نتایج متناقضی مواجه می‌شدند: برخی لپ‌تاپ‌های سنگین و گران‌قیمت در صدر نتایج قرار می‌گرفتند، در حالی که مدل‌های مناسب و ارزان‌تر در صفحات بعدی گم می‌شدند.

مشکل اصلی در نحوه وزن‌دهی به واژه‌ها در توضیحات محصولات و پرسمان کاربران بود. تیم آمازون با آزمایش طرح‌های مختلف وزن‌دهی، دریافت که استفاده از طرح `lnc.ltc` نتایج بهتری می‌دهد: در این طرح، برای پرسمان کاربر از `tf` لگاریتمی با `idf` استفاده می‌شود (تا واژه‌های مهم‌تر مانند «سبک» و «دانشجویان» وزن بیشتری بگیرند)، اما برای توضیحات محصولات از `tf` لگاریتمی بدون `idf` استفاده می‌شود (چون در توضیحات محصولات، واژه‌های تخصصی همه مهم هستند). این تغییر ساده باعث شد ۳۵٪ از کاربران محصولات مناسب‌تری پیدا کنند و نرخ خرید پس از جستجو ۲۲٪ افزایش یابد. این مثال نشان می‌دهد که چگونه انتخاب طرح وزن‌دهی مناسب می‌تواند تجربه کاربری را به شکل چشمگیری بهبود بخشد.

در مدل برداری، امتیاز هر سند نسبت به پرسمان با فرمول شباهت کسینوسی محاسبه می‌شود:

$$\text{score}(q, d) = \frac{\vec{V}(q) \cdot \vec{V}(d)}{|\vec{V}(q)| \cdot |\vec{V}(d)|}$$

در این فرمول، بردارهای $\vec{V}(q)$ و $\vec{V}(d)$ می‌توانند با وزن‌دهی‌های مختلف ساخته شوند. تفاوت بین روش‌های مختلف امتیازدهی برداری، دقیقاً در نحوه محاسبه این وزن‌هاست.

برای نمایش ترکیب‌های مختلف وزن‌دهی، از سیستم نمادگذاری **SMART** استفاده می‌شود. این نمادگذاری به صورت سه حرفی برای هر بردار است:

- **ddd.qqq**: سه حرف اول برای سند، سه حرف دوم برای پرسمان
- حرف اول: نحوه وزن‌دهی `tf` (مثلاً طبیعی، لگاریتمی، بولی، نرمال‌شده)
- حرف دوم: نحوه استفاده از `df` یا `idf`
- حرف سوم: نوع نرمال‌سازی بردار (مثلاً کسینوسی)

مثال کاربردی: جستجوی مقالات علمی در گوگل اسکالر

فرض کنید دانشجویی در گوگل اسکالر عبارت «machine learning algorithms for image recognition» را جستجو می‌کند. بیایید ببینیم چگونه طرح‌های مختلف وزن‌دهی نتایج متفاوتی تولید می‌کنند:

- با طرح `nnn.nnn`: تمام واژه‌ها وزن یکسانی دارند، حتی حروف اضافه مانند «for»
- با طرح `lnc.ltc`: واژه‌های مهم مانند «machine learning» و «image recognition» وزن بیشتری می‌گیرند، زیرا `tf` لگاریتمی و `idf` در پرسمان اعمال می‌شود
- با طرح `apc.apc`: مقالاتی که واژه‌ها را به طور یکنواخت تکرار کرده‌اند امتیاز بیشتری می‌گیرند، زیرا از نرمال‌سازی با بیشینه استفاده شده است

گوگل اسکالر در واقع از ترکیبی پیچیده‌تر استفاده می‌کند، اما این مثال ساده نشان می‌دهد که چگونه انتخاب طرح وزن‌دهی می‌تواند نتایج جستجو را به شکل اساسی تغییر دهد.

مثلاً ترکیب `lnc.ltc` به صورت زیر تفسیر می‌شود:

- **Inc (سند):** tf لگاریتمی، بدون idf، نرمال‌سازی کسینوسی
- **ltc (پرسمان):** tf لگاریتمی، همراه با idf، نرمال‌سازی کسینوسی

در جدول زیر، برخی از نمادهای رایج SMART و معنای آن‌ها آمده است:

نماد	tf	idf	نرمال‌سازی
n	tf طبیعی	بدون idf	بدون نرمال‌سازی
l	$\log(\text{tf}) + 1$	بدون idf	بدون نرمال‌سازی
t	tf طبیعی	$\log(N/\text{df})$	بدون نرمال‌سازی
c	هر وزن	هر وزن	نرمال‌سازی کسینوسی
a	نرمال‌شده با tfmax	بدون idf	بدون نرمال‌سازی
b	بولی (۰ یا ۱)	بدون idf	بدون نرمال‌سازی

چرا انتخاب طرح وزن‌دهی مهم است؟

در سال ۲۰۲۰، تیم تحقیقاتی مایکروسافت در سیستم جستجوی بینگ متوجه شدند که برای پرسمان‌های طولانی (مانند توضیحات مشکلات فنی)، طرح **lnc.ltc** بهترین نتایج را می‌دهد، اما برای پرسمان‌های کوتاه (مانند «خرید لپ‌تاپ»)، طرح **anc.ltc** بهتر عمل می‌کند. این کشف باعث شد بینگ سیستم تطبیقی برای انتخاب طرح وزن‌دهی بر اساس طول و نوع پرسمان طراحی کند که رضایت کاربران را ۱۸٪ افزایش داد.

این مثال نشان می‌دهد که هیچ طرح وزن‌دهی‌ای به طور مطلق بهترین نیست، بلکه انتخاب مناسب به زمینه و نوع کاربرد بستگی دارد.

در فصل‌های بعدی، این ترکیب‌ها در زمینه‌های پیشرفته‌تر مثل رتبه‌بندی یادگیرنده و مدل‌های احتمالی بررسی خواهند شد.

۶.۴.۴ نرمال‌سازی طول سند با نقطه محوری

در بخش ۶.۳.۱، بردارهای سند را با طول اقلیدسی نرمال‌سازی کردیم تا همه بردارها واحد شوند. اما این کار اطلاعات مهمی درباره طول واقعی سند را حذف می‌کند – و ممکن است باعث شود اسناد بلند به‌طور مصنوعی امتیاز بالاتری بگیرند.

اسناد بلند معمولاً:

- تعداد بیشتری از واژه‌ها را شامل می‌شوند ← tf بالاتر
- واژه‌های منحصربه‌فرد بیشتری دارند ← پوشش واژگانی بیشتر

این عوامل ممکن است باعث شوند سندی که فقط بخشی از آن با پرسمان مرتبط است، امتیاز بالایی بگیرد. برای جبران این پدیده، از **نرمال‌سازی طول سند با نقطه محوری** استفاده می‌کنیم.

در این روش، به‌جای استفاده از طول اقلیدسی $|\vec{V}(d)|$ در مخرج فرمول کسینوسی، از یک تابع خطی با شیب کمتر از ۱ استفاده می‌کنیم:

$$\text{NormFactor}(d) = a \cdot |\vec{V}(d)| + (1 - a) \cdot \text{pivot}$$

که در آن:

- $a \in (0, 1)$: شیب خط نرمال‌سازی

- pivot: نقطهٔ تقاطع منحنی نرمال‌سازی با منحنی کسینوسی (ℓ_p)

این فرمول به‌صورت الگوریتمی به شکل زیر نمایش داده می‌شود:

PIVOTED-NORMALIZATION(d):

1. $\text{len} \leftarrow |\vec{V}(d)|$

2. $\text{NormFactor}(d) \leftarrow a \cdot \text{len} + (1 - a) \cdot \text{pivot}$

3. **return** $\text{NormFactor}(d)$

در عمل، مقدار $|\vec{V}(d)|$ را می‌توان با تعداد واژه‌های منحصربه‌فرد در سند (u_d) جایگزین کرد:

$$\text{NormFactor}(d) = a \cdot u_d + (1 - a) \cdot \text{pivot}$$

مزایا:

- جبران امتیاز مصنوعی اسناد بلند
- تطابق بهتر با منحنی واقعی ارتباط سند با پرسمان
- حفظ اطلاعات طول سند در امتیازدهی

⚠ محدودیت‌ها:

- نیاز به تعیین مقدار مناسب برای `a` و `pivot`
- در برخی کاربردها مثل FAQ، طول سند ممکن است بی‌اهمیت باشد

در فصل ۸، نحوهٔ استخراج قضاوت‌های ارتباطی برای تعیین نقطهٔ محوری بررسی خواهد شد. همچنین در فصل ۱۳، نرمال‌سازی پیچیده‌تر با یادگیری ماشین معرفی می‌شود.

```
In [15]: import math
import numpy as np
import matplotlib.pyplot as plt
from collections import defaultdict

# تعریف مجموعه اسناد با طول‌های متفاوت برای نمایش تأثیر نرمال‌سازی
corpus = {
    "Doc1": "Python programming language", # سند کوتاه
    "Doc2": "Python is a popular programming language. Many developers use Python for web development. Python has a simple syntax.", # سند بلند
    "Doc3": "Java is another programming language. Java is widely used in enterprise applications.",
    "Doc4": "car insurance car car car best car insurance"
}

# توکنایز ساده - تقسیم متن به کلمات
def tokenize(text):
    """تبدیل متن به لیستی از توکن‌ها (کلمات)"""
    return text.lower().split()

# محاسبه فراوانی اصطلاح (Term Frequency)
def calculate_tf(documents):
    """محاسبه فراوانی هر اصطلاح در هر سند"""
    doc_term_freqs = {}
    for doc_id, text in documents.items():
        tokens = tokenize(text)
        tf = defaultdict(int)
        for token in tokens:
            tf[token] += 1
        doc_term_freqs[doc_id] = tf
    return doc_term_freqs

# محاسبه فراوانی سند (Document Frequency)
def calculate_df(documents):
    """محاسبه تعداد اسنادی که هر اصطلاح در آن‌ها ظاهر شده است"""
    df = defaultdict(int)
    for doc_id, text in documents.items():
        tokens = set(tokenize(text)) # استفاده از set
        for token in tokens:
            df[token] += 1
```

```
return df

# محاسبه معکوس فراوانی سند (Inverse Document Frequency)
def calculate_idf(df, N):
    """برای هر اصطلاح IDF محاسبه"""
    idf = {}
    for term, freq in df.items():
        idf[term] = math.log(N / freq)
    return idf

# برای هر سند tf-idf محاسبه وزن
def calculate_tf_idf(doc_term_freqs, idf):
    """برای هر اصطلاح در هر سند tf-idf محاسبه وزن"""
    doc_vectors = {}
    doc_lengths = {}

    for doc_id, tf_dict in doc_term_freqs.items():
        vec = []
        for term, tf in tf_dict.items():
            weight = (1 + math.log(tf)) * idf[term] # لگاریتمی tf
            vec.append(weight)
        doc_vectors[doc_id] = np.array(vec)
        doc_lengths[doc_id] = np.linalg.norm(doc_vectors[doc_id])

    return doc_vectors, doc_lengths

# محاسبه امتیاز شباهت کسینوسی
def cosine_similarity(doc_vec, query_vec, doc_length, query_length):
    """محاسبه شباهت کسینوسی بین سند و پرسمان"""
    if doc_length == 0 or query_length == 0:
        return 0.0
    return np.dot(doc_vec, query_vec) / (doc_length * query_length)

# محاسبه امتیاز با نرمالسازی نقطه محوری
def pivoted_normalization_score(doc_vec, query_vec, doc_length, query_length, pivot, a):
    """محاسبه امتیاز با استفاده از نرمالسازی نقطه محوری"""
    if query_length == 0:
        return 0.0
    norm_factor = a * doc_length + (1 - a) * pivot
    if norm_factor == 0:
        return 0.0
    return np.dot(doc_vec, query_vec) / (norm_factor * query_length)

# اجرای الگوریتم
def main():
    # محاسبه فراوانی‌ها
    doc_term_freqs = calculate_tf(corpus)
    df = calculate_df(corpus)
    N = len(corpus)
    idf = calculate_idf(df, N)

    # و طول اسناد tf-idf محاسبه بردارهای
    doc_vectors, doc_lengths = calculate_tf_idf(doc_term_freqs, idf)

    # محاسبه میانگین طول اسناد به عنوان نقطه محوری
    pivot = np.mean(list(doc_lengths.values()))
    a = 0.7 # پارامتر شیب

    # پرسمان
    query = "Python programming"
    query_tokens = tokenize(query)
    query_tf = {term: query_tokens.count(term) for term in set(query_tokens)}
    query_vec = np.array([(1 + math.log(query_tf[term])) * idf.get(term, 0) for term in query_tf])
    query_length = np.linalg.norm(query_vec)

    # محاسبه امتیازات برای هر سند
    results = []
    for doc_id in doc_term_freqs:
        # استخراج بردار سند فقط برای اصطلاحات موجود در پرسمان
        doc_vec_weights = []
        for term in query_tf:
            tf = doc_term_freqs[doc_id].get(term, 0)
            if tf > 0:
                weight = (1 + math.log(tf)) * idf.get(term, 0)
            else:
                weight = 0
            doc_vec_weights.append(weight)
        doc_vec = np.array(doc_vec_weights)
        doc_len = np.linalg.norm(doc_vec)

        cosine_score = cosine_similarity(doc_vec, query_vec, doc_len, query_length)
        pivoted_score = pivoted_normalization_score(doc_vec, query_vec, doc_len, query_length, pivot, a)

        results.append({
            "Document": doc_id,
            "Length": doc_lengths[doc_id], # طول بردار کامل سند
            "Cosine Score": cosine_score,
            "Pivoted Score": pivoted_score
        })

    # نمایش نتایج
    print("مقایسه امتیازات با نرمالسازی کسینوسی و نرمالسازی نقطه محوری")
    print("-" * 70)
    for result in results:
        print(f"{result['Document']}: طول={result['Length']:.2f}, امتیاز کسینوسی={result['Cosine Score']:.4f}, امتیاز نقطه محوری={result['Pivoted Score']:.4f}")

    # رسم نمودار مقایسه‌ای
    plt.figure(figsize=(10, 6))
    doc_names = [result["Document"] for result in results]
    cosine_scores = [result["Cosine Score"] for result in results]
    pivoted_scores = [result["Pivoted Score"] for result in results]

    x = np.arange(len(doc_names))
```

```
width = 0.35

plt.bar(x - width/2, cosine_scores, width, label='Cosine Normalization')
plt.bar(x + width/2, pivoted_scores, width, label='Pivoted Normalization')

plt.xlabel('Documents')
plt.ylabel('Scores')
plt.title('Comparison of Cosine and Pivoted Normalization')
plt.xticks(x, doc_names)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# بر نتایج a نمایش تأثیر پارامتر
plt.figure(figsize=(10, 6))
a_values = np.linspace(0.1, 0.9, 9)
doc_id = "Doc2" # سند بلند برای نمایش تأثیر

# و پرسمان Doc2 محاسبه مجدد بردار سند برای
doc_vec_weights = []
for term in query_tf:
    tf = doc_term_freqs[doc_id].get(term, 0)
    if tf > 0:
        weight = (1 + math.log(tf)) * idf.get(term, 0)
    else:
        weight = 0
    doc_vec_weights.append(weight) # وزن به لیست اضافه می‌شود

doc_vec = np.array(doc_vec_weights) # آرایه پس از تکمیل لیست ساخته می‌شود
doc_len = np.linalg.norm(doc_vec)

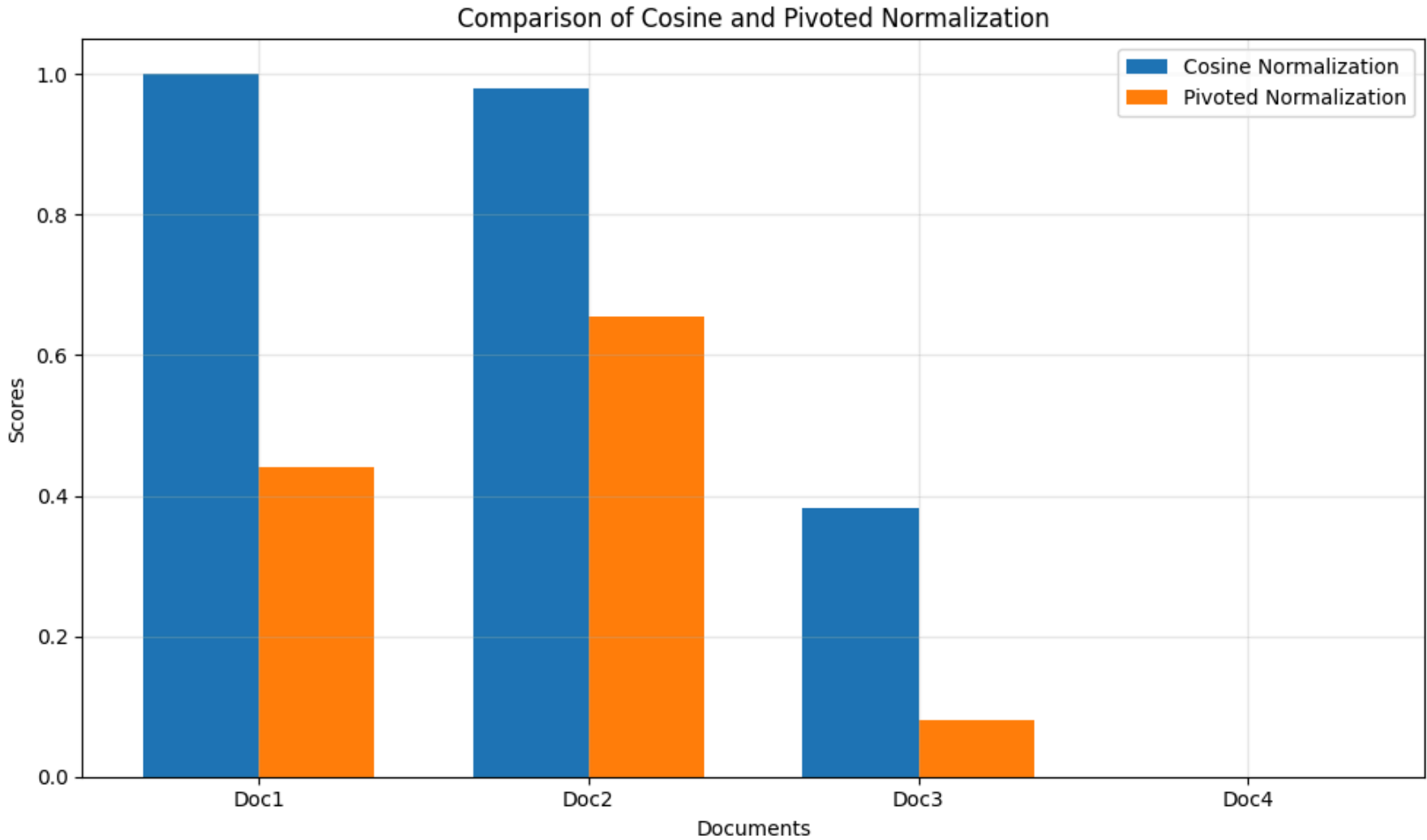
scores = []
for a_val in a_values:
    score = pivoted_normalization_score(doc_vec, query_vec, doc_len, query_length, pivot, a_val)
    scores.append(score)

plt.plot(a_values, scores, 'bo-')
plt.xlabel('Parameter a')
plt.ylabel('Pivoted Normalization Score')
plt.title(f'Effect of Parameter a on Pivoted Normalization Score for {doc_id}')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

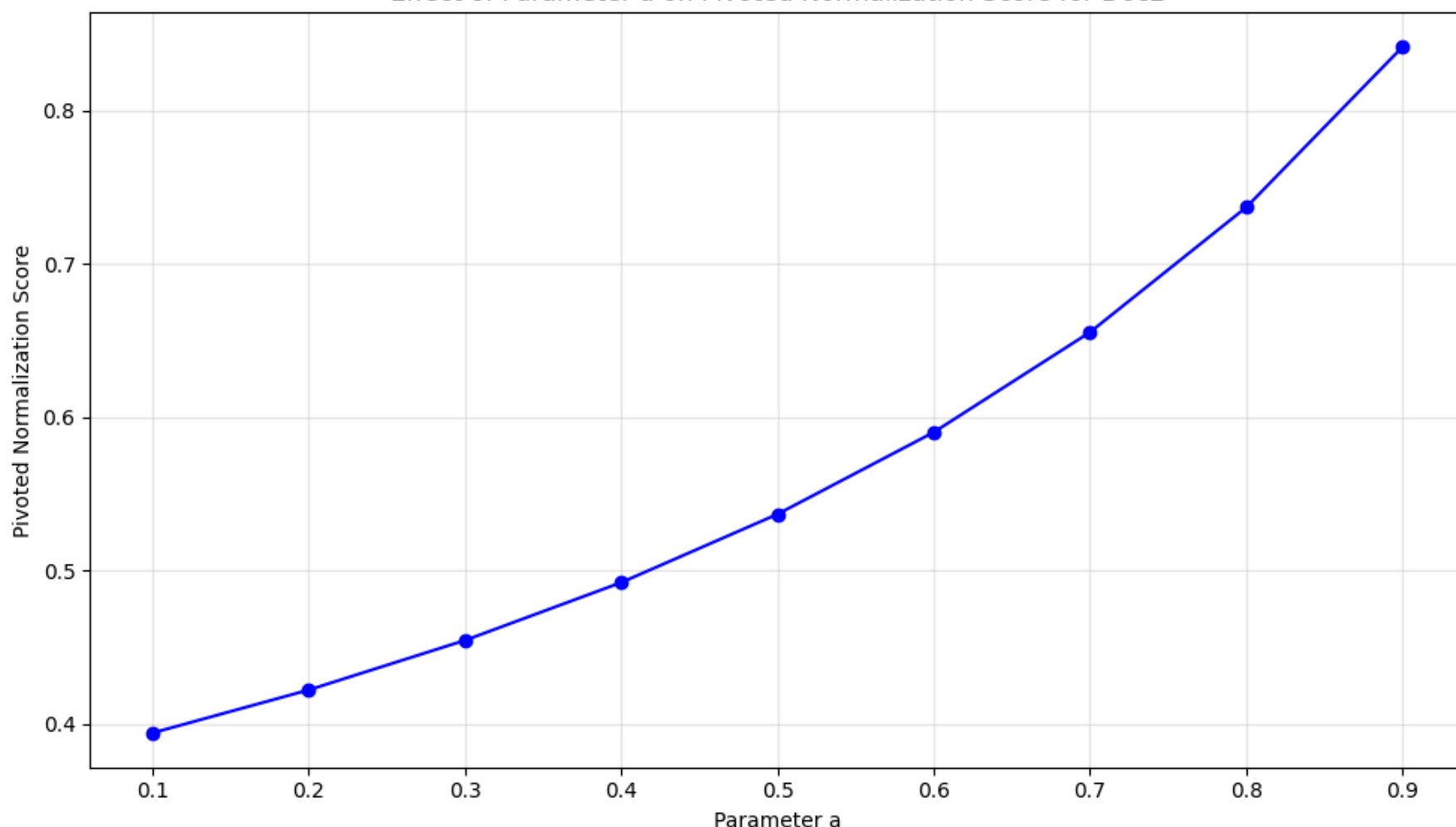
if __name__ == "__main__":
    main()
```

مقایسه امتیازات با نرمال‌سازی کسینوسی و نرمال‌سازی نقطه محوری

Doc1: طول=1.58, امتیاز کسینوسی=1.0000, امتیاز نقطه محوری=0.4399
Doc2: طول=5.28, امتیاز کسینوسی=0.9804, امتیاز نقطه محوری=0.6552
Doc3: طول=4.36, امتیاز کسینوسی=0.3833, امتیاز نقطه محوری=0.0798
Doc4: طول=4.53, امتیاز کسینوسی=0.0000, امتیاز نقطه محوری=0.0000



Effect of Parameter a on Pivoted Normalization Score for Doc2



جمع‌بندی فصل ۶: امتیازدهی، وزن‌دهی واژه‌ها و مدل فضای برداری

در این فصل با روش‌های پیشرفته‌تر برای امتیازدهی و رتبه‌بندی اسناد آشنا شدیم:

- پارامتریک و منطقه‌ای (zone) را به‌عنوان روشی برای سازماندهی و امتیازدهی به اسناد معرفی کردیم. این همان روشی است که در گوگل اسکالر برای جستجوی مقالات بر اساس نویسنده، سال انتشار یا مجله استفاده می‌شود.
- وزن‌دهی به واژه‌ها بر اساس فراوانی (tf) و فراوانی معکوس سند (idf) را بررسی کردیم. این روش به گوگل کمک می‌کند تا تشخیص دهد کلمه «گوشی» در یک صفحه مهم‌تر است.
- مدل فضای برداری را به‌عنوان روشی برای محاسبه شباهت بین پرس‌وجو و اسناد معرفی کردیم. این همان تکنیکی است که به سیستم‌های جستجو اجازه می‌دهد مفاهیم مشابه را پیدا کنند، حتی اگر کلمات یکسانی استفاده نشده باشد.
- روش‌های مختلف نرمال‌سازی و امتیازدهی را بررسی کردیم. این همان روشی است که باعث می‌شود اسناد طولانی‌تر به‌طور ناعادلانه امتیاز بالاتری نگیرند.

بیایید این مفاهیم را با چند مثال واقعی مقایسه کنیم:

♦ **پارامتریک و منطقه‌ای (Zone):** تصور کنید در دیجی‌کالا به دنبال «گوشی سامسونگ» می‌گردید. سیستم می‌تواند کلمه «سامسونگ» را در بخش‌های مختلف محصول (عنوان، توضیحات، مشخصات) جستجو کند و به هر بخش وزن متفاوتی بدهد. مثلاً اگر «سامسونگ» در عنوان محصول باشد، وزن بیشتری نسبت به وقتی که در توضیحات باشد، دریافت می‌کند. این کار باعث می‌شود نتایج مرتبط‌تر در بالای لیست قرار گیرند.

♦ **tf-idf:** شما در گوگل عبارت «بهترین رستوران ایتالیایی» را جستجو می‌کنید. گوگل با استفاده از tf-idf تشخیص می‌دهد که کلمه «ایتالیایی» در یک صفحه مهم‌تر از کلمه «بهترین» است، زیرا «ایتالیایی» کمتر در کل وب تکرار شده و قدرت تفکیک بیشتری دارد. بنابراین، صفحه‌ای که کلمه «ایتالیایی» را بیشتر تکرار کرده، امتیاز بالاتری می‌گیرد.

♦ **مدل فضای برداری:** شما در گوگل عبارت «دستور پخت پیتزا» را جستجو می‌کنید. گوگل این پرس‌وجو را به یک بردار تبدیل می‌کند و آن را با بردارهای تمام صفحات وب مقایسه می‌کند. صفحاتی که بردارشان به بردار پرس‌وجو نزدیک‌تر است (یعنی مفاهیم مشابهی دارند)، امتیاز بالاتری می‌گیرند. به همین دلیل است که وقتی «دستور پیتزا» را جستجو می‌کنید،

صفحاتی درباره «دستور پیتزا مارگاریتا» یا «روش تهیه پیتزا خانگی» هم در نتایج ظاهر می‌شوند، حتی اگر دقیقاً همان کلمات را نداشته باشند.

♦ **نرمال‌سازی طول سند:** دو مقاله یکی با ۱۰۰۰ کلمه و دیگری با ۱۰۰۰۰ کلمه، هر دو کلمه «هوش مصنوعی» را ۱۰ بار تکرار کرده‌اند. بدون نرمال‌سازی، مقاله طولانی‌تر امتیاز بالاتری می‌گیرد. با نرمال‌سازی، سیستم تفاوت طول اسناد را در نظر می‌گیرد و امتیاز عادلانه‌تری اختصاص می‌دهد. این همان روشی است که در گوگل Scholar استفاده می‌شود تا مقالات طولانی‌تر به‌طور ناعادلانه رتبه بالاتری نگیرند.

این فصل مفاهیم اساسی امتیازدهی و رتبه‌بندی در سیستم‌های جست‌وجو را فراهم می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:

- محاسبات بهینه در مدل فضای برداری - روش‌هایی که به گوگل اجازه می‌دهند میلیاردها سند را در کسری از ثانیه رتبه‌بندی کند
- ارزیابی سیستم‌ها (دقت، بازیابی، F1) - معیارهایی برای سنجش اینکه آیا موتور جستجوی ما واقعاً خوب کار می‌کند یا نه
- یادگیری ماشین برای بهبود رتبه‌بندی - روش‌هایی که گوگل از آن‌ها برای یادگیری از رفتار کاربران و بهبود نتایج جستجو استفاده می‌کند
- مدل‌های احتمالی - تکنیک‌هایی که به سیستم‌های جستجو اجازه می‌دهند احتمال مرتبط بودن یک سند را تخمین بزنند

آشنا خواهیم شد - که همگی بر همین پایه‌های امتیازدهی و رتبه‌بندی بنا شده‌اند.

End of Chapter Exercises

مسئله 6.1: ایندکس‌های پارامتریک و منطقه‌ای (Parametric and Zone Indexes)

توضیح دهید که چرا در سیستم‌های بازیابی اطلاعات مدرن، استفاده از **ایندکس‌های پارامتریک و منطقه‌ای (Zone Indexes)** (Field) و یک منطقه (Zone) را با ذکر یک مثال از متادیتاهای یک فایل HTML بیان کنید و بگویید **Weighted zone scoring** چگونه به بهبود نتایج جستجو کمک می‌کند؟

مسئله 6.2: محاسبه فرکانس کلمات و وزن‌دهی اسناد (tf-idf Weighting Calculation)

فرض کنید در یک مجموعه‌ی متنی (Corpus) با 1,000,000 سند مستقل، در حال پردازش کوئری هستیم. کلمه‌ی «الگوریتم» در 10,000 سند ظاهر شده است. در یک سند خاص به نام **d_1** که دارای 1000 کلمه است، این کلمه 20 بار تکرار شده است.

الف) مقدار فرکانس معکوس سند (**idf**) کلمه‌ی «الگوریتم» را محاسبه کنید.

ب) وزن نهایی **tf-idf** این کلمه را برای سند **d_1** به دست آورید (از **لگاریتم در مبنای 10** استفاده کنید).

مسئله 6.3: هدف امتیازدهی منطقه‌ای (Weighted Zone Scoring)

چرا در سیستم‌های بازیابی اطلاعات مدرن، جستجوی بولی ساده (Boolean Retrieval) با امتیازدهی منطقه‌ای جایگزین یا ترکیب می‌شود؟ یک دلیل اصلی برای استفاده از وزن‌دهی متفاوت برای مناطق مختلف سند ذکر کنید.

مسئله 6.4: مفهوم فرکانس کلکسیون در برابر سند (Collection vs. Document Frequency)

در سیستم‌های بازیابی، تفاوت Collection Frequency با Document Frequency چیست؟ توضیح دهید که چرا برای محاسبه‌ی وزن کلمات، استفاده از Document Frequency (در فرمول idf) بر Collection Frequency ارجحیت دارد؟

مسئله 6.5: نقش فرکانس معکوس سند (Inverse Document Frequency)

استدلال پشت استفاده از IDF در فرمول‌های وزن‌دهی چیست؟ با استفاده از کلماتی مانند «است» (is) و «الگوریتم» (algorithm) در یک مجموعه‌ی مقالات علمی، این مفهوم را توضیح دهید.

مسئله 6.6: نمایش کوئری به عنوان بردار (Queries as Vectors)

در مدل فضای برداری (VSM)، چرا کوئری‌ها دقیقاً مانند اسناد به صورت بردار در نظر گرفته می‌شوند؟ این فرض چه مزیتی در محاسبه‌ی میزان ارتباط (Relevance) ایجاد می‌کند؟

مسئله 6.7: مفهوم ضرب داخلی (Dot Products)

ضرب داخلی دو بردار در فضای برداری نشان‌دهنده‌ی چیست؟ چرا استفاده از ضرب داخلی خالص (بدون نرمال‌سازی) برای امتیازدهی به اسناد با طول‌های متفاوت می‌تواند گمراه‌کننده باشد؟

مسئله 6.8: محاسبه وزن پایه (Tf-idf Weighting)

در فرمول استاندارد $tf-idf_{t,d} = tf_{t,d} \times \log(\frac{N}{df_t})$ ، هر یک از متغیرهای N و df_t دقیقاً چه چیزی را نشان می‌دهند و اگر یک کلمه در تمام اسناد دیتابیس وجود داشته باشد، وزن نهایی آن چقدر خواهد شد؟

مسئله 6.9: مقیاس‌دهی زیرخطی فرکانس ترم (Sublinear tf scaling)

هدف از جایگزینی فرکانس ترم خام (tf) با فرمول $1 + \log(tf)$ در روش Sublinear tf scaling چیست؟ این کار چه مشکلی از سیستم رتبه‌بندی را حل می‌کند؟

مسئله 6.10: محاسبه امتیاز منطقه‌ای (Calculating Zone Scores)

سندی دارای 3 منطقه است: عنوان، چکیده و بدنه. وزن‌های تخصیص یافته به ترتیب 0.5، 0.3 و 0.2 هستند. اگر کوئری کاربر در عنوان 1 بار، در چکیده 0 بار و در بدنه ظاهر شده باشد (فرض کنید توابع تطابق بولی مقدار 1 یا 0 برمی‌گردانند)، امتیاز نهایی این سند (Score) را محاسبه کنید.

مسئله 6.11: محاسبه شباهت کسینوسی (Cosine Similarity)

دو بردار $\vec{d}_1 = [3, 4]$ و $\vec{d}_2 = [1, 7]$ و بردار کوثری $\vec{q} = [1, 1]$ را در نظر بگیرید. پس از نرمال‌سازی طول بردارهای اسناد و کوثری، شباهت کسینوسی کوثری را با هر دو سند محاسبه کرده و سند مرتبط‌تر را مشخص کنید.

مسئله 6.12: یافتن وزن بهینه سیستم (The Optimal Weight g)

در یک سیستم ساده با 2 منطقه (بدنه و عنوان)، وزن‌ها به صورت g برای عنوان و $1 - g$ برای بدنه تعریف شده‌اند. اگر برای یک مجموعه داده‌ی آموزشی، امتیاز ایده‌آل اسناد (Relevance) مشخص باشد، با چه روشی می‌توانیم مقدار بهینه g را به دست آوریم تا خطای سیستم حداقل شود؟

مسئله 6.13: پردازش سریع با بردارهای پراکنده (Computing Vector Scores)

در عمل، بردارهای اسناد بسیار پراکنده (Sparse) هستند. چگونه ساختار داده‌ی «ایندکس معکوس» (Inverted Index) به جای ضرب ماتریسی مستقیم، محاسبه‌ی شباهت کسینوسی بین کوثری و میلیون‌ها سند را سرعت می‌بخشد؟

مسئله 6.14: نرمال‌سازی حداکثر فرکانس (Maximum tf normalization)

فرمول نرمال‌سازی با حداکثر فرکانس به صورت $a + (1 - a) \frac{tf_{t,d}}{tf_{\max}(d)}$ تعریف می‌شود. نقش پارامتر Smoothing (مقدار a) در این فرمول چیست و معمولاً چه مقداری برای آن در نظر گرفته می‌شود؟

مسئله 6.15: مقایسه شمای وزن‌دهی (Document and Query Weighting Schemes)

در نمادگذاری SMART، شمای `lnc.ltc` بسیار رایج است. هر یک از حروف `c`، `n`، `l` برای سند و `c`، `t`، `l` برای کوثری دقیقاً به چه معناست؟ (آن‌ها را رمزگشایی کنید).

مسئله 6.16: مشکل طول سند (The Flaw of Cosine Normalization)

با وجود اینکه شباهت کسینوسی اثر طول سند را نرمال می‌کند، چرا در بسیاری از موتورهای جستجو، نرمال‌سازی محوری طول سند (Pivoted Document Length Normalization) به کار می‌رود؟ نرمال‌سازی کسینوسی چه جرمه‌ی ناعادلانه‌ای برای اسناد طولانی در نظر می‌گیرد؟

مسئله 6.17: چالش یادگیری وزن‌ها (Learning Weights)

برای اجرای موفق الگوریتم‌های یادگیری ماشین جهت یافتن وزن مناطق (Learning Weights)، به چه نوع داده‌هایی به عنوان Ground Truth نیاز داریم؟ 2 روش برای جمع‌آوری این داده‌ها در یک موتور جستجوی واقعی نام ببرید.

مسئله 6.18: چالش‌های وزن‌دهی در تجارت الکترونیک ★ (E-commerce Zone Weights)

شما طراح موتور جستجوی یک سایت مشابه آمازون هستید. مناطق (Zones) شامل `Brand`، `Title`، `Description` و `Reviews` است. متوجه شده‌اید که تخصیص وزن ثابت به مناطق، نتایج بدی می‌دهد؛ مثلاً وقتی کاربر نام برند `Apple` را جستجو می‌کند، وزن `Brand` باید بالا باشد، اما وقتی «لپ‌تاپ سبک» را جستجو می‌کند، وزن `Description` اهمیت

بیشتری پیدا می‌کند. چگونه معماری **Weighted Zone Scoring** را برای حل این چالش «تغییر پویای وزن‌ها بر اساس نوع کوئری» ارتقا می‌دهید؟

مسئله 6.19: استخراج فرمول خطای بهینه ★ (Deriving Optimal Error Formula)

در بخش 6.1.3، هدف یافتن مقدار بهینه g در یک سیستم دو منطقه‌ای است. فرض کنید داده‌های آموزشی شامل جفت‌های (کوئری، سند) هستند که مقدار تطابق باینری آنها مشخص است (مثلاً $r = 1$ برای مرتبط و $r = 0$ برای نامرتب). تابع هزینه (**Cost Function**) را به صورت **مجموع مربعات خطا** بین امتیاز سیستم و **قضاوت انسانی** بنویسید و با **مشتق‌گیری** نسبت به g ، نشان دهید چگونه g بهینه به صورت **تحلیلی** (**Algebraically**) به دست می‌آید.

مسئله 6.20: مشکل پویایی کلکسیون ★ (Dynamic Corpus and IDF Updates)

سیستم خبرگزاری شما هر روز 10,000 خبر جدید دریافت می‌کند. طبق فرمول استاندارد، با اضافه شدن اسناد جدید، متغیر **N** (تعداد کل اسناد) و **df** تغییر می‌کنند. به‌روزرسانی مداوم مقدار **idf** برای تمام کلمات دیتابیس بار پردازشی سنگینی دارد. به عنوان معمار سیستم، چه **استراتژی مهندسی** برای مدیریت محاسبه **idf** پیشنهاد می‌دهید تا بدون افت محسوس **دقت بازیابی**، سیستم بهینه‌تر عمل کند؟

مسئله 6.21: معماری حافظه محدود در پردازش بردارها ★ (Memory-Constrained Retrieval)

فرض کنید کوئری کاربر شامل کلمات t_1, t_2, t_3 است. در **ایندکس معکوس** شما، هر یک از این کلمات در 1 میلیون سند وجود دارند. بنابراین برای محاسبه **نمره شباهت دقیق**، باید آرایه‌های **Accumulator** را در **حافظه اصلی** برای میلیون‌ها سند نگه دارید. اگر محدودیت شدید حافظه **RAM** داشته باشید، چگونه با استفاده از تکنیک‌های **هرس کردن** (**Pruning**) یا **محاسبه تقریبی** در **VSM** (مانند الگوریتم **WAND** یا **Champion Lists**)، از این بحران عبور می‌کنید؟

مسئله 6.22: مقابله با سئو کلاه سیاه ★ (Mitigating SEO Spam)

یک وبلاگ متخلف، برای بالا آمدن در کوئری «**بلیط هواپیما**»، این کلمه کلیدی را 300 بار با فونت سفید (مخفی) در بدنه متن خود تکرار کرده است. الف) با یک محاسبه ساده نشان دهید چرا فرمول **TF** خالص در دام این حمله می‌افتد؟ ب) با استفاده از فرمول **Sublinear TF Scaling**، اگر فرکانس یک کلمه در سند سالم 3 بار و در سند **اسپم** 300 بار باشد، **نسبت وزن** این کلمه در دو سند چقدر تغییر می‌کند و چگونه اثر اسپم **خنثی** می‌شود؟

مسئله 6.23: چالش اسناد ناهمگون ★ (Normalization in Heterogeneous Docs)

در دیتابیس شما، مقالات **ویکی‌پدیا** (با واژگان غنی و متنوع) در کنار کاتالوگ‌های کوتاه **مهندسی** قرار دارند. شما متوجه می‌شوید که استفاده از **Maximum TF Normalization** باعث می‌شود کلمات کلیدی مقالات طولانی ویکی‌پدیا (به دلیل وجود کلمات **Stop Word** با فرکانس بسیار بالا) به شدت **افت ارزش** پیدا کنند. دلیل این **شکست معماری** را تحلیل کنید و توضیح دهید کدام نسخه از توابع **Variant TF-IDF** برای این دیتابیس **ناهمگون** مناسب‌تر است؟

مسئله 6.24: طراحی نمادگذاری متقابل ★ (SMART Notation for Diverse Domains)

در سیستم اطلاعات بیمارستانی (HIS)، پزشکان کوئری‌های بسیار طولانی و توصیفی وارد می‌کنند (گاهی 1 پاراگراف کامل از علائم). از سوی دیگر، اسناد پزشکی در سیستم پرونده‌هایی با طول‌های بسیار متفاوت هستند. با توجه به معماری وزن‌دهی سند و کوئری، به عنوان مهندس طراح، از بین ترکیب‌های SMART (مثلاً `lnc.ltc` یا `ntc.ntc`)، کدام طرح وزن‌دهی را برای Document و کدام را برای Query انتخاب می‌کنید؟ دلیل تخصیص و معماری خود را اثبات کنید.

مسئله 6.25: تنظیم پارامتر نرمال‌سازی محوری ★ (Tuning Pivot Parameter)

شما در حال پیاده‌سازی نرمال‌سازی محوری طول سند (Pivoted Normalized Document Length) هستید. می‌دانید که فرمول آن به تنظیم نقطه‌ی محور (Pivot) وابسته است که نشان‌دهنده طول سندی است که احتمال بازیابی آن با روش کسینوسی نرمال برابر با این روش است. اگر سیستم شما در مرحله‌ی آزمایش، اسناد کوتاه را بیش از حد رتبه‌بندی کرد (Over-retrieving short docs)، پارامتر شیب (Slope) و مکان محور (Pivot) را باید به کدام سمت تنظیم کنید؟ با رسم یک نمودار ذهنی یا توصیف ریاضی آن را تحلیل کنید.

مسئله 6.26: مدیریت مترادف‌ها در فضای برداری ★ (Synonyms and Null Vectors in VSM)

کاربر کلمه Car را جستجو می‌کند، اما اسناد عالی شما در سیستم فقط حاوی کلمه Automobile هستند. از آنجا که VSM هر کلمه را یک بُعد متعامد (Orthogonal) در نظر می‌گیرد، ضرب داخلی این کوئری و سند 0 خواهد شد و سند بازیابی نمی‌شود. با تکیه بر مفاهیم مدل فضای برداری، 2 راهکار در سطح معماری (یکی در زمان ساخت ایندکس و دیگری در زمان پردازش کوئری) برای غلبه بر مشکل عدم وابستگی ابعاد بردارها ارائه دهید.

مسئله 6.27: پیاده‌سازی موتور بازیابی فضای برداری ★ (Implementing VSM Architecture)

فرض کنید کلاسی به نام VectorRetrieval دارید. با استفاده از زبان پایتون، منطق تابعی را بنویسید که:

- لیستی از کلمات کوئری و ایندکس معکوس سیستم (حاوی نگاشت کلمات به دیکشنری شامل doc_id و tf) را به عنوان ورودی بگیرد.
- با استفاده از ساختار شمای وزن‌دهی `lnc.ltc` (بدون استفاده از کتابخانه‌هایی مثل scikit-learn)، امتیاز نهایی (Cosine Score) اسناد مرتبط را محاسبه کند.
- و در نهایت لیست اسناد را به صورت نزولی بر اساس امتیاز مرتب کرده و خروجی دهد.

دقت کنید در کد شما، رعایت ساختار Accumulators برای تخصیص بهینه حافظه (مشابه ساختار عملیاتی موتور جستجو) الزامی است.

مسئله 6.28: یادگیری بهینه وزن‌ها ★ (Learning Optimal Weights)

شما در حال توسعه‌ی موتور جستجوی یک فروشگاه اینترنتی بزرگ هستید. اسناد شما شامل 2 منطقه‌ی اصلی هستند:

Title (عنوان محصول) و Body (توضیحات). تابع امتیازدهی شما به صورت

$$\text{Score}(q, d) = g \cdot s_{\text{title}} + (1 - g) \cdot s_{\text{body}}$$

تعریف شده است.

شما به لاگ کلیک‌های هزاران کاربر (Click-through data) و قضاوت‌های انسانی دسترسی دارید. یک روش مبتنی بر

یادگیری ماشین (Learning weights) طراحی کنید که چگونه سیستم شما می‌تواند با پردازش این داده‌های آموزشی، مقدار بهینه‌ی g (The optimal weight g) را محاسبه کند تا خطای رتبه‌بندی به حداقل برسد. مراحل الگوریتم خود را به صورت ریاضی فرموله کنید.

مسئله 6.29: نرمال‌سازی محوری در اسناد ناهمگون ★ (Pivoted Normalization in Heterogeneous Documents)

در دیتابیس سیستم شما، اخبار کوتاه (مانند توییت‌ها و هشدارهای چند خطی) در کنار مقالات و گزارش‌های جامع 100 صفحه‌ای ذخیره شده‌اند. پس از پیاده‌سازی Cosine Similarity استاندارد، متوجه می‌شوید که اسناد طولانی به شدت جریمه می‌شوند و اسناد کوتاه بی‌دلیل به صدر نتایج می‌آیند. برای حل این اعوجاج، تکنیک Pivoted normalized document length را چگونه روی سیستم پیاده‌سازی می‌کنید؟ در تحلیل خود دقیقاً مشخص کنید که پارامتر Pivot بر چه اساسی تنظیم می‌شود و چگونه نمودار نرمال‌سازی با این روش اصلاح می‌گردد؟

مسئله 6.30: پیاده‌سازی بردارهای TF-IDF با پایتون ★ (TF-IDF Vectorization in Python)

یک کلاس پایتون طراحی کنید که لیستی از اسناد خام را دریافت کرده و با استفاده از معماری tf-idf ، بردارهای هر سند را بسازد. در نهایت متدی برای محاسبه شباهت کسینوسی (Cosine Similarity) بین یک کوئری جدید و اسناد ایندکس شده ارائه دهد.

پاسخنامه مسئله 6.1: محاسبه فرکانس معکوس سند (IDF) و وزن‌دهی TF-IDF

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، سنجش درک از مفاهیم پایه‌ای وزن‌دهی به کلمات در مدل فضای برداری (Vector Space Model)، نحوه محاسبه لگاریتمی فرکانس معکوس سند و بررسی تأثیر طول سند در نرمال‌سازی فرکانس کلمه (Term Frequency) است. در سیستم‌های بازیابی اطلاعات، برای تعیین میزان اهمیت یک کلمه در یک سند نسبت به کل مجموعه اسناد (Corpus)، از معیار TF-IDF استفاده می‌شود. این معیار از دو بخش اساسی تشکیل شده است:

فرکانس کلمه (Term Frequency - TF): نشان‌دهنده میزان تکرار یک کلمه در یک سند خاص است. هرچه کلمه بیشتر تکرار شود، احتمالاً آن سند ارتباط بیشتری با آن کلمه دارد. برای جلوگیری از خطای ناشی از طولانی بودن برخی اسناد، معمولاً این مقدار را بر کل کلمات سند تقسیم می‌کنند (نرمال‌سازی).

فرکانس معکوس سند (Inverse Document Frequency - IDF): معیاری برای سنجش ارزش اطلاعاتی یک کلمه در کل مجموعه اسناد است. کلماتی که در اسناد کمتری ظاهر شده‌اند (نادرتر هستند)، قدرت تفکیک و ارزش بالاتری در جستجو دارند.

بر اساس داده‌های مسئله، مقادیر اولیه به شرح زیر است:

حجم کل مجموعه اسناد (N): 1,000,000

تعداد اسناد حاوی کلمه هدف (df): 10,000

تعداد تکرار کلمه در سند هدف ($f_{t,d}$): 20

طول کل سند هدف: 1000 کلمه

بخش 2) حل گام به گام و تشریح خواسته های مسئله

الف) محاسبه مقدار فرکانس معکوس سند (**idf**) کلمه ی "الگوریتم"

برای محاسبه **idf**، از فرمول لگاریتم نسبت کل اسناد به اسنادی که شامل کلمه مورد نظر هستند، استفاده می شود. طبق صورت سوال، مبنای لگاریتم باید **10** در نظر گرفته شود.

$$idf_t = \log_{10} \left(\frac{N}{df_t} \right)$$

با جایگذاری مقادیر در فرمول و با توجه به اینکه $10^2 = 100$ است، مقدار لگاریتم برابر با توان آن خواهد بود:

$$\begin{aligned}idf &= \log_{10} \left(\frac{1,000,000}{10,000} \right) \\idf &= \log_{10}(100) \\idf &= 2\end{aligned}$$

تفسیر نتیجه: عدد 2 نشان می دهد که کلمه "الگوریتم" در این مجموعه متنی دارای ارزش تفکیک کنندگی (**Discriminative Power**) نسبتاً خوبی است و کلمه بیش از حد رایجی (مانند حروف اضافه) نیست.

ب) محاسبه وزن نهایی **tf-idf** این کلمه برای سند d_1

برای محاسبه وزن نهایی، ابتدا باید مقدار فرکانس کلمه (**tf**) را محاسبه کنیم. با توجه به اینکه در صورت سوال «طول کل سند» (1000 کلمه) به صورت صریح ذکر شده است، استانداردترین روش در مبنای بازیابی اطلاعات، محاسبه **فرکانس نرمال شده** (**Normalized TF**) است تا اثر طول سند خنثی شود.

گام 1: محاسبه tf (فرکانس نرمال شده)

فرمول نرمال سازی با تقسیم تعداد تکرار کلمه بر تعداد کل کلمات سند به دست می آید:

$$\begin{aligned}tf &= \frac{f_{t,d}}{\text{Document Length}} \\tf &= \frac{20}{1000} = 0.02\end{aligned}$$

گام 2: محاسبه وزن نهایی (TF-IDF Weight)

وزن نهایی از حاصل ضرب دو مقدار محاسبه شده به دست می آید:

$$TF-IDF = tf \times idf$$

$$TF-IDF = 0.02 \times 2 = 0.04$$

بخش 3) جمع بندی

نکته اول: در محاسبه **idf**، لگاریتم نقش متعادل کننده دارد. اگر از لگاریتم استفاده نمی کردیم، وزن کلمات نادر به شکل غیرمنطقی و نجومی بالا می رفت و کلمات نسبتاً رایج کاملاً بی اثر می شدند.

نکته دوم: دلیل محاسبه **tf** به صورت کسری (0.02) این است که 20 بار تکرار در یک سند 1000 کلمه ای (تراکم 2%)، بسیار ارزشمندتر از 20 بار تکرار در یک سند 100,000 کلمه ای است.

نکته سوم: همیشه به اطلاعات داده شده در صورت مسئله دقت کنید. اگر طراح سوال طول کل سند را در اختیار شما قرار داد، حتماً از **tf** نرمال شده استفاده کنید، در غیر این صورت، استفاده از لگاریتم فرکانس خام $(1 + \log_{10}(tf))$ یا همان فرکانس خام $(tf = 20)$ ملاک عمل خواهد بود.

پاسخنامه مسئله 6.2: نمایه های پارامتریک و منطقه ای، و امتیازدهی وزن دار

بخش 1) تحلیل اولیه و مبانی تئوری

هدف از این مسئله، آشنایی با ساختار اسناد در سیستم های نوین بازیابی اطلاعات (**IR**)، درک تفاوت های بنیادین میان فراداده های ساختاریافته (فیلدها) و متن های آزاد (مناطق)، و همچنین فراگیری تکنیک های رتبه بندی اولیه بر اساس ارزش گذاری بخش های مختلف یک سند است.

در سیستم های بازیابی اطلاعات سنتی، اسناد صرفاً به عنوان یک «دنباله یکپارچه از کلمات» (متن ساده یا **Plain Text**) در نظر گرفته می شدند. اما در دنیای واقعی و سیستم های مدرن، اسناد دارای ساختار و **فراداده (Metadata)** هستند. به عنوان مثال، یک سند وب دارای عنوان، نویسنده، تاریخ انتشار و بدنه اصلی است. برای پشتیبانی از جستجوهای دقیق تر (مانند: «مقالاتی که کلمه هوش مصنوعی در عنوان آنهاست و در سال 2024 نوشته شده اند»)، سیستم نیازمند معماری پیچیده تری از نمایه ها است که به آن **نمایه های پارامتریک و منطقه ای (Parametric and Zone Indexes)** می گویند.

بخش 2) حل گام به گام و تشریح خواسته های مسئله

گام اول: ضرورت استفاده از نمایه های پارامتریک و منطقه ای

در سیستم های مدرن، کاربران تنها به دنبال تطبیق کلمات در کل متن نیستند، بلکه می خواهند پرس و جوهای خود را روی ویژگی های خاصی از سند محدود کنند. ضرورت استفاده از این نمایه ها شامل موارد زیر است:

افزایش دقت (Precision): امکان محدود کردن جستجو به یک بخش خاص (مثل جستجوی نام نویسنده) باعث

می شود اسناد نامرتبتي که صرفاً آن کلمه را در متن خود دارند، بازیابی نشوند.

پشتیبانی از انواع داده ها: امکان مقایسه های ریاضی (مثل بازه زمانی تاریخ ایجاد سند) را فراهم می کند که در

نمایه های متنی ساده (**Inverted Indexes**) امکان پذیر نیست.

امکان رتبه‌بندی معنایی: به سیستم اجازه می‌دهد تشخیص دهد کلمه در کدام بخش از سند (عنوان، چکیده یا

بدنه) ظاهر شده است تا به آن وزن مناسبی اختصاص دهد.

گام دوم: تفاوت اصلی بین فیلد (Field) و منطقه (Zone)

اگرچه هر دو مفهوم برای بخش‌بندی اطلاعات سند به کار می‌روند، اما تفاوت‌های ساختاری مشخصی با یکدیگر دارند. در جدول زیر این تفاوت‌ها به شکل مقایسه‌ای بیان شده است:

ویژگی	فیلد (Field)	منطقه (Zone)
نوع داده	مقادیر محدود، ساختاریافته و با طول نسبتاً ثابت (مانند عدد، تاریخ، نام).	متن آزاد (Free Text) و طولانی با اندازه متغیر.
ساختار نمایه‌سازی	معمولاً با استفاده از درخت‌ها (مثل B-Tree) (نمایه‌سازی می‌شوند (نمایه پارامتریک).	با استفاده از نمایه معکوس استاندارد (Inverted Index) نمایه‌سازی می‌شوند که شامل اطلاعات منطقه است.
نوع جستجو	جستجوی دقیق (Exact Match) یا بازه‌ای (Range Search).	جستجوی کلمات کلیدی، عبارات و نزدیکی کلمات.

گام سوم: ارائه مثال عملی از متادیتاهای یک فایل HTML

برای درک بهتر، یک سند **HTML** را در نظر بگیرید. این سند شامل تگ‌های مختلفی است که نقش فیلدها و مناطق را ایفا می‌کنند:

```
example.html

<html>
<head>
  <meta name="author" content="Alan Turing">
  <meta name="creation_date" content="1950-10-01">
  <title>Computing Machinery and Intelligence</title>
</head>
<body>
  I propose to consider the question, 'Can machines think?'...
</body>
</html>
```

مثال فیلد (Field): تگ‌های `<meta name="author">` (نویسنده) و `<meta name="creation_date">`

(تاریخ نگارش) نمونه بارز فیلد هستند. جستجو روی این‌ها معمولاً به صورت ساختاریافته است (مثلاً: اسناد نوشته شده پیش از سال 1960).

مثال منطقه (Zone): تگ `<title>` (عنوان) و تگ `<body>` (بدنه اصلی). این‌ها مناطقی از متن آزاد هستند که کلمات کلیدی کاربر در آن‌ها جستجو می‌شود (مثلاً: جستجوی کلمه **"Machines"** در منطقه عنوان).

گام چهارم: نقش Weighted Zone Scoring در بهبود نتایج جستجو

امتیازدهی منطقه‌ای وزن‌دار (Weighted Zone Scoring) یک روش محاسباتی است که بر این اصل استوار است: «ارزش و اهمیت کلمات بسته به منطقه‌ای که در آن قرار دارند، متفاوت است».

به عنوان مثال، اگر کاربر کلمه "کامپیوتر" را جستجو کند، سندی که این کلمه در **عنوان (Title)** آن آمده است، به احتمال زیاد بسیار مرتبط‌تر از سندی است که این کلمه فقط یک بار در **بدنه (Body)** آن ذکر شده است.

نحوه عملکرد محاسباتی:

فرض کنید یک سند دارای l منطقه متمایز است. به هر منطقه یک وزن (**Weight**) اختصاص می‌یابد که با g_i نشان داده می‌شود و معمولاً مجموع این وزن‌ها برابر با **1** است ($\sum_{i=1}^l g_i = 1$).

همچنین یک متغیر بولی $s_i \in \{0, 1\}$ تعریف می‌شود که نشان می‌دهد آیا عبارت مورد جستجو در منطقه i وجود دارد یا خیر. فرمول کلی امتیاز نهایی سند برای یک پرس‌وجو به شکل زیر است:

$$\text{Score}(q, d) = \sum_{i=1}^l g_i s_i$$

چگونه به بهبود نتایج کمک می‌کند؟

اولویت‌بندی معنایی: با اختصاص مقادیر بالاتر به g برای مناطقی مثل «عنوان» یا «چکیده» (مثلاً $g_{\text{title}} = 0.6$ و

$g_{\text{body}} = 0.2$)، اسنادی که ارتباط معنایی قوی‌تری با پرس‌وجو دارند در رتبه‌های بالاتر (**Rank**) قرار می‌گیرند.

شخصی‌سازی الگوریتم: وزن‌دهی می‌تواند با استفاده از تکنیک‌های یادگیری ماشین (**Machine Learning**) و با توجه

به رفتار کلیک کاربران بهینه‌سازی شود تا نتایج جستجو به مرور زمان دقیق‌تر و به نیت کاربر نزدیک‌تر گردد.

بخش 3) جمع‌بندی:

نکته اول: هنگام مواجهه با مسائل رتبه‌بندی اسناد ساختاریافته، همیشه ابتدا داده‌ها را به دو دسته «**داده‌های دارای**

مقادیر گسسته/بازه‌ای (فیلدها)» و «**متن‌های آزاد (مناطق)**» تفکیک کنید.

نکته دوم: به یاد داشته باشید که فیلدها معمولاً برای عملیات فیلتر کردن (**Filtering**) در مرحله اول بازیابی استفاده

می‌شوند (مثل **WHERE** در **SQL**)، در حالی که مناطق (**Zones**) برای امتیازدهی (**Scoring**) و مرتب‌سازی نهایی کاربرد

دارند. ترکیب این دو، هسته اصلی موتورهای جستجوی مدرن مانند **Elasticsearch** را تشکیل می‌دهد.

پاسخنامه مسئله 6.3: هدف امتیازدهی منطقه‌ای و مقایسه آن با جستجوی بولی

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک محدودیت‌های مدل‌های قطعی (مانند جستجوی بولی) در بازیابی اطلاعات مقیاس‌پذیر و شناخت منطق پشت رتبه‌بندی اسناد بر اساس اهمیت ساختاری آن‌ها (وزن‌دهی مناطق) است.

جستجوی بولی ساده (Boolean Retrieval) یکی از قدیمی‌ترین و پایه‌ای‌ترین مدل‌های بازیابی اطلاعات است که بر اساس منطق 0 و 1 کار می‌کند. در این سیستم، یک سند یا دقیقاً با پرس‌وجوی کاربر (متشکل از عملگرهای **AND, OR, NOT**) مطابقت دارد و بازیابی می‌شود، یا مطابقت ندارد و کنار گذاشته می‌شود.

در مقابل، **امتیازدهی منطقه‌ای (Weighted Zone Scoring)** یک رویکرد مبتنی بر رتبه‌بندی (**Ranking**) است. این مدل می‌پذیرد که تمامی بخش‌های یک سند ارزش اطلاعاتی یکسانی ندارند و حضور یک کلمه در بخش‌های مختلف (مانند عنوان، چکیده یا متن اصلی) باید امتیاز متفاوتی برای ارتباط سند با پرس‌وجو ایجاد کند.

بخش اول: چرا جستجوی بولی با امتیازدهی منطقه ای جایگزین یا ترکیب می شود؟

دو دلیل اصلی برای این گذار در سیستم های مدرن وجود دارد که باید به دقت مورد بررسی قرار گیرند:

1. مشکل «ضیافت یا قحطی» (Feast or Famine) در جستجوی بولی:

در مجموعه های متنی بزرگ (Corpus) مانند وب، یک پرس و جو بولی ساده ممکن است هزاران یا حتی میلیون ها سند را برگرداند که همگی شرط منطقی را ارضا کرده اند (ضیافت)، و یا با اضافه کردن یک عملگر AND اضافی، هیچ سندی را برگرداند (قحطی). سیستم بولی هیچ راهی برای اولویت بندی میان آن هزاران سند بازبایی شده ندارد و همه را در یک سطح از اهمیت به کاربر نمایش می دهد.

2. نیاز به رتبه بندی (Ranking):

کاربران سیستم های مدرن توانایی و حوصله بررسی هزاران نتیجه را ندارند؛ آن ها به دنبال یافتن مرتبط ترین اسناد در همان صفحه اول نتایج هستند. ترکیب جستجوی بولی با امتیازدهی منطقه ای به این شکل عمل می کند که ابتدا از منطق بولی برای **فیلتر کردن** اولیه اسناد (بازبایی مجموعه جواب) استفاده می شود و سپس از **Weighted Zone Scoring** برای **رتب سازی و رتبه بندی** آن مجموعه بر اساس میزان ارتباط، بهره گیری می گردد.

بخش دوم: دلیل اصلی برای استفاده از وزن دهی متفاوت در مناطق مختلف

دلیل اصلی و بنیادین این کار، **تفاوت در چگالی معنایی (Semantic Density) و ارزش توصیفی بخش های مختلف یک سند** است.

تشریح منطق: منطقه ای مانند «عنوان» (Title) به طور ذاتی خلاصه ای فشرده از کل محتوای سند است. اگر کلمه مورد جستجوی کاربر در عنوان ظاهر شود، احتمال اینکه کل سند دقیقاً درباره همان موضوع باشد بسیار بالاست. اما اگر همان کلمه در «پاورقی» یا در میانه «بدنه اصلی» (Body) ظاهر شود، ممکن است تنها یک اشاره گذرا به موضوع شده باشد.

مثال کاربردی: اگر کاربری کلمه "پایتون" را جستجو کند، اختصاص وزن بالاتر به منطقه عنوان (مثلاً $g_{title} = 0.7$) نسبت به بدنه متن (مثلاً $g_{body} = 0.3$)، تضمین می کند کتابی که نامش «آموزش پایتون» است بالاتر از

یک رمان قرار گیرد که در متن آن به صورت تصادفی نام یک مار پایتون ذکر شده است.

بخش 3) جمع بندی:

نکته اول: یک سیستم بازبایی قطعی (مانند بولی) خروجی اش یک «مجموعه بدون ترتیب» (Unordered Set) است، اما یک سیستم مبتنی بر امتیازدهی (مانند Zone Scoring) خروجی اش یک «لیست مرتب شده» (Ranked List) است.

نکته دوم: در پاسخ به سوالات تحلیلی مشابه، همیشه تاکید کنید که سیستم های مدرن معمولاً مدل بولی را به طور کامل حذف نمی کنند، بلکه از آن در لایه اولیه پردازش (برای کاهش فضای جستجو و ایندکس) استفاده کرده و لایه امتیازدهی (Scoring) را برای بهبود تجربه کاربر (User Experience) و افزایش دقت (Precision) در نتایج برتر به آن می افزایند.

پاسخنامه مسئله 6.4: نقش فرکانس معکوس سند (IDF) در وزن دهی کلمات

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک استدلال منطقی و ریاضیاتی نهفته در محاسبه **IDF**، آشنایی با مفهوم ارزش اطلاعاتی (**Information Value**) کلمات، و توانایی تحلیل مکانیزم تضعیف کلمات پرکاربرد (**Stop Words**) در سیستم‌های بازیابی است. در مدل فضای برداری (**Vector Space Model**)، اگر برای رتبه‌بندی اسناد صرفاً به میزان تکرار یک کلمه در سند (**Term Frequency** یا **TF**) تکیه کنیم، با یک خطای محاسباتی بزرگ مواجه می‌شویم. در هر زبان طبیعی، کلماتی وجود دارند که از نظر ساختار گرامری به شدت پرتکرار هستند اما بار معنایی خاصی برای متمایز کردن موضوع اسناد ندارند.

معیار **فرکانس معکوس سند (IDF)** به عنوان یک فاکتور تعدیل‌کننده (**Scaling Factor**) معرفی شده است تا این نقص را جبران کند. منطق پایه‌ای این معیار ریشه در نظریه اطلاعات (**Information Theory**) دارد: «هرچه یک رویداد (یا کلمه) نادرتر باشد، وقوع آن حاوی اطلاعات و ارزش بیشتری است.»

فرمول استاندارد محاسبه **IDF** برای کلمه t به صورت زیر است:

$$\text{idf}_t = \log_{10} \left(\frac{N}{\text{df}_t} \right)$$

که در آن **N** تعداد کل اسناد مجموعه و **df_t** تعداد اسنادی است که کلمه t در آن‌ها وجود دارد.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

گام اول: استدلال پشت استفاده از IDF در فرمول‌های وزن دهی

دلیل اصلی استفاده از **IDF**، اعمال جریمه (**Penalty**) برای کلمات عمومی و پاداش (**Reward**) برای کلمات تخصصی است. سیستم بازیابی باید بتواند میزان «قدرت تفکیک‌کنندگی» (**Discriminative Power**) هر کلمه را بسنجد. کلمه‌ای که در تمام اسناد یک مجموعه متنی وجود دارد، هیچ کمکی به کاربر برای یافتن یک سند خاص در میان انبوه اسناد نمی‌کند. **IDF** به صورت ریاضیاتی وزن چنین کلماتی را به 0 (یا مقادیر بسیار ناچیز) میل می‌دهد و در عوض، وزن کلماتی که در تعداد معدودی از اسناد ظاهر شده‌اند را تقویت می‌کند.

گام دوم: تشریح مفهوم با یک مثال عملی (کلمات "است" و "الگوریتم")

فرض کنید در حال پردازش یک مجموعه متنی شامل 100,000 مقاله علمی هستیم (**N = 100,000**). رفتار فرمول **IDF** را برای دو کلمه متفاوت بررسی می‌کنیم:

حالت اول: کلمه عمومی "است" (is)

این کلمه یک فعل ربطی است و به احتمال بسیار زیاد در تمامی 100,000 مقاله علمی حداقل یک‌بار به کار رفته است. بنابراین فرکانس سند برای آن $\text{df} \approx 100,000$ خواهد بود.

$$\text{idf} = \log_{10} \left(\frac{100,000}{100,000} \right) = \log_{10}(1) = 0$$

تحلیل: خروجی 0 به این معناست که کلمه "است" هیچ ارزش جستجویی ندارد. حتی اگر این کلمه در یک سند 500 بار تکرار شده باشد (TF بالا)، به دلیل ضرب شدن در IDF برابر با 0، وزن نهایی (TF-IDF) آن 0 خواهد شد و در رتبه‌بندی تأثیری نخواهد گذاشت.

حالت دوم: کلمه تخصصی "الگوریتم" (algorithm)

این کلمه دارای بار معنایی مشخصی است و تنها در مقالات مرتبط با علوم کامپیوتر یا ریاضیات یافت می‌شود. فرض کنید این کلمه در 1,000 مقاله از کل مجموعه وجود داشته باشد (df = 1,000).

$$\text{idf} = \log_{10} \left(\frac{100,000}{1,000} \right) = \log_{10}(100) = 2$$

تحلیل: وزن کلمه "الگوریتم" برابر با 2 محاسبه شد. این عدد نشان می‌دهد که کلمه مذکور توانایی بالایی در فیلتر کردن اسناد نامرتب دارد. هر سندی که حاوی این کلمه باشد، در فرآیند ضرب TF در IDF، امتیاز بالایی کسب کرده و در صدر نتایج جستجو به کاربر نمایش داده می‌شود.

بخش 3) جمع‌بندی:

نکته: نقش IDF در سیستم‌های جستجو، فیلتر کردن هوشمندانه و پویا است. در سیستم‌های مدرن، به جای اینکه یک لیست ثابت از کلمات توقف (Stop Words) مانند حروف اضافه تعریف کنیم تا از ایندکس حذف شوند، الگوریتم IDF به صورت خودکار و بر اساس توزیع آماری کلمات در همان مجموعه (Corpus)، اثر کلمات بی‌ارزش را خنثی می‌کند.

رابطه ریاضی: همیشه به یاد داشته باشید که رابطه بین df و وزن کلمه، یک رابطه معکوس است (هرچه کلمه در اسناد بیشتری باشد، ارزش آن کمتر است)، اما به دلیل استفاده از تابع لگاریتم، این کاهش ارزش به صورت خطی نیست، بلکه شیبی ملایم و کنترل‌شده دارد تا از تغییرات شدید در امتیازدهی جلوگیری شود.

پاسخنامه مسئله 6.5: مفهوم فرکانس مجموعه در برابر فرکانس سند و ارجحیت آن در وزن‌دهی

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک تفاوت‌های آماری کلمات در یک پیکره متنی (Corpus)، شناخت ویژگی‌های توزیع کلمات، و درک منطق ریاضیاتی نهفته در فرمول IDF برای متمایز کردن کلمات کلیدی از کلمات عمومی است.

در تحلیل آماری و معماری سیستم‌های بازیابی اطلاعات، هنگام ساخت نمایه (Index Construction)، دو معیار شمارشی متمایز برای هر کلمه (t) در کل مجموعه اسناد در نظر گرفته می‌شود:

فرکانس مجموعه (cf_t - Collection Frequency): تعداد کل دفعاتی که کلمه t در تمامی اسناد مجموعه تکرار شده

است. (مجموع فرکانس خام کلمه در تک تک اسناد).

فرکانس سند (df_t - Document Frequency): تعداد اسناد یکتایی که کلمه t حداقل یک بار در آن‌ها ظاهر شده

است.

از نظر ریاضی، همواره رابطه زیر برقرار است، زیرا در یک سند مشخص، یک کلمه ممکن است چندین بار تکرار شود که باعث افزایش **cf** می‌شود، اما به **df** تنها 1 واحد اضافه می‌کند:

$$df_t \leq cf_t$$

بخش 2) حل گام به گام و تشریح خواسته‌های مسئله

گام اول: بررسی تفاوت **cf** و **df** با یک مثال مفهومی

برای درک بهتر تفاوت، دو کلمه را در یک مجموعه متنی بزرگ در نظر بگیرید:

1. کلمه‌ای عمومی (مانند "استفاده" یا "روش").

2. کلمه‌ای به شدت تخصصی (مانند "قانون_هیپ" - **Heaps' Law**).

فرض کنید آمار این دو کلمه در مجموعه به شکل زیر است:

کلمه (t)	فرکانس مجموعه (cf_t)	فرکانس سند (df_t)	تحلیل توزیع
روش	100,000	80,000	این کلمه در اکثر اسناد پخش شده است (توزیع گسترده).
قانون هیپ	100,000	50	این کلمه به شدت در چند سند خاص و تخصصی تکرار شده است (توزیع متمرکز).

در اینجا، هر دو کلمه در کل مجموعه به یک اندازه (100,000 بار) تولید شده‌اند (**cf** برابر). اما کلمه تخصصی تنها در 50 سند متمرکز است، در حالی که کلمه عمومی در 80,000 سند پراکنده است.

گام دوم: چرا برای محاسبه‌ی وزن کلمات، **df** بر **cf** ارجحیت دارد؟

دلیل اصلی این ارجحیت، مفهوم «**قدرت تفکیک‌کنندگی**» (**Discriminative Power**) است.

هدف از محاسبه‌ی لگاریتم فرکانس معکوس سند (**idf**) در فرمول زیر، این است که به کلماتی که به ما در پیدا کردن سند مرتبط کمک می‌کنند، وزن بیشتری بدهیم و اثر کلماتی که در همه جا هستند را خنثی کنیم:

$$idf = \log\left(\frac{N}{df}\right)$$

اگر از **cf استفاده می‌کردیم:** در مثال بالا، چون فرکانس مجموعه برای هر دو کلمه برابر (100,000) است، سیستم به

کلمه‌ی بسیار تخصصی "قانون_هیپس" و کلمه‌ی عمومی "روش" وزن کاملاً **یکسانی** می‌داد! این کار باعث می‌شد

موتور جستجو نتواند اسناد تخصصی را از اسناد نامرتبط تشخیص دهد و کیفیت بازیابی به شدت افت می‌کند.

با استفاده از df: سیستم متوجه می‌شود که کلمه‌ی تخصصی تنها در 50 سند وجود دارد (نادر بودن در سطح اسناد).

بنابراین، با قرار دادن عدد کوچکتر در مخرج کسر **idf**، وزن به مراتب بالاتری به آن اختصاص می‌دهد و در هنگام

جستجو، اسناد حاوی این کلمه را به درستی در صدر نتایج رتبه‌بندی می‌کند.

بخش 3) جمع‌بندی:

نکته: به زبان ساده، **df** به ما می‌گوید که یک کلمه چقدر «**نادر و متمایزکننده**» است، در حالی که **cf** صرفاً به ما می‌گوید

که یک کلمه چقدر «**پرتکرار**» است. در رتبه‌بندی اسناد، تمایز (**Rarity**) بسیار مهم‌تر از تکرار مطلق است.

کاربرد واقعی cf در معماری IR: اگرچه **df** قهرمانِ بلامنازعِ وزن‌دهی (**TF-IDF**) است، اما **cf** هرگز بی‌مصرف نیست. در

طراحی معماری‌های پیشرفته بازیابی اطلاعات، از مقدار **cf** برای تخمین حجم حافظه مورد نیاز جهت ذخیره‌سازی،

بهینه‌سازی طول لیست‌های پستینگ (**Posting Lists**) و انتخاب تکنیک‌های فشرده‌سازی کدهای طول متغیر (**Variable**

Byte Code Compression) استفاده می‌شود.

پاسخنامه مسئله 6.6: هدف امتیازدهی منطقه‌ای و مقایسه آن با جستجوی بولی

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک محدودیت‌های مدل‌های قطعی (مانند جستجوی بولی) در بازیابی اطلاعات مقیاس‌پذیر و شناخت

منطق پشت رتبه‌بندی اسناد بر اساس اهمیت ساختاری آن‌ها (وزن‌دهی مناطق) است.

جستجوی بولی ساده (Boolean Retrieval) یکی از قدیمی‌ترین و پایه‌ای‌ترین مدل‌های بازیابی اطلاعات است که بر اساس

منطق 0 و 1 کار می‌کند. در این سیستم، یک سند یا دقیقاً با پرس‌وجوی کاربر (متشکل از عملگرهای **AND, OR, NOT**)

مطابقت دارد و بازیابی می‌شود، یا مطابقت ندارد و کنار گذاشته می‌شود.

در مقابل، **امتیازدهی منطقه‌ای (Weighted Zone Scoring)** یک رویکرد مبتنی بر رتبه‌بندی (**Ranking**) است. این مدل

می‌پذیرد که تمامی بخش‌های یک سند ارزش اطلاعاتی یکسانی ندارند و حضور یک کلمه در بخش‌های مختلف (مانند عنوان،

چکیده یا متن اصلی) باید امتیاز متفاوتی برای ارتباط سند با پرس‌وجو ایجاد کند.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

بخش اول: چرا جستجوی بولی با امتیازدهی منطقه‌ای جایگزین یا ترکیب می‌شود؟

دو دلیل اصلی برای این گذار در سیستم‌های مدرن وجود دارد که باید به دقت مورد بررسی قرار گیرند:

1. مشکل «ضیافت یا قحطی» (Feast or Famine) در جستجوی بولی:

در مجموعه‌های متنی بزرگ (**Corpus**) مانند وب، یک پرس‌وجوی بولی ساده ممکن است هزاران یا حتی میلیون‌ها

سند را برگرداند که همگی شرط منطقی را ارضا کرده‌اند (ضیافت)، و یا با اضافه کردن یک عملگر **AND** اضافی، هیچ

سندی را برنگرداند (قحطی). سیستم بولی هیچ راهی برای اولویت‌بندی میان آن هزاران سند بازیابی شده ندارد و همه

را در یک سطح از اهمیت به کاربر نمایش می‌دهد.

2. نیاز به رتبه‌بندی (Ranking):

کاربران سیستم‌های مدرن توانایی و حوصله بررسی هزاران نتیجه را ندارند؛ آن‌ها به دنبال یافتن مرتبط‌ترین اسناد در همان صفحه اول نتایج هستند. ترکیب جستجوی بولی با امتیازدهی منطقه‌ای به این شکل عمل می‌کند که ابتدا از منطق بولی برای **فیلتر کردن** اولیه اسناد (بازیابی مجموعه جواب) استفاده می‌شود و سپس از **Weighted Zone Scoring** برای **مرتب‌سازی و رتبه‌بندی** آن مجموعه بر اساس میزان ارتباط، بهره‌گیری می‌گردد.

بخش دوم: دلیل اصلی برای استفاده از وزن‌دهی متفاوت در مناطق مختلف

دلیل اصلی و بنیادین این کار، **تفاوت در چگالی معنایی (Semantic Density) و ارزش توصیفی بخش‌های مختلف یک سند** است.

تشریح منطق: منطقه‌ای مانند «عنوان» (**Title**) به طور ذاتی خلاصه‌ای فشرده از کل محتوای سند است. اگر کلمه مورد جستجوی کاربر در عنوان ظاهر شود، احتمال اینکه کل سند دقیقاً درباره همان موضوع باشد بسیار بالاست. اما اگر همان کلمه در «پاورقی» یا در میانه «بدنه اصلی» (**Body**) ظاهر شود، ممکن است تنها یک اشاره گذرا به موضوع شده باشد.

مثال کاربردی: اگر کاربری کلمه "پایتون" را جستجو کند، اختصاص وزن بالاتر به منطقه عنوان (مثلاً $g_{title} = 0.7$) نسبت به بدنه متن (مثلاً $g_{body} = 0.3$)، تضمین می‌کند کتابی که نامش «آموزش پایتون» است بالاتر از یک رمان قرار گیرد که در متن آن به صورت تصادفی نام یک مار پایتون ذکر شده است.

بخش 3) جمع‌بندی:

نکته کلیدی برای به خاطر سپردن: یک سیستم بازیابی قطعی (مانند بولی) خروجی‌اش یک «مجموعه بدون ترتیب» (**Unordered Set**) است، اما یک سیستم مبتنی بر امتیازدهی (مانند **Zone Scoring**) خروجی‌اش یک «لیست مرتب‌شده» (**Ranked List**) است.

نکته دوم: در پاسخ به سوالات تحلیلی مشابه، همیشه تاکید کنید که سیستم‌های مدرن معمولاً مدل بولی را به طور کامل حذف نمی‌کنند، بلکه از آن در لایه اولیه پردازش (برای کاهش فضای جستجو و ایندکس) استفاده کرده و لایه امتیازدهی (**Scoring**) را برای بهبود تجربه کاربر (**User Experience**) و افزایش دقت (**Precision**) در نتایج برتر به آن می‌افزایند.

پاسخنامه مسئله 6.7: تشریح متغیرهای فرمول TF-IDF و اثر کلمات همه‌جا حاضر

بخش 1) تحلیل اولیه و مبانی تئوری

هدف از این مسئله، کالبدشکافی دقیق متغیرهای آماری در فرمول استاندارد وزن‌دهی، درک ماهیت مخرج کسر در محاسبه‌ی فرکانس معکوس سند (**IDF**)، و اثبات ریاضیاتی بی‌اثر بودن کلمات عمومی در فرآیند رتبه‌بندی است.

فرمول وزن‌دهی **TF-IDF (Term Frequency - Inverse Document Frequency)** ستون فقرات بسیاری از سیستم‌های کلاسیک و مدرن بازیابی اطلاعات است. این فرمول از ضرب دو معیار مستقل به دست می‌آید: یکی نمایانگر میزان تکرار کلمه در یک سند خاص (**TF**) و دیگری نمایانگر میزان نادر بودن آن کلمه در کل پایگاه داده (**IDF**).

$$\text{TF-IDF}_{t,d} = \text{tf}_{t,d} \times \log\left(\frac{N}{\text{df}_t}\right)$$

برای اینکه موتور جستجو بتواند تصمیم بگیرد کدام کلمه ارزش بالاتری برای یافتن مقصود کاربر دارد، باید رفتار دقیق متغیرهای کسر داخل لگاریتم را به درستی درک کنیم.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

الف) تعریف دقیق متغیرهای N و df_t

در بخش فرکانس معکوس سند (IDF)، دو متغیر کلیدی ایفای نقش می‌کنند که ماهیت کلان (Global) دارند و به سطح کل پایگاه داده مربوط می‌شوند، نه یک سند خاص:

متغیر (N (Total Number of Documents): این متغیر نشان‌دهنده **حجم کل پایگاه داده (Corpus Size)** است؛

یعنی تعداد کل اسناد مستقلی که در سیستم نمایه‌سازی (Index) شده‌اند. به عنوان مثال، اگر دیتابیس ما شامل یک میلیون مقاله علمی باشد، N دقیقاً برابر با 1,000,000 خواهد بود. این عدد ثابت است و به کلمه‌ی مورد جستجو بستگی ندارد.

متغیر (df_t (Document Frequency): این متغیر نشان‌دهنده **فرکانس سند برای کلمه‌ی t** است؛ یعنی تعداد اسناد یکتایی در پایگاه داده که کلمه‌ی t حداقل یک‌بار در آن‌ها ظاهر شده است. این متغیر مستقیماً میزان «رایج بودن» یک کلمه را نشان می‌دهد. هرچه این عدد بزرگتر باشد، کلمه عمومی‌تر است و قدرت تفکیک‌کنندگی کمتری دارد.

ب) محاسبه وزن نهایی برای کلمه‌ای که در تمام اسناد وجود دارد

فرض کنید کلمه‌ای مانند "و" یا "در" (یا در متون انگلیسی کلماتی مانند "the" یا "is") به قدری پرکاربرد است که در تک‌تک اسناد پایگاه داده حداقل یک‌بار به کار رفته است.

گام 1: تعیین مقادیر بر اساس فرض مسئله

اگر کلمه در تمامی اسناد حضور داشته باشد، تعداد اسناد حاوی کلمه (df_t) دقیقاً برابر با کل اسناد پایگاه داده (N) خواهد بود:

$$\text{df}_t = N$$

گام 2: جایگذاری در فرمول IDF

حال این تساوی را در بخش لگاریتمی فرمول قرار می‌دهیم:

$$\text{idf}_t = \log\left(\frac{N}{N}\right) = \log(1)$$

در ریاضیات، لگاریتم عدد 1 در هر مبنایی (چه مبنای 10، چه مبنای 2 و چه لگاریتم طبیعی) همواره برابر با 0 است. بنابراین:

$$\text{idf}_t = 0 \implies \text{TF-IDF} = \text{tf} \times 0 = 0$$

نتیجه‌گیری ریاضی: وزن نهایی **TF-IDF** برای چنین کلمه‌ای دقیقاً 0 خواهد شد.

بخش 3) جمع‌بندی:

نکته اول: خروجی 0 در این محاسبه یک باگ یا خطای سیستمی نیست، بلکه دقیقاً **هدف و شاهکار معماری فرمول IDF** است. این ویژگی ریاضی باعث می‌شود که سیستم به صورت خودکار کلمات بی‌ارزش و فاقد قدرت جستجو (**Stop words**) را فیلتر کند.

نکته دوم: به خاطر داشته باشید که اگر وزن **TF-IDF** کلمه‌ای 0 شود، به این معناست که آن کلمه در فضای برداری (**Vector Space Model**) هیچ تأثیری در تغییر زاویه بردار سند و در نتیجه هیچ نقشی در فرآیند رتبه‌بندی (**Ranking**) ایفا نخواهد کرد؛ حتی اگر آن کلمه هزاران بار در متن یک سند تکرار شده باشد (**TF** بسیار بالا). در بازیابی اطلاعات، «کمیبایی» بسیار ارزشمندتر از «تکرار مطلق» است.

پاسخنامه مسئله 6.8: مفهوم ضرب داخلی و چالش‌های طول سند در امتیازدهی

بخش 1) تحلیل اولیه و مبانی تئوری

هدف از این مسئله، شناخت مفهوم هندسی و محاسباتی ضرب داخلی بردارها به عنوان معیار شباهت، و درک ضرورت نرمال‌سازی طول (**Length Normalization**) برای جلوگیری از سوگیری (**Bias**) به نفع اسناد طولانی‌تر است. در مدل فضای برداری (**VSM**)، هر سند (d) و هر پرس‌وجو (q) به عنوان بردارهایی در یک فضای V -بعدی (که V اندازه واژه‌نامه است) نمایش داده می‌شوند. هر درایه از این بردارها، نمایانگر وزن یک کلمه خاص (غالباً بر اساس فرکانس کلمه و فرکانس معکوس سند) است. پایه‌ای‌ترین ابزار ریاضی برای محاسبه میزان اشتراک این دو بردار، **ضرب داخلی (Dot Product)** است.

از نظر ریاضی، ضرب داخلی دو بردار $\vec{q}$ و $\vec{d}$ برابر است با مجموع حاصل‌ضرب درایه‌های متناظر آن‌ها:

$$\vec{q} \cdot \vec{d} = \sum_{i=1}^V q_i d_i$$

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

الف) ضرب داخلی دو بردار در فضای برداری نشان‌دهنده‌ی چیست؟

ضرب داخلی نشان‌دهنده‌ی **میزان هم‌پوشانی مطلق (Absolute Overlap)** یا اشتراک وزنی خام بین پرس‌وجو و سند است. هرچه کلمات مشترک بیشتری بین پرس‌وجو و سند وجود داشته باشد و وزن آن کلمات در هر دو بردار بزرگتر باشد، مقدار

عددی ضرب داخلی افزایش می‌یابد. از منظر هندسی، این معیار متناسب است با تصویر (**Projection**) بردار پرس‌وجو بر روی جهت بردار سند، که در طول بردار سند ضرب شده است. در نتیجه، یک خروجی عددی بدون سقف تولید می‌کند که صرفاً نشان‌دهنده میزان قدرت ترکیب کلمات مشترک است.

ب) چرا استفاده از ضرب داخلی خالص برای اسناد با طول متفاوت گمراه‌کننده است؟

استفاده از ضرب داخلی خام بدون نرمال‌سازی، یک خطای سیستماتیک به نام **سوگیری طول سند (Document Length Bias)** در سیستم بازیابی ایجاد می‌کند. این خطای تحلیلی به دو دلیل ساختاری رخ می‌دهد:

1. افزایش طبیعی فرکانس کلمات (**Higher TF**):

در یک سند طولانی (مثلاً یک کتاب 500 صفحه‌ای)، کلمات به طور طبیعی دفعات بیشتری تکرار می‌شوند تا یک سند کوتاه (مثلاً یک چکیده نیم‌صفحه‌ای). این تکرار باعث می‌شود درایه‌های بردار سند طولانی (مقادیر d_i) به شکل قابل توجهی بزرگتر شوند و در نتیجه حاصل‌ضرب $q_i d_i$ به صورت کاذب رشد کند.

2. تنوع کلمات بیشتر (**More Unique Terms**):

اسناد طولانی‌تر دایره واژگان وسیع‌تری دارند. احتمال اینکه این اسناد حداقل یک یا چند کلمه از پرس‌وجوی کاربر را (حتی به صورت کاملاً تصادفی و حاشیه‌ای) در خود داشته باشند، بسیار بالاتر از اسناد کوتاه است.

یک مثال نقض:

فرض کنید کاربر عبارت «مبانی بازیابی اطلاعات» را جستجو کرده است.

▪ **سند الف (کوتاه):** یک مقاله تخصصی 3 صفحه‌ای که دقیقاً روی همین موضوع تمرکز دارد.

▪ **سند ب (طولانی):** یک دایرةالمعارف 1000 صفحه‌ای درباره تاریخ علوم که در یک پاورقی گذرا به این موضوع اشاره کرده،

اما کلمات «مبانی» و «اطلاعات» بارها در فصول مختلف و نامربوط آن تکرار شده‌اند.

در محاسبه با ضرب داخلی خالص، «سند ب» صرفاً به خاطر حجم عظیمش، بردار بسیار بلندی دارد (اندازه $|\vec{d}|$ بسیار بزرگ است) و امتیاز بالاتری می‌گیرد. در حالی که «سند الف» بدون شک مرتبط‌ترین پاسخ است.

راه حل: نرمال‌سازی کسینوسی (Cosine Normalization)

برای رفع این مشکل بنیادین، در سیستم‌های استاندارد، نتیجه ضرب داخلی را بر حاصل‌ضرب طول (اندازه اقلیدسی) دو بردار تقسیم می‌کنند. این تکنیک باعث می‌شود به جای اندازه فیزیکی بردارها، تنها **زاویه (جهت)** آن‌ها معیار شباهت معنایی باشد:

$$\text{Cosine}(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \times |\vec{d}|}$$

با این عملیات، تمامی بردارها از نظر ریاضی به بردارهای یکه (**Unit Vectors**) با طول 1 تبدیل می‌شوند و برتری کاذب اسناد طولانی خنثی می‌گردد.

بخش 3) جمع‌بندی:

نکته اول: ضرب داخلی به تنهایی، ترکیبی تفکیک‌ناپذیر از «ارتباط معنایی» و «حجم فیزیکی متن» را می‌سنجد. برای طراحی یک سیستم بازیابی عادلانه و دقیق، ما باید اثر متغیر «طول متن» را ایزوله و حذف کنیم تا فقط «ارتباط معنایی» خالص برای رتبه‌بندی باقی بماند.

نکته دوم: اگر در حل مسئله‌ای بردارهای دو سند با طول نابرابر به شما داده شد و طراح سوال از شما خواست سند مرتبط‌تر به یک پرس‌وجو را پیدا کنید، **هرگز** به محاسبه ضرب داخلی خام اکتفا نکنید. ابتدا طول هر بردار (نرم اقلیدسی یا **L2 Norm**) را با فرمول $\sqrt{\sum d_i^2}$ محاسبه کرده و سپس مقادیر نرمال‌شده را با هم مقایسه نمایید.

پاسخنامه مسئله 6.9: مقیاس‌دهی زیرخطی فرکانس کلمه (Sublinear TF Scaling)

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک رفتار لگاریتمی فرکانس کلمات در متون، شناخت پدیده بازده نزولی (**Diminishing Returns**) در ارتباط سند با پرس‌وجو، و رفع خطای محاسباتی فرض خطی بودن میزان ارتباط با تکرار کلمه است. مدل‌های اولیه و ساده‌انگارانه در بازیابی اطلاعات بر این فرض استوار بودند که وزن یک کلمه در سند، مستقیماً و به صورت خطی با فرکانس خام آن (تعداد دفعات تکرار یا **tf**) متناسب است. بر اساس این فرض، اگر سندی یک کلمه کلیدی را 20 بار تکرار کرده باشد، دقیقاً 20 برابر مرتبط‌تر از سندی است که آن کلمه را تنها 1 بار به کار برده است.

در واقعیت زبان‌شناسی و تحلیل معنایی متون، این فرض کاملاً غلط است. میزان «ارتباط معنایی» یک سند با یک موضوع، هم‌پای تکرار یک کلمه به صورت خطی رشد نمی‌کند. روش **مقیاس‌دهی زیرخطی (Sublinear tf scaling)** یک اصلاح ریاضی است که برای حل این ناهمخوانی ساختاری معرفی شده است.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

الف) هدف از جایگزینی **tf** خام با فرمول $1 + \log(\text{tf})$ چیست؟

هدف اصلی این جایگزینی، اعمال قانون **بازده نزولی (Law of Diminishing Returns)** در محاسبه وزن کلمات است. تابع مقیاس‌دهی زیرخطی به صورت قطعه‌ای (**Piecewise**) و معمولاً در مبنای 10 به شکل زیر تعریف می‌شود:

$$w_{t,d} = \begin{cases} 1 + \log_{10}(\text{tf}_{t,d}) & \text{if } \text{tf}_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

استدلال منطقی: اولین باری که یک کلمه‌ی مورد جستجو در متنی ظاهر می‌شود، بیشترین ارزش اطلاعاتی را دارد (زیرا ثابت می‌کند که سند حداقل ارتباطی با موضوع دارد). تکرارهای دوم، سوم و چهارم نیز اهمیت دارند و ارتباط سند را تقویت می‌کنند، اما میزان این اهمیت افزوده، رفته‌رفته کمتر می‌شود. لگاریتم به دلیل ماهیت رفتار نمودارش (رشد کندشونده)، این پدیده را به بهترین شکل مدل‌سازی می‌کند؛ به طوری که اثر تکرارهای اولیه حفظ شده و اثر تکرارهای مازاد به شدت تعدیل (**Dampen**) می‌شود.

ب) این کار چه مشکلی از سیستم رتبه‌بندی را حل می‌کند؟

این تکنیک مستقیماً مشکل «سوگیری تکرار» (Repetition Bias) و انحراف سیستم به سمت اسنادی که یک کلمه را به صورت غیرطبیعی تکرار کرده‌اند، حل می‌کند.

برای درک دقیق اثربخشی این روش، مثال عددی زیر را بررسی می‌کنیم. فرض کنید کاربر کلمه‌ی "پایتون" را جستجو کرده است:

سند الف: کلمه 1 بار تکرار شده است.

وزن لگاریتمی: $1 + \log_{10}(1) = 1 + 0 = 1$

سند ب: کلمه 10 بار تکرار شده است.

وزن لگاریتمی: $1 + \log_{10}(10) = 1 + 1 = 2$

سند ج: کلمه 100 بار تکرار شده است.

وزن لگاریتمی: $1 + \log_{10}(100) = 1 + 2 = 3$

مقایسه مشکل و راه حل:

اگر از **tf** خام استفاده می‌کردیم، موتور جستجو به صورت کورکورانه نتیجه می‌گرفت که سند «ج» 100 برابر مرتبط‌تر از سند «الف» است و بی‌قیدوشرط آن را در رتبه اول قرار می‌داد (که می‌تواند بستری برای تقلب یا **Term Spamming** باشد). اما با اعمال مقیاس‌دهی زیرخطی، سیستم درک می‌کند که سند «ج» قطعاً قوی‌تر و مرتبط‌تر از سند «الف» است، اما برتری آن منطقی و تنها **3 برابر** است. این کار از سلطه‌ی اسناد طولانی یا پر از تکرار بر نتایج جستجو جلوگیری می‌کند.

بخش 3) جمع‌بندی:

نکته اول: در تحلیل سیستم‌های بازیابی پیشرفته، همیشه به یاد داشته باشید که رابطه بین «تعداد تکرار یک کلمه» و «میزان ارتباط سند»، یک رابطه خطی نیست. فرمول $1 + \log(tf)$ به عنوان یک مهارکننده (**Dampener**) عمل می‌کند تا جهش‌های غیرمنطقی در امتیازدهی اسناد را خنثی کند.

نکته دوم: اگر در صورت مسئله برای محاسبه‌ی وزن‌دهی از عبارت‌های **Sublinear TF** یا **Log-frequency weighting** استفاده شده بود، پیش از ضرب کردن مقدار در فرمول **IDF**، حتماً فرکانس خام (**tf**) را وارد فرمول لگاریتمی کنید. همچنین به شرط پایه‌ای تابع دقت کنید: اگر کلمه‌ای در سند وجود نداشت ($tf = 0$)، وزن آن مستقیماً **0** در نظر گرفته می‌شود و نیازی به محاسبه لگاریتم **0** (که تعریف نشده است) نیست.

پاسخنامه مسئله 6.10: محاسبه شباهت کسینوسی (Cosine Similarity)

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، سنجش مهارت محاسباتی در فضای برداری 2 بعدی، درک فرآیند نرمال‌سازی طول بردارهای اسپارس (**Sparse Vectors**)، و مشاهده‌ی عملی تأثیر زاویه بردارها در تعیین میزان ارتباط اسناد است.

در مدل فضای برداری، میزان شباهت معنایی بین یک پرس‌وجو ($\vec{q}$) و یک سند ($\vec{d}$)، از طریق محاسبه‌ی کسینوس زاویه‌ی بین آن دو بردار در فضا اندازه‌گیری می‌شود. این معیار که **شباهت کسینوسی (Cosine Similarity)** نام دارد، اثری که طول اسناد ممکن است در محاسبات ایجاد کند را از بین می‌برد.

فرمول استاندارد شباهت کسینوسی به شکل زیر است:

$$\text{Cosine}(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \times |\vec{d}|}$$

که در آن صورت کسر نشان‌دهنده‌ی ضرب داخلی (**Dot Product**) و مخرج کسر نشان‌دهنده‌ی حاصل ضرب طول (نرم اقلیدسی یا **L2 Norm**) دو بردار است.

داده‌های مسئله:

برداری پرس‌وجو: $\vec{q} = [1, 1]$

برداری سند اول: $\vec{d}_1 = [3, 4]$

برداری سند دوم: $\vec{d}_2 = [1, 7]$

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

گام اول: محاسبه طول بردارها

برای نرمال‌سازی، ابتدا باید طول هر بردار را از طریق جذر مجموع مربعات درایه‌های آن به دست آوریم:

1. طول بردار کوئری $(|\vec{q}|)$:

$$|\vec{q}| = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.414$$

2. طول بردار سند اول $(|\vec{d}_1|)$:

$$|\vec{d}_1| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

3. طول بردار سند دوم $(|\vec{d}_2|)$:

$$|\vec{d}_2| = \sqrt{1^2 + 7^2} = \sqrt{1 + 49} = \sqrt{50} \approx 7.071$$

گام دوم: محاسبه ضرب داخلی

اکنون ضرب داخلی برداری پرس‌وجو را با هر یک از اسناد محاسبه می‌کنیم:

1. ضرب داخلی با سند اول:

$$\vec{q} \cdot \vec{d}_1 = (1 \times 3) + (1 \times 4) = 3 + 4 = 7$$

$$\vec{q} \cdot \vec{d}_2 = (1 \times 1) + (1 \times 7) = 1 + 7 = 8$$

گام سوم: محاسبه شباهت کسینوسی و نرمال سازی نهایی

با در اختیار داشتن مقادیر صورت و مخرج، شباهت نهایی را برای هر دو سند به دست می آوریم:

1. امتیاز سند اول:

$$\text{Cosine}(\vec{q}, \vec{d}_1) = \frac{7}{\sqrt{2} \times 5} \approx \frac{7}{7.071} \approx 0.9899$$

2. امتیاز سند دوم:

$$\text{Cosine}(\vec{q}, \vec{d}_2) = \frac{8}{\sqrt{2} \times \sqrt{50}} = \frac{8}{\sqrt{100}} = \frac{8}{10} = 0.8$$

گام چهارم: نتیجه گیری و رتبه بندی

با مقایسه دو امتیاز به دست آمده ($0.9899 > 0.8$)، مشخص می شود که زاویه بین بردار کوئری و بردار سند اول بسیار کوچکتر از زاویه آن با سند دوم است. بنابراین، **سند اول** (d_1) مرتبطتر ارزیابی شده و در رتبه بالاتری قرار می گیرد.

بخش 3) جمع بندی:

نکته کلیدی و تحلیلی: این مسئله یک نمونه ی کلاسیک و بسیار مهم برای درک خطای «ضرب داخلی خام» است. اگر به گام دوم نگاه کنید، ضرب داخلی سند دوم با کوئری برابر با 8 بود (بیشتر از سند اول که 7 بود). اگر سیستم فقط از ضرب داخلی استفاده می کرد، به اشتباه سند دوم را به عنوان خروجی بهتر نمایش می داد.

دلیل برتری سند اول: سند دوم به دلیل داشتن عدد بزرگ 7 در یکی از درایه هایش، بردار بلندی در فضا ساخته بود که ضرب داخلی را به طور کاذب بالا می برد، اما از نظر توزیع کلمات (نسبت 1 به 7)، انحراف زیادی از کوئری (نسبت 1 به 1) داشت. سند اول (نسبت 3 به 4) از نظر ساختار معنایی و توزیع کلمات، تناسب بسیار بیشتری با کوئری داشت که این حقیقت تنها پس از **نرمال سازی طول (Length Normalization)** و محاسبه ی دقیق کسینوسی خود را نشان داد.

برای امتحانات: در مسائل محاسباتی مشابه، هرگز به مقادیر ظاهری و درایه های بزرگ یک بردار اعتماد نکنید. همیشه محاسبات را تا مرحله کسر نهایی (تقسیم بر حاصل ضرب طول ها) پیش ببرید تا اثر سوگیری طول (**Length Bias**) کاملاً خنثی شود. مقادیر نهایی شباهت کسینوسی در این حوزه همواره عددی بین 0 و 1 خواهند بود.

بخش 1) تحلیل اولیه و مبانی تئوری

هدف از این مسئله، سنجش توانایی در اعمال مدل‌های وزن‌دهی خطی (**Linear Weighting Models**)، درک منطق باینری (0 و 1) در ارزیابی حضور کلمات در مناطق مختلف سند، و تجمیع امتیازات بر اساس وزن‌های از پیش تعیین شده است. در سیستم‌های بازایی اطلاعات که از ساختار **امتیازدهی منطقه‌ای وزن دار (Weighted Zone Scoring)** استفاده می‌کنند، سند به بخش‌های معنایی مستقلی (مانند عنوان، چکیده، بدنه، نویسنده و غیره) تقسیم می‌شود. به هر منطقه (i)، یک وزن اختصاصی (g_i) تعلق می‌گیرد که نشان‌دهنده اهمیت نسبی آن بخش در کل سند است. از نظر ریاضی، مجموع این وزن‌ها همواره برابر با 1 است ($\sum g_i = 1$).

برای محاسبه امتیاز نهایی یک سند در برابر یک پرس‌وجو، سیستم از یک تابع تطابق بولی (**Boolean Match Function**) استفاده می‌کند. خروجی این تابع یک متغیر باینری (s_i) است که نشان می‌دهد آیا کلمه مورد جستجو در منطقه i حضور دارد ($s_i = 1$) یا خیر ($s_i = 0$).

رابطه ریاضی حاکم بر این سیستم به شکل زیر تعریف می‌شود:

$$\text{Score} = \sum_{i=1}^n g_i s_i$$

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

گام اول: استخراج داده‌ها و پارامترهای مسئله

بر اساس صورت سوال، سند ما دارای 3 منطقه (**n = 3**) است. ابتدا پارامترهای وزن (g_i) و متغیرهای حضور بولی (s_i) را برای هر منطقه استخراج می‌کنیم:

1. منطقه اول (عنوان - Title):

وزن اختصاص یافته: $g_1 = 0.5$

کوئری کاربر در این منطقه «1 بار» ظاهر شده است. در منطق بولی، تعداد تکرار اهمیتی ندارد؛ همین که حداقل 1 بار

حضور داشته باشد، ارزش آن برابر با 1 خواهد بود. $s_1 = 1$

2. منطقه دوم (چکیده - Abstract):

وزن اختصاص یافته: $g_2 = 0.3$

کوئری کاربر در این منطقه «0 بار» ظاهر شده است (حضور ندارد) $s_2 = 0$

3. منطقه سوم (بدنه - Body):

وزن اختصاص یافته: $g_3 = 0.2$

کوئری در این منطقه «ظاهر شده است» $s_3 = 1$

گام دوم: جایگذاری در فرمول و محاسبه امتیاز

اکنون مقادیر به دست آمده را در فرمول اصلی تجمیع خطی قرار می‌دهیم:

$$\begin{aligned}\text{Score} &= (g_1 \times s_1) + (g_2 \times s_2) + (g_3 \times s_3) \\ &= (0.5 \times 1) + (0.3 \times 0) + (0.2 \times 1) \\ &= 0.5 + 0 + 0.2 = 0.7\end{aligned}$$

امتیاز نهایی این سند برای کوئری وارد شده برابر با **0.7** است. این عدد در نهایت با امتیاز سایر اسناد مقایسه شده و برای مرتب‌سازی (**Ranking**) خروجی نهایی به کاربر استفاده می‌شود.

نکته اول: تفاوت بنیادین تطابق بولی (**Boolean Match**) با فرکانس کلمه (**Term Frequency**) در همین مسئله نهفته است. تابع بولی به تکرارها حساس نیست و صرفاً یک «کلید قطع و وصل» (**On/Off Switch**) است.

پاسخنامه مسئله 6.12: یافتن وزن بهینه سیستم و مقدمه‌ای بر یادگیری ماشین در بازیابی

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، آشنایی با گذر از روش‌های وزن‌دهی دستی (**Heuristic**) به روش‌های مبتنی بر داده (**Data-Driven**)، درک مفهوم تابع هزینه (**Loss Function**)، و چگونگی استفاده از ریاضیات پایه برای کمینه‌سازی خطای سیستم در تطبیق با قضاوت‌های انسانی است.

در سیستم‌های سنتی، طراحان موتور جستجو مقادیر وزن‌ها را بر اساس حدس و گمان یا تجربه تنظیم می‌کردند (مثلاً وزن 0.7 برای عنوان). اما در سیستم‌های مدرن، از رویکردی به نام «**یادگیری برای رتبه‌بندی**» (**Learning to Rank - LTR**) استفاده می‌شود.

در این روش، ما یک **مجموعه داده آموزشی** (**Training Set**) در اختیار داریم که شامل تعدادی پرس‌وجو، اسناد بازیابی شده، و قضاوت‌های ایده‌آل انسانی (**Relevance Judgments**) است. قضاوت انسانی به ما می‌گوید که از نظر یک کاربر واقعی، امتیاز ارتباط این سند با این پرس‌وجو چقدر باید باشد (مثلاً متغیر r). هدف مسئله این است که سیستم به صورت خودکار مقداری برای g (که همواره $0 \leq g \leq 1$ است) پیدا کند که خروجی فرمول سیستم، کمترین فاصله و اختلاف را با قضاوت‌های انسانی داشته باشد.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

گام اول: تشکیل معادله امتیازدهی سیستم (**Scoring Equation**)

بر اساس صورت سوال، سند تنها دارای 2 منطقه است. اگر متغیرهای بولی تطابق برای عنوان و بدنه را به ترتیب **S_{title}** و **S_{body}** بنامیم، فرمول امتیازدهی سیستم برای یک سند و پرس‌وجوی مشخص به شکل زیر بسط داده می‌شود:

$$\text{Score}(q, d) = g \cdot s_{\text{title}} + (1 - g) \cdot s_{\text{body}}$$

گام دوم: تعریف تابع خطا یا هزینه (Loss Function)

برای یافتن بهترین g ، ابتدا باید بتوانیم میزان «اشتباه» یا خطای فعلی سیستم را اندازه‌گیری کنیم. استانداردترین روش برای این کار، استفاده از فرمول **مجموع مربعات خطا (Sum of Squared Errors - SSE)** است.

فرض کنید قضاوت ایده‌آل و واقعی برای هر جفت پرس‌وجو و سند را با $r(q, d)$ نشان دهیم. خطای کل سیستم برای تمامی اسناد موجود در مجموعه آموزشی برابر است با:

$$\text{Error} = \sum_{q,d} \left(\text{Score}(q, d) - r(q, d) \right)^2$$

با جایگذاری معادله‌ی گام اول در این فرمول، به معادله زیر می‌رسیم:

$$\text{Error}(g) = \sum_{q,d} \left(g \cdot s_{\text{title}} + (1 - g) \cdot s_{\text{body}} - r(q, d) \right)^2$$

گام سوم: بهینه‌سازی و کمینه‌سازی خطا از طریق مشتق‌گیری

اکنون ما یک تابع ریاضی (**Error**) داریم که متغیر مجهول آن g است. بر اساس قوانین حساب دیفرانسیل و انتگرال، برای یافتن نقطه کمینه (**Minimum**) یک تابع، باید مشتق اول آن تابع نسبت به متغیر مجهول را محاسبه کرده و برابر با صفر قرار دهیم:

$$\frac{d}{dg} \text{Error}(g) = 0$$

با بسط دادن مشتق معادله‌ی فوق و حل این معادله جبری ساده، دقیق‌ترین و بهینه‌ترین مقدار ممکن برای متغیر g به دست می‌آید که خطای سیستم را روی آن مجموعه داده آموزشی به حداقل مطلق می‌رساند.

گام چهارم: الگوریتم‌های جایگزین برای مقیاس‌های بزرگتر

در دنیای واقعی، اسناد به جای 2 منطقه ممکن است ده‌ها منطقه (نویسنده، تاریخ، تگ‌ها و...) داشته باشند. در چنین شرایطی، حل جبری معادله مشتق بسیار پیچیده می‌شود. در اینجا سیستم‌های بازیابی از الگوریتم‌های تکرارشونده و عددی مانند **گرادیان کاهشی (Gradient Descent)** استفاده می‌کنند تا با قدم‌های کوچک ریاضی، مقدار g را بهینه‌سازی کرده و به صورت تدریجی به کمترین میزان خطا برسند.

بخش 3) جمع‌بندی:

نکته اول: این مسئله نشان‌دهنده‌ی پل ارتباطی بسیار مهمی میان دو علم «بازیابی اطلاعات (IR)» و «یادگیری ماشین (ML)» است. موتورهای جستجو هوشمندی خود را از طریق همین فرآیند بهینه‌سازی خطای توابع به دست می‌آورند.

توصیه برای پیاده‌سازی عملی: برای درک عمیق‌تر این محاسبات، توسعه‌ی یک اسکریپت برنامه‌نویسی با زبان پایتون که بتواند تابع خطا را پیاده‌سازی کرده و مقدار بهینه g را استخراج کند، تمرین بسیار مفیدی است. این رویکرد به ویژه در

طراحی سامانه‌های ارزیابی خودکار (Autograders) و تحلیل عملکرد الگوریتم‌ها روی مجموعه‌های داده با حجم بالا، کاربرد فراوانی دارد.

پاسخنامه مسئله 6.13: نرمال‌سازی حداکثر فرکانس و نقش پارامتر هموارسازی

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک روش‌های پیشرفته‌ی نرمال‌سازی فرکانس کلمات برای مقابله با خطای ناشی از طول سند، و شناخت مفهوم هموارسازی (Smoothing) ریاضی برای حفظ ارزش اطلاعاتی کلمات کم‌تکرار است.

در سیستم‌های بازیابی اطلاعات، اسناد دارای طول‌های متفاوتی هستند. در یک سند بسیار طولانی، یک کلمه ممکن است ده‌ها بار تکرار شود، نه به این دلیل که سند ارتباط عمیقی با آن کلمه دارد، بلکه صرفاً به این دلیل که حجم متن زیاد است. برای خنثی کردن این اثر (Document Length Bias)، یکی از روش‌های کلاسیک، نرمال‌سازی با حداکثر فرکانس (Maximum TF Normalization) است.

در این روش، فرکانس خام کلمه‌ی هدف ($tf_{t,d}$) بر فرکانس «پرتکرارترین کلمه‌ی همان سند» ($tf_{max}(d)$) تقسیم می‌شود. این کار باعث می‌شود تمام مقادیر فرکانس در یک سند، به بازه‌ی بین 0 و 1 نگاشت (Scale) شوند. اما این تقسیم ساده به تنهایی کافی نیست و می‌تواند مشکلاتی ایجاد کند، به همین دلیل ساختار خطی فرمول به شکل زیر اصلاح شده است:

$$w_{t,d} = a + (1 - a) \frac{tf_{t,d}}{tf_{max}(d)}$$

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

گام اول: نقش پارامتر هموارسازی (Smoothing Parameter) یا a چیست؟

پارامتر a به عنوان یک **کف حداقلی (Baseline/Floor)** یا عامل هموارساز در معادله عمل می‌کند. نقش و چرایی استفاده از آن را می‌توان در دو دلیل اصلی خلاصه کرد:

1. جلوگیری از جریمه‌ی شدید کلمات نادر:

فرض کنید در یک سند، پرتکرارترین کلمه (مثلاً حرف اضافه "در") 1000 بار تکرار شده باشد ($tf_{max} = 1000$). کلمه کلیدی مورد جستجوی کاربر (مثلاً "الگوریتم") تنها 1 بار در این سند آمده است. اگر پارامتر a وجود نداشت (یعنی $a = 0$)، وزن این کلمه برابر با کسر بسیار ناچیز 0.001 می‌شد. این جریمه‌ی شدید باعث می‌شود کلمه کلیدی عملاً در محاسبات بی‌اثر شود.

با اضافه شدن پارامتر a ، سیستم تضمین می‌کند که «صرف حضور یک کلمه در متن، بدون توجه به نسبت آن با پرتکرارترین کلمه، یک ارزش پایه‌ای و تضمین‌شده دارد».

2. کاهش واریانس و هموارسازی توزیع:

این پارامتر تغییرات شدید و نوسانات بزرگ بین مقادیر tf در یک متن را ملایم (Smooth) می‌کند. در واقع، این فرمول یک ترکیب خطی (Linear Interpolation) بین یک مقدار ثابت (a) و یک مقدار متغیر (نسبت فرکانس‌ها) است.

گام دوم: معمولاً چه مقداری برای پارامتر a در نظر گرفته می‌شود؟

بر اساس تحقیقات کلاسیک و تجربیات عملی در حوزه بازیابی اطلاعات (به ویژه کارهای جرارد سالتون و کریس باکلی در سیستم **SMART**)، مقدار استاندارد و بهینه‌ی این پارامتر معمولاً **0.4** در نظر گرفته می‌شود (در برخی متون **0.5** نیز ذکر شده است).

اگر مقدار a را **0.4** فرض کنیم، فرمول به شکل عملی زیر درمی‌آید:

$$w_{t,d} = 0.4 + 0.6 \times \frac{tf_{t,d}}{tf_{\max}(d)}$$

تحلیل ریاضی این مقدار:

حداکثر وزن: اگر کلمه‌ی مورد جستجو، همان پرتکرارترین کلمه‌ی سند باشد ($tf_{t,d} = tf_{\max}$)، کسر برابر با 1 می‌شود و وزن نهایی برابر با **1** خواهد شد (چون $0.4 + 0.6 = 1$).

حداقل وزن: اگر کلمه در سند حضور داشته باشد اما فرکانس آن بسیار ناچیز باشد (کسر به سمت 0 میل کند)، وزن کلمه هرگز از **0.4** کمتر نخواهد شد.

بخش 3) جمع‌بندی:

نکته اول: پارامتر a یک مکانیزم دفاعی است. این پارامتر به الگوریتم می‌فهماند که: «درست است که این کلمه نسبت به کلمات دیگر سند خیلی کم تکرار شده است، اما ارزش آن 0 نیست؛ به آن حداقل امتیاز پایه‌ای (a) را بده تا در فرآیند رتبه‌بندی باقی بماند.»

نکته دوم: هنگام مواجهه با فرمول‌های هموارسازی خطی (مانند **Jelinek-Mercer** در مدل‌های زبانی یا همین **Maximum TF**)، به یاد داشته باشید که همیشه ساختار $x + (1 - x) \cdot y$ وجود دارد تا مجموع ضرایب برابر با 1 باقی بماند و مقیاس خروجی از بازه‌ی استاندارد فراتر نرود. هرگز در محاسبات امتحانی، عدد 1 را از ضریب بخش دوم حذف نکنید.

پاسخنامه مسئله 6.14: پردازش سریع با بردارهای پراکنده و نقش نمایه معکوس در محاسبه

شباهت کسینوسی

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک پیچیدگی زمانی (**Time Complexity**) در مدل فضای برداری، شناخت ویژگی پراکندگی (**Sparsity**) در ماتریس‌های متنی، و فراگیری الگوریتم‌های کارآمد مبتنی بر آرایه انباشتگر (**Accumulator Array**) برای محاسبه‌ی سریع ضرب داخلی در مقیاس‌های میلیونی است.

در سیستم‌های بازیابی اطلاعات، واژه‌نامه (**Vocabulary**) یک مجموعه متنی بزرگ ممکن است شامل میلیون‌ها کلمه متمایز

باشد (مثلاً $10^6 \approx |V|$). اگر بخواهیم شباهت کسینوسی یک پرس‌وجو را با یک سند از طریق «ضرب ماتریسی مستقیم» محاسبه کنیم، باید دو بردار با اندازه میلیونی را درایه به درایه در هم ضرب نماییم. این عملیات برای جستجو در میان میلیون‌ها سند، از نظر پردازشی فاجعه‌بار و دارای مرتبه زمانی $O(|V| \times N)$ است.

با این حال، یک ویژگی آماری مهم به کمک ما می‌آید: **پراکندگی شدید بردارها (Extreme Sparsity)**.

یک سند معمولی تنها شامل کسر بسیار کوچکی از کل کلمات واژه‌نامه است (بیش از **99%** درایه‌های بردار سند، 0 هستند). بردار پرس‌وجو (**Query**) از این هم پراکنده‌تر است و معمولاً تنها شامل 2 الی 5 کلمه غیرصفر است. در ریاضیات، ضرب هر عدد در 0 برابر با 0 است؛ بنابراین ساختار داده‌ی **نمایه معکوس (Inverted Index)** طراحی شده است تا سیستم تنها روی ابعاد «غیرصفر» متمرکز شود و از پردازش بی‌دلیل میلیون‌ها 0 جلوگیری کند.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

چگونه نمایه معکوس این محاسبات را سرعت می‌بخشد؟ فرآیند حل این مشکل در سیستم‌های مدرن، از طریق الگوریتمی به نام **پردازش کلمه-به-کلمه (TAAT یا Term-at-a-Time)** و با استفاده از ساختار انباشتگرها انجام می‌شود:

گام اول: حذف ابعاد زائد از طریق بردار پرس‌وجو

در ضرب داخلی $\sum(q_i \times d_i)$ ، اگر وزن کلمه در پرس‌وجو (q_i) برابر با 0 باشد، حاصل‌ضرب آن بُعد قطعاً 0 خواهد شد؛ مهم نیست که وزن آن در سند (d_i) چقدر باشد. بنابراین، سیستم موتور جستجو در قدم اول تمامی کلمات واژه‌نامه را نادیده می‌گیرد و **تنها به سراغ کلماتی می‌رود که در پرس‌وجوی کاربر وجود دارند**.

گام دوم: فراخوانی لیست‌های پستینگ (Posting Lists)

برای هر کلمه موجود در پرس‌وجو، موتور جستجو به ساختار نمایه معکوس مراجعه کرده و لیست پستینگ آن کلمه را استخراج می‌کند. لیست پستینگ، دقیقاً لیست اسنادی است که درایه آن‌ها برای آن کلمه **غیرصفر** است (همراه با وزن کلمه در آن سند). با این کار، سیستم حتی نیازی به بررسی اسنادی که کلمه را ندارند، نخواهد داشت.

گام سوم: استفاده از آرایه انباشتگر (Accumulator Array)

به جای محاسبه‌ی کامل امتیاز یک سند و سپس رفتن به سراغ سند بعدی، سیستم یک آرایه در حافظه (به نام **Scores**) با طول برابر با تعداد کل اسناد ایجاد می‌کند که مقدار اولیه تمامی خانه‌های آن 0 است. فرآیند به این شکل پیش می‌رود:

1. سیستم لیست پستینگ کلمه اول کوئری را می‌خواند.

2. برای هر سند موجود در این لیست، حاصل‌ضرب $q_i \times d_i$ را محاسبه کرده و به خانه مربوط به آن سند در آرایه **Scores**

اضافه (**Accumulate**) می‌کند.

3. این کار برای کلمه دوم و سوم کوئری نیز تکرار می‌شود.

گام چهارم: نرمال‌سازی طول (Length Normalization)

پس از پردازش تمامی کلمات کوئری، خانه‌هایی از آرایه **Scores** که مقدارشان 0 باقی مانده است، به این معنا هستند که هیچ اشتراکی با کوئری نداشته‌اند و به طور کامل از چرخه حذف می‌شوند. در نهایت، سیستم مقادیر غیرصفر در آرایه **Scores** را بر طول از پیش محاسبه‌شده‌ی آن اسناد (طول بردار سند) تقسیم می‌کند تا نرمال‌سازی کسینوسی کامل شود و سپس **K** سند برتر را مرتب‌سازی کرده و برمی‌گرداند.

```
TAAT_Scoring.py

def compute_vector_scores(query_terms, inverted_index, N, doc_lengths):
    # Initialize accumulator array for N documents
    scores = [0.0] * N

    # Process only the non-zero dimensions (terms in the query)
    for term in query_terms:
        if term in inverted_index:
            query_weight = compute_query_weight(term)
            posting_list = inverted_index[term]

            # Update scores only for documents containing the term
            for doc_id, doc_weight in posting_list:
                scores[doc_id] += query_weight * doc_weight

    # Apply length normalization to non-zero scores
    final_results = []
    for doc_id in range(N):
        if scores[doc_id] > 0:
            normalized_score = scores[doc_id] / doc_lengths[doc_id]
            final_results.append((doc_id, normalized_score))

    return sort_by_score(final_results)
```

بخش 4) جمع‌بندی:

نکته اول: نمایه معکوس (**Inverted Index**) یک ماتریس اسپارس را از حالت متراکم (**Dense**) خارج کرده و تنها مقادیر غیرصفر آن را ذخیره می‌کند. این تغییر معماری، پیچیدگی زمانی سیستم را از $O(|V| \times N)$ به $O(|q| \times L)$ کاهش می‌دهد؛ که در آن $|q|$ تعداد کلمات کوئری (مثلاً 3 کلمه) و L میانگین طول لیست پستینگ‌ها است.

نکته دوم: اگر در مسئله‌ای از شما خواسته شد دلیل سرعت بالای سیستم‌های بازیابی مبتنی بر فضای برداری را توضیح دهید، حتماً روی سه کلیدواژه‌ی **پراکندگی بردارها (Sparsity)**، **نادیده گرفتن ابعاد صفر**، و **آرایه انباشتگر (Accumulators)** تأکید کنید. سیستم بازیابی، ریاضیات جبر خطی را دور نمی‌زند، بلکه با هوشمندی، بخش‌های بی‌حاصل ضرب ماتریسی را حذف می‌کند.

پاسخنامه مسئله 6.15: رمزگشایی نمادگذاری SMART و مقایسه شمای وزن‌دهی

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، آشنایی با نمادگذاری استاندارد سیستم کلاسیک **SMART**، درک معماری سه‌بخشی در محاسبه وزن بردارها (**TF, DF** و نرمال‌سازی)، و شناخت استدلال ریاضیاتی در تفاوت وزن‌دهی بین سند و پرس‌وجو است. در سیستم‌های بازیابی اطلاعات، فرمول‌های وزن‌دهی بسیار متنوع هستند. برای استانداردسازی و توصیف سریع ترکیبات مختلف این فرمول‌ها، از نمادگذاری سیستم **SMART** (یکی از پیشگامان موتورهای جستجوی علمی در دانشگاه کرنل) استفاده می‌شود.

در این نمادگذاری، شمای وزن‌دهی به صورت یک عبارت دوقسمتی **ddd.qqq** نمایش داده می‌شود که در آن **ddd** نمایانگر الگوریتم وزن‌دهی برای **سند (Document)** و **qqq** نمایانگر الگوریتم وزن‌دهی برای **پرس‌وجو (Query)** است.

هر بخش از 3 حرف تشکیل شده است که به ترتیب نشان‌دهنده‌ی 3 مرحله‌ی اساسی محاسبات هستند:

1. **حرف اول:** نحوه محاسبه فرکانس کلمه (**Term Frequency**)

2. **حرف دوم:** نحوه محاسبه فرکانس سند (Document Frequency)

3. **حرف سوم:** نحوه نرمال سازی طول بردار (Normalization)

بخش 2) حل گام به گام و تشریح خواسته های مسئله

نماد **Inc.Itc** یکی از موفق ترین و رایج ترین پیکربندی ها در بازیابی فضای برداری است. اکنون هر یک از این حروف را به صورت دقیق رمزگشایی می کنیم:

الف) رمزگشایی **Inc** (محاسبه وزن اسناد)

این 3 حرف مراحل ساخت بردار سند را تعیین می کنند:

حرف **l** (Logarithmic tf)

نشان دهنده استفاده از مقیاس دهی زیرخطی یا لگاریتمی برای فرکانس کلمه است. به جای فرکانس خام (tf)، از فرمول $1 + \log_{10}(tf)$ استفاده می شود تا اثر تکرار بیش از حد کلمات در یک سند مهار شود.

حرف **n** (No df / No idf)

نشان دهنده این است که در بردار سند، از هیچ گونه فرکانس معکوس سند (**IDF**) استفاده **نمی شود** و مقدار آن معادل عدد 1 در نظر گرفته می شود. (دلیل این امر در بخش جمع بندی توضیح داده شده است).

حرف **c** (Cosine normalization)

نشان دهنده استفاده از نرمال سازی کسینوسی است. یعنی در نهایت، تمام درایه های بردار سند بر طول آن بردار (نرم اقلیدسی) تقسیم می شوند تا بردارهایی با طول واحد (1) به دست آید و سوگیری اسناد طولانی خنثی شود.

ب) رمزگشایی **Itc** (محاسبه وزن پرس وجو)

این 3 حرف مراحل ساخت بردار پرس وجو را تعیین می کنند:

حرف **l** (Logarithmic tf)

مشابه سند، در اینجا نیز اگر کلمه ای در پرس وجو بیش از 1 بار تکرار شده باشد، وزن آن با فرمول $1 + \log_{10}(tf)$ محاسبه می شود.

حرف **t** (Inverse document frequency - idf)

نشان دهنده اعمال فرکانس معکوس سند است. در این مرحله، وزن کلمه در فرمول استاندارد $\log(\frac{N}{df})$ ضرب می شود. بنابراین کلمات عمومی و بی ارزش موجود در کوئری جریمه شده و کلمات تخصصی تقویت می شوند.

حرف **c** (Cosine normalization)

مشابه سند، بردار پرس وجو نیز با تقسیم بر طول خودش نرمال سازی می شود تا مقیاس محاسبات در ضرب داخلی (**Dot Product**) استاندارد باقی بماند.

بخش 3) جمع بندی:

چرا **IDF** فقط روی کوئری اعمال می شود (حرف **t**) و روی سند اعمال نمی شود (حرف **n**)؟

این یکی از مهم ترین سوالات مفهومی در طراحی سیستم های **IR** است. از آنجایی که امتیاز نهایی حاصل ضرب داخلی

بردار کوثری و بردار سند است، اگر **IDF** را روی هر دو بردار اعمال کنیم، در واقع وزن نهایی را در IDF^2 (مربع **IDF**) ضرب کرده‌ایم که باعث می‌شود اثر کلمات نادر به شدت و به صورت غیرطبیعی بزرگ شود. از نظر ریاضی، اعمال **IDF** بر روی فقط یکی از بردارها (معمولاً کوثری) برای رسیدن به جریمه‌ی مطلوب کلمات عمومی کفایت می‌کند.

نکته: نمادگذاری **SMART** را به عنوان یک چک‌لیست محاسباتی در نظر بگیرید. اگر طراح سوال شمای **Inc.Itc** را به شما داد، بلافاصله باید بدانید که برای ساخت بردار سند نیازی به محاسبه $\log(\frac{N}{df})$ ندارید و این کار به شدت در زمان محاسبه شما در جلسه امتحان صرفه‌جویی خواهد کرد.

پاسخنامه مسئله 6.16: نقص نرمال‌سازی کسینوسی و معرفی نرمال‌سازی محوری

بخش 1) تحلیل اولیه و مبانی تئوری

هدف از این مسئله، درک محدودیت‌های هندسی در مدل فضای برداری، شناخت پدیده «جریمه‌ی ناعادلانه اسناد طولانی»، و آشنایی با روش‌های پیشرفته‌ی نرمال‌سازی مبتنی بر توزیع آماری و طول میانگین اسناد (مانند آنچه در الگوریتم‌های مدرن نظیر **BM25** استفاده می‌شود) است.

نرمال‌سازی کسینوسی (تقسیم بردار سند بر طول آن یا $|\vec{d}|$) یک دستاورد بزرگ در سیستم‌های اولیه بازیابی اطلاعات بود، زیرا مانع از آن می‌شد که اسناد صرفاً به خاطر طولانی بودنشان در نتایج جستجو تسلط پیدا کنند. با این حال، محققان متوجه شدند که این روش از آن سوی بام افتاده است؛ به این معنا که نرمال‌سازی کسینوسی، اسناد طولانی را **بیش از حد نیاز** جریمه می‌کند.

برای رفع این نقص سیستماتیک، ایده **نرمال‌سازی محوری طول سند (Pivoted Document Length Normalization)** مطرح شد. این روش به جای تکیه مطلق بر طول هندسی بردار، یک «نقطه محور» (**Pivot**) بر اساس میانگین طول اسناد مجموعه تعریف می‌کند و شیب جریمه را به صورت کنترل‌شده تغییر می‌دهد.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

الف) نرمال‌سازی کسینوسی چه جریمه‌ی ناعادلانه‌ای برای اسناد طولانی در نظر می‌گیرد؟

برای درک این جریمه، باید ماهیت اسناد طولانی را کالبدشکافی کنیم. یک سند چرا طولانی می‌شود؟ معمولاً به این دلیل که آن سند دارای **گستره‌ی وسیع‌تری از موضوعات (Multi-topic)** و در نتیجه دایره واژگان (**Vocabulary**) بسیار بزرگتری است. در فضای برداری، هرچه تعداد کلمات یکتای یک سند بیشتر باشد، بردار آن سند ابعاد غیرصفر بیشتری خواهد داشت و در نتیجه طول آن بردار $|\vec{d}|$ به شدت بزرگ می‌شود.

حال فرض کنید کاربری درباره‌ی موضوع "X" جستجو می‌کند. دو سند داریم:

1. سند کوتاه: یک مقاله‌ی 1 صفحه‌ای که منحصراً درباره "X" است.

2. سند طولانی: یک کتاب مرجع 1000 صفحه‌ای که یک فصل کامل و بسیار جامع (مثلاً 50 صفحه) را به "X" اختصاص داده و بقیه کتاب درباره موضوعات دیگر است.

نقص سیستم: از دیدگاه کاربر، فصل 50 صفحه‌ای کتاب مرجع ممکن است بسیار غنی‌تر و مرتبط‌تر از آن مقاله‌ی 1 صفحه‌ای باشد. اما در نرمال‌سازی کسینوسی، چون کتاب مرجع دارای هزاران کلمه‌ی غیرمرتبط با "X" در سایر فصول خود است، مخرج

کسر (یعنی $|\vec{d}|$) برای آن به عدد گول‌پیکری تبدیل می‌شود. با تقسیم امتیازِ تطابق بر این مخرجِ بزرگ، وزن کلمه‌ی "X" در کتاب مرجع به شدت افت کرده (**Shrinkage**) و سند کوتاه به ناحق در رتبه بالاتری قرار می‌گیرد.

ب) چرا نرمال‌سازی محوری (Pivoted Normalization) به کار می‌رود؟

هدف از نرمال‌سازی محوری، هم‌راستا کردن «احتمال بازیابی سند» با «احتمال واقعی مرتبط بودن سند» است. این الگوریتم بر این اصل استوار است که طول بردار سند به تنهایی معیار خوبی برای جریمه نیست و باید آن را نسبت به میانگین طول اسناد پایگاه داده (Average Document Length - avdl) سنجید.

در این روش، جریمه‌ی طول با فرمولی شبیه به تابع خطی زیر تعدیل می‌شود:

$$\text{Normalizer} = (1 - s) + s \frac{|\vec{d}|}{\text{avdl}}$$

که در آن s (Slope یا شیب) یک پارامتر تنظیمی است.

نحوه عملکرد (چرخش حول محور):

1. اسناد با طول متوسط ($|\vec{d}| = \text{avdl}$): مقدار کسر برابر با 1 می‌شود و مخرج بدون تغییر باقی می‌ماند. (نقطه محور)

2. اسناد بسیار طولانی ($|\vec{d}| > \text{avdl}$): به جای اعمال کل جریمه‌ی کسینوسی، ضریب s (که معمولاً کمتر از 1 است) شدت رشد مخرج را مهار می‌کند (تضعیف جریمه).

3. اسناد بسیار کوتاه ($|\vec{d}| < \text{avdl}$): مخرج کمی بزرگتر از حالت کسینوسی خام محاسبه می‌شود تا جلوی برتری ناعادلانه اسناد بسیار کوتاه نیز گرفته شود (افزایش جریمه).

بخش 3) جمع‌بندی:

نکته اول: به زبان ساده، نرمال‌سازی کسینوسی فرض می‌کند که «تمام یک سند طولانی باید درباره کوئری کاربر باشد تا امتیاز بالایی بگیرد». اما نرمال‌سازی محوری واقع‌بینانه‌تر است و می‌پذیرد که «یک سند طولانی حق دارد به موضوعات دیگر هم پرداخته باشد و نباید به خاطر داشتن اطلاعات اضافه، در موضوع اصلی خودش نقره‌داغ شود.»

نکته دوم: اگر در مباحث پیشرفته‌ی بازیابی اطلاعات یا در تحلیل موتورهای جستجوی واقعی با الگوریتم **Okapi BM25** مواجه شدید، به خاطر داشته باشید که پارامتر معروف **b** در فرمول **BM25**، دقیقاً پیاده‌سازی همین مفهوم «نرمال‌سازی محوری طول سند» است که به سیستم اجازه می‌دهد میزان سخت‌گیری روی طول اسناد را بهینه‌سازی کند.

پاسخنامه مسئله 6.17: چالش یادگیری وزن‌ها و جمع‌آوری داده‌های پایه (Ground Truth)

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک پیش‌نیازهای داده‌محور برای اجرای رویکرد «یادگیری برای رتبه‌بندی» (**Learning to Rank**)، شناخت مفهوم قضاوت‌های ارتباط (**Relevance Judgments**) به عنوان قطب‌نمای الگوریتم، و مقایسه روش‌های عملیاتی در

موتورهای جستجوی واقعی برای تأمین این داده‌ها است.

الگوریتم‌های یادگیری ماشین برای بهینه‌سازی پارامترهای خود (مانند یافتن وزن ایده‌آل مناطق یا g) نیازمند یک قطب‌نما یا الگو هستند تا میزان خطای خود را بسنجند. به این الگوی مرجع و بی‌نقص، **داده‌های پایه (Ground Truth)** گفته می‌شود.

بدون وجود **Ground Truth**، ماشین نمی‌تواند تشخیص دهد که آیا تغییر دادن وزن منطقه «عنوان» از 0.5 به 0.7 باعث بهبود نتایج جستجو شده است یا باعث افت کیفیت آن. بنابراین، پیش از هرگونه کدنویسی یا بهینه‌سازی ریاضی، مهندسی داده‌های آموزشی (**Training Data**) حیاتی‌ترین قدم در طراحی سیستم‌های بازبایی مدرن است.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

الف) نوع داده‌های مورد نیاز به عنوان Ground Truth

برای اجرای موفق الگوریتم‌های یادگیری وزن‌ها، داده‌ی **Ground Truth** باید در قالب مجموعه‌ای از **سه‌تایی‌های آموزشی (Training Tuples)** ساختاریافته باشد. هر نمونه در این مجموعه داده از 3 عنصر اساسی تشکیل می‌شود:

- 1. پرس‌وجو (q):** یک کوئری واقعی که ممکن است توسط کاربر در سیستم وارد شود.
- 2. سند (d):** یکی از اسناد موجود در پایگاه داده که برای این کوئری ارزیابی می‌شود.
- 3. قضاوت ارتباط یا امتیاز ایده‌آل (r):** یک برچسب (**Label**) که نشان می‌دهد سند d تا چه حد به پرس‌وجوی q مرتبط است.

این برچسب ارتباط (r) می‌تواند به 2 صورت تعریف شود:

باینری (Binary): شامل 2 حالت مرتبط (1) و نامرتبط (0).

درجه‌بندی‌شده (Graded): یک طیف امتیازی؛ به عنوان مثال از 0 (کاملاً نامرتبط) تا 4 (ارتباط کامل و دقیق).

سیستم با دریافت هزاران نمونه از این سه‌تایی‌ها (q, d, r)، تلاش می‌کند وزن‌هایی را بیاموزد که خروجی فرمول رتبه‌بندی‌اش، بیشترین تطابق را با مقادیر r داشته باشد.

ب) دو روش برای جمع‌آوری این داده‌ها در یک موتور جستجوی واقعی

در مقیاس صنعتی و موتورهای جستجویی مانند گوگل یا بینگ، جمع‌آوری این سه‌تایی‌ها به 2 روش کاملاً متمایز و مکمل انجام می‌پذیرد:

روش اول: ارزیابی صریح انسانی (Explicit Human Assessment / Editorial Judgments)

در این روش، موتورهای جستجو از انسان‌ها (ارزیاب‌های کنترل کیفیت یا متخصصان دامنه) استفاده می‌کنند. به یک ارزیاب انسانی یک کوئری و یک لیست از اسناد داده می‌شود و از او خواسته می‌شود تا با دقت متن اسناد را بخواند و بر اساس دستورالعمل‌های استاندارد، به ارتباط آن‌ها با کوئری امتیاز دهد.

مزیت: کیفیت بسیار بالا، دقت معنایی و عاری بودن از خطاهای الگوریتمی (بهترین نوع **Ground Truth**).

چالش: بسیار پرهزینه، زمان‌بر و غیرقابل مقیاس‌پذیری برای میلیاردها کوئری جدید در روز. (غالباً از پلتفرم‌های جمع‌سپاری مانند **Amazon Mechanical Turk** برای کاهش هزینه‌ها استفاده می‌شود).

روش دوم: بازخورد ضمنی کاربران (Implicit User Feedback / Click-through Logs)

در این روش، سیستم به جای پرسش مستقیم، رفتار طبیعی کاربران واقعی را در لاگ‌های موتور جستجو (**Search Logs**) تحلیل می‌کند. زمانی که کاربر کوئری q را جستجو می‌کند، موتور جستجو اسنادی را نمایش می‌دهد. اگر کاربر روی سند سوم کلیک کند، مدت زمان زیادی را در آن صفحه بگذراند (**Dwell Time**) و دیگر به صفحه نتایج برنگردد، سیستم این رفتار را به عنوان یک «سیگنال ضمنی ارتباط» برای آن سند در نظر می‌گیرد.

مزیت: کاملاً رایگان، در حجم عظیم (**Big Data**) و در لحظه تولید می‌شود، و منعکس‌کننده نیاز واقعی کاربر است.

چالش: دارای نویز (**Noise**) بسیار زیاد است. (مثلاً کاربر ممکن است صرفاً به خاطر جذابیتِ کاذبِ یک تیتِر روی آن کلیک کند، در حالی که محتوای آن نامرتب است؛ که به این پدیده **Clickbait** می‌گویند).

بخش 3) جمع‌بندی:

نکته ای برای طراحی معماری: در موتورهای جستجوی پیشرفته، هیچ‌گاه به یک روش اکتفا نمی‌شود. معمولاً از «ارزیابی انسانی» برای ساخت مجموعه داده‌های تست (**Test Sets**) طلایی و تنظیم اولیه معماری استفاده می‌شود، و از «بازخورد ضمنی لاگ‌ها» برای تغذیه پیوسته مدل‌های یادگیری عمیق و بهینه‌سازی روزمره (**Fine-tuning**) بهره می‌گیرند.

نکته دوم: اگر سوالی پیرامون چالش‌های یادگیری برای رتبه‌بندی (**LTR**) مطرح شد، به یاد داشته باشید که گلوگاه اصلی این الگوریتم‌ها، قدرت پردازشی یا پیچیدگی ریاضیاتی نیست؛ بلکه **کیفیت و کمیتِ قضاوت‌های انسانی (Relevance Judgments)** است. هر الگوریتم هوشمندی، اگر با داده‌های پر از نویز تغذیه شود، خروجی ضعیفی خواهد داشت (**Garbage In, Garbage Out**).

پاسخنامه مسئله 6.18: چالش‌های وزن‌دهی در تجارت الکترونیک و معماری وزن‌های پویا

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، آشنایی با محدودیت‌های مدل‌های خطی ایستا (**Static Linear Models**) در دنیای واقعی، درک مفهوم «نیت پرس‌وجو» (**Query Intent**)، و فراگیری نحوه ارتقای معماری سیستم‌های بازیابی از طریق توابع وزن‌دهی وابسته به پرس‌وجو و یادگیری ماشین است.

در معماری استاندارد **Weighted Zone Scoring**، فرض بر این است که اهمیت مناطق مختلف یک سند (مانند عنوان، بدنه و...) ثابت است و برای تمام کاربران و تمامی کوئری‌ها یکسان عمل می‌کند. این مدل در پایگاه‌های داده همگن (مانند مقالات علمی) به خوبی کار می‌کند، اما در سامانه‌های ناهمگن مانند تجارت الکترونیک (**E-commerce**) کاملاً شکست می‌خورد.

همان‌طور که در صورت مسئله اشاره شده است، رفتار و نیاز کاربر بسته به کلماتی که جستجو می‌کند تغییر می‌یابد:

جستجوی نام تجاری (Brand Search): کاربری که "Apple" را جستجو می‌کند، نیت صریح و مشخصی

(**Navigational**) دارد. در اینجا حضور کلمه در منطقه **Brand** باید وزن و امتیاز قاطع و برتری داشته باشد.

جستجوی توصیفی/ویژگی (Feature Search): کاربری که "لپ‌تاپ سبک" را جستجو می‌کند، به دنبال کشف و

مقایسه (**Informational**) است. در اینجا مناطق **Description** و **Reviews** به دلیل داشتن اطلاعات فنی و تجربیات

کاربری، اهمیت بسیار بالاتری پیدا می‌کنند.

بنابراین، سیستم نیازمند ارتقا از حالت **تخصیص وزن ایستا (Static Allocation)** به حالت **تخصیص وزن پویا (Dynamic Allocation)** است.

بخش 2) حل گام‌به‌گام و ارتقای معماری سیستم

برای حل این چالش بنیادین، معماری موتور جستجو باید در 4 فاز متوالی ارتقا یابد:

فاز اول: درک نیت پرس‌وجو (Query Intent Classification)

پیش از آنکه موتور جستجو لیست پستینگ‌ها (**Posting Lists**) را پردازش کند، باید یک لایه پردازشی جدید به نام «تحلیل‌گر پرس‌وجو» اضافه شود. وظیفه این لایه، طبقه‌بندی کوئری وارد شده است. سیستم با استفاده از واژه‌نامه‌های تخصصی (**Dictionaries**) یا مدل‌های زبانی تشخیص می‌دهد که کوئری متعلق به کدام دسته است.

به عنوان مثال:

اگر کوئری در **Brand_Dictionary** وجود داشت $\Leftarrow$ دسته: **Brand_Intent**

اگر کوئری شامل صفات و ویژگی‌ها بود $\Leftarrow$ دسته: **Feature_Intent**

فاز دوم: بازتعریف ریاضیاتی فرمول وزن‌دهی (Dynamic Functions)

در فرمول کلاسیک، g_i یک ثابت عددی بود. در معماری جدید، ما متغیر g_i را به یک **تابع وابسته به کوئری** ($g_i(q)$) تبدیل می‌کنیم.

رابطه ریاضی به شکل زیر ارتقا می‌یابد:

$$\text{Score}(q, d) = \sum_{i=1}^n g_i(q) \cdot s_i$$

در این معماری، به جای یک بردار وزن ثابت، موتور جستجو یک **ماتریس وزن‌ها (Weight Matrix)** را در اختیار دارد و بر اساس خروجی فاز اول، ردیف مناسب را انتخاب می‌کند:

نوع نیت (Intent)	gbrand	gtitle	gdescription	greviews	مجموع
Brand_Intent	0.7	0.2	0.05	0.05	1.0
Feature_Intent	0.1	0.3	0.4	0.2	1.0

فاز سوم: پیاده‌سازی سودو کد

برای پیاده‌سازی این منطق در لایه بک‌اند (**Backend**) سیستم، ساختار کد به شکل پویایی تغییر می‌کند تا پیش از محاسبه نهایی، ماتریس مناسب بارگذاری شود.

dynamic_weighting.py

```
def get_dynamic_weights(query, brand_dictionary):
    if query in brand_dictionary:
        intent = "BRAND_SEARCH"
        return {"brand": 0.7, "title": 0.2, "description": 0.05, "reviews": 0.05}
```

```
else:
    intent = "FEATURE_SEARCH"
    return {"brand": 0.1, "title": 0.3, "description": 0.4, "reviews": 0.2}

def compute_dynamic_score(query, document, brand_dictionary):
    weights = get_dynamic_weights(query, brand_dictionary)
    score = 0.0

    score += weights["brand"] * document.match("brand", query)
    score += weights["title"] * document.match("title", query)
    score += weights["description"] * document.match("description", query)
    score += weights["reviews"] * document.match("reviews", query)

    return score
```

فاز چهارم: گذار نهایی به Learning to Rank (LTR)

اگرچه تخصیص وزن‌های پویا به صورت ماتریسی یک بهبود بزرگ است، اما در مقیاس آمازون (با میلیون‌ها کوئری متنوع)، تعریف دستی این دسته‌بندی‌ها غیرممکن می‌شود.

ارتقای نهایی سیستم، حذف کامل فرمول خطی و استفاده از درخت‌های تصمیم (**Decision Trees**) یا شبکه‌های عصبی (**Neural Networks**) است. در این حالت، حضور کوئری در منطقه **Brand**، صرفاً یک **ویژگی (Feature)** است که به الگوریتم ماشین لرنینگ داده می‌شود. ماشین خودش یاد می‌گیرد که اگر کوئری حاوی کلمات نام‌آشنا بود، به صورت خودکار اهمیت ویژگی **Brand** را در محاسبه نهایی بالا ببرد.

بخش 3) جمع‌بندی:

نکته اول: یک موتور جستجوی مدرن، هرگز کورکورانه به جستجوی کلمات نمی‌پردازد. اولین و مهم‌ترین قدم در سیستم‌های پیشرفته، «**فهمیدن خواسته‌ی کاربر**» است. فرمول‌های ریاضی و وزن‌ها باید تابعی مستقیم از این خواسته باشند.

نکته دوم: در مسائل طراحی سیستم‌های **IR** ناهمگن (**Heterogeneous**)، همیشه اشاره کنید که وزن g نباید ایستا باشد. ایده تبدیل وزن‌ها به تابعی از ویژگی‌های کوئری ($g(q)$) نشان‌دهنده‌ی درک عمیق شما از تفاوت میان «نظریه کلاسیک بازیابی» و «مهندسی واقعی موتورهای جستجو» است. همچنین در کدنویسی‌ها دقت کنید که منطق و نام‌گذاری متغیرها همیشه از یک استاندارد مشخص و تمیز (مانند حروف لاتین) پیروی کند تا از مشکلات انکودینگ (**Encoding**) در مقیاس کلان جلوگیری شود.

پاسخنامه مسئله 6.19: استخراج فرمول خطای بهینه به صورت تحلیلی

(Deriving Optimal Error Formula Algebraically)

بخش 1) تحلیل اولیه و تعریف متغیرها

هدف این مسئله، آشنایی با مبانی ریاضیاتی «یادگیری برای رتبه‌بندی» (**Learning to Rank**)، توانایی مدل‌سازی ریاضی یک سیستم امتیازدهی، و استفاده از حساب دیفرانسیل برای کمینه‌سازی تابع هزینه و یافتن پارامترهای بهینه در مدل فضای برداری است.

برای مدل‌سازی دقیق این مسئله، ابتدا باید متغیرهای سیستم را به زبان ریاضی تعریف کنیم. فرض می‌کنیم یک مجموعه داده‌ی آموزشی (**Training Set**) به نام T در اختیار داریم که شامل تعدادی جفت پرس‌وجو و سند است.

برای هر نمونه‌ی j در این مجموعه (که در آن $j \in T$)، متغیرهای زیر را تعریف می‌کنیم:

متغیر s_{1j} : وضعیت تطابق کوثری در منطقه‌ی اول (مثلاً عنوان). مقداری باینری است ($s_{1j} \in \{0, 1\}$).

متغیر s_{2j} : وضعیت تطابق کوثری در منطقه‌ی دوم (مثلاً بدنه). مقداری باینری است ($s_{2j} \in \{0, 1\}$).

متغیر r_j : قضاوت انسانی (**Ground Truth**) برای میزان ارتباط. مقداری باینری است ($r_j \in \{0, 1\}$).

متغیر g : وزن اختصاص یافته به منطقه‌ی اول.

متغیر $1 - g$: وزن اختصاص یافته به منطقه‌ی دوم.

بخش 2) حل گام به گام و اثبات ریاضی

گام اول: تعریف تابع امتیازدهی سیستم (System Score)

امتیاز نهایی که موتور جستجو برای نمونه‌ی j محاسبه می‌کند، یک ترکیب خطی از تطابق مناطق است:

$$\text{score}_j = g \cdot s_{1j} + (1 - g) \cdot s_{2j}$$

برای ساده‌سازی محاسبات بعدی، این معادله را بر اساس g فاکتورگیری می‌کنیم:

$$\text{score}_j = s_{2j} + g(s_{1j} - s_{2j})$$

گام دوم: تشکیل تابع هزینه (Cost Function)

هدف ما این است که امتیاز سیستم (score_j) تا حد امکان به قضاوت انسانی (r_j) نزدیک باشد. برای اندازه‌گیری میزان این خطا در کل مجموعه داده‌ی آموزشی، از فرمول **مجموع مربعات خطا (Sum of Squared Errors - SSE)** به عنوان تابع هزینه $J(g)$ استفاده می‌کنیم:

$$J(g) = \sum_{j \in T} (\text{score}_j - r_j)^2$$

با جایگذاری تابع امتیاز (از گام اول) در معادله‌ی فوق، تابع هزینه به شکل زیر درمی‌آید:

$$J(g) = \sum_{j \in T} \left(s_{2j} + g(s_{1j} - s_{2j}) - r_j \right)^2$$

گام سوم: کمینه‌سازی خطا از طریق مشتق‌گیری (Optimization)

در ریاضیات، برای یافتن نقطه‌ی مینیمم یک تابع، باید مشتق اول آن نسبت به متغیر مستقل (در اینجا g) را محاسبه کرده و برابر با صفر قرار دهیم.

با استفاده از قاعده زنجیره‌ای (**Chain Rule**) مشتق می‌گیریم. مشتق عبارت داخل پرانتز نسبت به g ، برابر با $(s_{1j} - s_{2j})$ است. بنابراین:

$$\frac{\partial J}{\partial g} = \sum_{j \in T} 2(s_{2j} + g(s_{1j} - s_{2j}) - r_j)(s_{1j} - s_{2j}) = 0$$

گام چهارم: حل جبری برای یافتن g بهینه

برای به دست آوردن فرمول نهایی g ، معادله‌ی به دست آمده در گام سوم را بسط داده و ساده می‌کنیم. ابتدا طرفین را بر عدد 2 تقسیم می‌کنیم تا حذف شود:

$$\sum_{j \in T} (s_{2j} + g(s_{1j} - s_{2j}) - r_j)(s_{1j} - s_{2j}) = 0$$

اکنون مجموع $(\sum)$ را روی دو بخش معادله توزیع می‌کنیم و g را که یک مقدار ثابت است از مجموع اول بیرون می‌کشیم:

$$g \sum_{j \in T} (s_{1j} - s_{2j})^2 + \sum_{j \in T} (s_{2j} - r_j)(s_{1j} - s_{2j}) = 0$$

عبارت دوم را به سمت راست تساوی منتقل می‌کنیم (علامت آن قرینه می‌شود):

$$g \sum_{j \in T} (s_{1j} - s_{2j})^2 = - \sum_{j \in T} (s_{2j} - r_j)(s_{1j} - s_{2j})$$

با اعمال علامت منفی در پرانتز اول سمت راست، $(s_{2j} - r_j)$ به $(r_j - s_{2j})$ تبدیل می‌شود:

$$g \sum_{j \in T} (s_{1j} - s_{2j})^2 = \sum_{j \in T} (r_j - s_{2j})(s_{1j} - s_{2j})$$

در نهایت، با تقسیم طرفین بر ضریب g ، **فرمول تحلیلی خطای بهینه** به دست می‌آید:

$$g = \frac{\sum_{j \in T} (r_j - s_{2j})(s_{1j} - s_{2j})}{\sum_{j \in T} (s_{1j} - s_{2j})^2}$$

بخش 3) جمع‌بندی:

تحلیل مخرج کسر: در مخرج فرمول نهایی، عبارت $(s_{1j} - s_{2j})^2$ را داریم. از آنجا که متغیرهای ما باینری هستند، این عبارت تنها زمانی برابر با 1 می‌شود که دو منطقه با هم اختلاف داشته باشند (یعنی یکی 1 و دیگری 0 باشد). اگر هر دو منطقه وضعیت یکسانی داشته باشند (هر دو 1 یا هر دو 0 باشند)، حاصل این پرانتز 0 می‌شود.

نکته مهم: این اثبات ریاضی یک حقیقت بسیار مهم را در طراحی موتورهای جستجو فاش می‌کند: «برای یادگیری اینکه کدام منطقه (**Zone**) اهمیت بیشتری دارد، سیستم نیازی به بررسی اسنادی که در هر دو منطقه کلمه را دارند، ندارد! وزن بهینه g منحصراً توسط اسنادی تعیین می‌شود که مناطق آن‌ها با یکدیگر اختلاف و تضاد دارند.» این درک عمیق به کاهش چشمگیر بار پردازشی در فرآیند آموزش مدل (**Model Training**) کمک شایانی می‌کند.

پاسخنامه مسئله 6.20: چالش پویایی کلکسیون و استراتژی‌های به‌روزرسانی نمایه‌ی معکوس

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک چالش‌های معماری در سیستم‌های بازیابی بی‌درنگ (**Real-Time IR**)، شناخت توازن میان «دقت مطلق ریاضی» و «کارایی مهندسی»، و فراگیری استراتژی‌های صنعتی برای مدیریت نمایه‌های در حال رشد است. در محیط‌های پویای عملیاتی مانند خبرگزاری‌ها، موتور جستجو با یک **پیکره‌ی متنی در حال رشد (Dynamic Corpus)** مواجه است. بر اساس فرمول استاندارد **$idf = \log(N/df)$** ، با ورود هر سند جدید، اندازه کل مجموعه (N) افزایش می‌یابد. همچنین اگر کلمات سند جدید در واژه‌نامه موجود باشند، مقدار فرکانس سند (df) برای آن کلمات نیز تغییر می‌کند.

ریشه‌ی مشکل (Bottleneck):

اگر سیستم بخواهد با ورود هر خبر جدید، مقدار **idf** را برای تمام کلمات بازنویسی و به‌روزرسانی کند، به دلیل حجم عظیم واژه‌نامه (**Vocabulary**) و ساختار نمایه‌ی معکوس روی دیسک، بار پردازشی (**I/O**) وحشتناکی به سرور تحمیل می‌شود که سیستم را عملاً از کار می‌اندازد. مهندسی نرم‌افزار در اینجا نیازمند فاصله‌گرفتن از محاسبه‌ی لحظه‌ای و حرکت به سمت **تقریب‌های کارآمد (Efficient Approximations)** است.

بخش 2) حل گام‌به‌گام و استراتژی‌های پیشنهادی معماری

برای حل این چالش در سیستم‌های مقیاس‌پذیر، معماران موتورهای جستجو معمولاً از ترکیب 3 استراتژی مهندسی زیر استفاده می‌کنند:

استراتژی اول: به‌روزرسانی دوره‌ای و دسته‌ای (**Batch & Periodic Updates**)

اولین و ساده‌ترین راهکار، متوقف کردن محاسبات لحظه‌ای است. سیستم به جای هر روز، مقادیر **idf** را به صورت دوره‌ای (مثلاً آخر هر هفته یا ماه) و در ساعات افت ترافیک (مثل 3 بامداد) مجدداً محاسبه می‌کند.

توجه ریاضی: در دیتابیس‌های بزرگ، تغییرات روزانه از نظر آماری ناچیز هستند. فرض کنید پایگاه داده خبرگزاری در حال حاضر شامل 10,000,000 سند است. اضافه شدن 10,000 خبر جدید، حجم کل را تنها 0.1 درصد افزایش می‌دهد. با توجه به اینکه فرمول **idf** بر پایه‌ی **لگاریتم** استوار است، این تغییر جزئی، اثر ملموسی بر خروجی لگاریتم و در نتیجه رتبه‌بندی نهایی نخواهد گذاشت و استفاده از مقادیر قبلی (**Stale IDF**) کاملاً قابل قبول است.

استراتژی دوم: معماری نمایه‌ی دوگانه (**Split Index Architecture**)

این روش استانداردترین معماری در موتورهای مانند **Elasticsearch** است. سیستم به جای یک نمایه، از 2 نمایه استفاده می‌کند:

1. **نمایه اصلی (Main Index):** بسیار بزرگ، ایستا (Static) و روی دیسک ذخیره می‌شود. مقادیر **idf** در این نمایه ثابت هستند.

2. **نمایه کمکی/حافظه (Auxiliary/Memory Index):** یک نمایه کوچک و سریع که در **RAM** قرار دارد و اخبار جدید روزانه وارد آن می‌شوند.

هنگامی که کاربر جستجو می‌کند، سیستم کوئری را همزمان به هر دو نمایه می‌فرستد و نتایج را در نهایت ادغام (**Merge**) می‌کند. برای محاسبه وزن در نمایه کمکی، سیستم از همان مقادیر **idf** نمایه اصلی استفاده می‌کند (قرض گرفتن **IDF**). پس از گذشت یک بازه زمانی یا رسیدن حجم نمایه کمکی به یک حد نصاب، این 2 نمایه با هم ترکیب شده و یک نمایه اصلی جدید با **idf**های به‌روز شده روی دیسک نوشته می‌شود.

استراتژی سوم: تعیین حد آستانه رشد (Threshold-based Triggering)

به جای به‌روزرسانی بر اساس زمان، سیستم با استفاده از یک اسکریپت کنترلی، نرخ رشد کلکسیون را مانیتور می‌کند و تنها زمانی بازنویسی **idf** را آغاز می‌کند که درصد تغییرات متغیر N از یک حد مشخص (مثلاً 5 درصد) عبور کرده باشد. نمایه منطقی این استراتژی در قالب کدهای پایتون به شکل زیر پیاده‌سازی می‌شود (تمامی توضیحات ساختاری جهت سازگاری با سیستم‌های استاندارد توسعه، به زبان انگلیسی درج شده‌اند):

```
idf_trigger.py

def should_update_idf(current_N, last_updated_N, threshold=0.05):
    # Calculate the proportional growth rate of the corpus
    growth_rate = (current_N - last_updated_N) / last_updated_N

    # Trigger a heavy re-indexing process only if threshold is exceeded
    if growth_rate > threshold:
        return True
    return False

# Example usage for the news system
N_last_month = 10000000
N_today = 10010000 # Added 10,000 news articles

if should_update_idf(N_today, N_last_month):
    print("Initiating global IDF recalculation...")
else:
    print("Skipping IDF update. Using stale index.")
```

بخش 3) جمع‌بندی:

نکته: در علوم کامپیوتر و طراحی سیستم‌ها، همیشه یک مصالحه (**Trade-off**) بین «دقت 100%» و «سرعت پاسخ‌گویی» وجود دارد. یک موتور جستجو که مقادیر **idf** آن مربوط به یک هفته پیش است اما در 0.1 ثانیه پاسخ می‌دهد، بسیار کارآمدتر از سیستمی است که دقیق‌ترین مقادیر لحظه‌ای را دارد اما برای هر جستجو 5 ثانیه زمان می‌برد.

پاسخنامه مسئله 6.21: معماری حافظه محدود در پردازش بردارها و تکنیک‌های هرس کردن

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک محدودیت‌های فیزیکی معماری مدل فضای برداری در مقیاس کلان، آشنایی با تفاوت بازیابی دقیق (**Exact Retrieval**) و بازیابی تقریبی (**Approximate Retrieval**)، و تسلط بر الگوریتم‌های پیشرفته جهت کاهش بار حافظه و

پردازش است.

در پردازش استاندارد کلمه-به-کلمه (**TAAT** یا **Term-at-a-Time**)، سیستم برای محاسبه شباهت کسینوسی نیازمند ایجاد یک آرایه انباشتگر (**Accumulators**) است که برای هر سند موجود در پایگاه داده یک خانه حافظه در نظر می‌گیرد.

در صورت مسئله، 3 کلمه داریم که هر کدام دارای لیست پستینگی به طول 1,000,000 سند هستند. اگر سیستم بخواهد تمامی این اسناد را پردازش کند، باید میلیون‌ها انباشتگر اعشاری (**Float**) را در حافظه اصلی (**RAM**) مقداردهی و به‌روزرسانی کند.

در طراحی موتورهای جستجوی فشرده تحت محدودیت‌های شدید حافظه (مشابه معماری‌های پیچیده‌ای چون نمایه‌ساز بین‌سیاره‌ای)، و یا در محیط‌های عملیاتی اشتراکی مانند سرورهای ارزیاب خودکار (**Autograder**) که کدهای پروژه‌ی یک کلاس 100 نفره را به صورت همزمان پردازش می‌کنند، پر شدن ظرفیت **RAM** یک بحران جدی (**Out of Memory**) ایجاد می‌کند. برای عبور از این بحران، مهندسی سیستم از پارادایم «محاسبه دقیق برای همه» به «محاسبه دقیق برای محتمل‌ترین‌ها» تغییر مسیر می‌دهد.

بخش 2) حل گام‌به‌گام و تشریح الگوریتم‌های هرس (Pruning)

برای دور زدن این بحران محاسباتی، از تکنیک‌های هرس کردن استفاده می‌شود که در آن‌ها بخش بزرگی از اسناد کم‌ارزش، پیش از ورود به چرخه محاسبات سنگین حذف می‌شوند. دو استراتژی اصلی برای این کار وجود دارد:

الف) رویکرد پیش‌پردازش: لیست‌های قهرمان (**Champion Lists / Top Docs**)

این رویکرد بر یک اصل ساده استوار است: کاربری که جستجو می‌کند، تنها به دنبال K سند برتر (مثلاً 10 نتیجه اول) است و نیازی به رتبه‌بندی دقیق سند شماره 900,000 ندارد.

نحوه عملکرد:

1. در زمان نمایه‌سازی (Indexing Time**):** برای هر کلمه در واژه‌نامه، به جای آنکه تنها یک لیست پستینگ استاندارد بسازیم، 2 لیست می‌سازیم. لیست دوم «لیست قهرمان» نام دارد. در این لیست، تنها شناسه اسنادی ذخیره می‌شود که بالاترین وزن (**tf-idf**) را برای آن کلمه خاص دارند (مثلاً 50,000 سند برتر به جای 1,000,000 سند).

2. در زمان جستجو (Query Time**):** هنگامی که کاربر کوئری t_1, t_2, t_3 را وارد می‌کند، سیستم آرایه انباشتگر را تنها برای اسنادی تشکیل می‌دهد که در **لیست قهرمان** این کلمات حضور دارند.

3. نتیجه: اندازه آرایه انباشتگر از مقیاس میلیونی به مقیاس چند ده هزاری کاهش می‌یابد که به راحتی در کش پردازنده یا حافظه محدود جا می‌گیرد. (اگر در لیست قهرمان به K سند نرسیدیم، به عنوان راهکار پشتیبان به لیست‌های اصلی مراجعه می‌کنیم).

ب) رویکرد ارزیابی پویا: الگوریتم وند (**WAND - Weak AND Algorithm**)

الگوریتم **WAND** یکی از شاهکارهای معماری در موتورهای جستجوی مدرن است. این الگوریتم متعلق به خانواده پردازش سند-به-سند (**DAAT** یا **Document-at-a-Time**) است و به جای ذخیره امتیاز تمام اسناد، از یک استراتژی حدس و پرش (**Skip**) استفاده می‌کند.

نحوه عملکرد:

1. **محاسبه کران بالا (Upper Bound):** در زمان ساخت نمایه، برای هر کلمه یک مقدار **UB** محاسبه می‌شود که برابر است با بیشترین وزنی که آن کلمه در هر سندی می‌تواند داشته باشد.

2. **تعیین آستانه پویا (Dynamic Threshold):** در زمان جستجو، سیستم یک صف اولویت (**Min-Heap**) به اندازه K (تعداد نتایج دلخواه کاربر) در حافظه نگه می‌دارد. کمترین امتیاز موجود در این صف، به عنوان «خط آستانه» (**Threshold**) عمل می‌کند.

3. **هرس بی‌درنگ (On-the-fly Pruning):** سیستم نشانگرها را روی لیست پستینگ کلمات t_1, t_2, t_3 حرکت می‌دهد. برای سند جاری، ابتدا جمع کران بالای کلمات موجود در آن سند محاسبه می‌شود ($\sum UB$).

- اگر $\sum UB < \text{Threshold}$: این بدان معناست که حتی در خوش‌بینانه‌ترین حالت ممکن، این سند نمی‌تواند وارد صف نتایج برتر (**Heap**) ما شود. بنابراین، سیستم **بدون محاسبه دقیق ضرب داخلی**، این سند را به طور کامل هرس (**Skip**) کرده و به سند بعدی می‌پرد.

- اگر $\sum UB \geq \text{Threshold}$: تنها در این صورت، سیستم پردازش کامل برداری را برای این سند انجام می‌دهد.

بخش 3) پیاده‌سازی الگوریتم ارزیابی

برای درک منطق هرس کردن در رویکرد ارزیابی پویا، قطعه کد زیر ساختار فیلتر کردن اسناد را نشان می‌دهد.

```
wand_algorithm.py

def process_wand_algorithm(query_terms, inverted_index, k_results):
    min_heap_top_k = PriorityQueue(max_size=k_results)
    current_threshold = 0.0

    # Moving through aligned posting lists DAAT style
    while has_unprocessed_documents(query_terms, inverted_index):
        current_doc = get_next_candidate_doc()

        # Calculate theoretical maximum score using Upper Bounds
        max_possible_score = 0.0
        for term in query_terms:
            if doc_contains_term(current_doc, term):
                max_possible_score += inverted_index[term].upper_bound

        # Pruning mechanism: skip exact math if upper bound cannot beat threshold
        if max_possible_score < current_threshold:
            skip_document(current_doc)
            continue

        # Exact evaluation only for promising candidates
        exact_score = compute_exact_cosine_similarity(query_terms, current_doc)

        if exact_score > current_threshold:
            min_heap_top_k.push((exact_score, current_doc))
            current_threshold = min_heap_top_k.get_minimum_score()

    return min_heap_top_k.get_results()
```

بخش 4) جمع‌بندی:

نکته اول: در محیط‌های محدود از نظر منابع، پارادایم طراحی باید از رویکرد اگزاستیو (**Exhaustive Search** - جستجوی موشکافانه همه اعضا) فاصله بگیرد. تکنیک‌هایی مانند **Champion Lists** و **WAND** به سیستم اجازه می‌دهند تا بدون کاهش محسوس کیفیت نتایج صفحه اول، 90 درصد از محاسبات زائد را دور بریزند.

نکته دوم: هنگام مقایسه این روش‌ها، همواره به خاطر داشته باشید که «لیست قهرمان» یک هرس **ایستا (Static)** است که در زمان ایجاد ایندکس اتفاق می‌افتد؛ اما الگوریتم **WAND** یک هرس **پویا (Dynamic)** است که در زمان اجرای

کوئری و بر اساس حد آستانه (**Threshold**) لحظه‌ای تصمیم‌گیری می‌کند. تسلط بر این دوگانگی، وجه تمایز درک معماری‌های پیشرفته نسبت به مدل‌های ساده‌ی مقدماتی است.

پاسخنامه مسئله 6.22: مقابله با سئو کلاه سیاه (Spam) و خنثی‌سازی آن با مقیاس‌دهی زیرخطی

بخش 1) تحلیل اولیه و مبانی تئوری

هدف از این مسئله، شناخت تکنیک‌های مخرب در بهینه‌سازی موتورهای جستجو (مانند **Keyword Stuffing**)، درک آسیب‌پذیری مدل‌های خطی وزن‌دهی، و ارزیابی ریاضیاتی اثربخشی توابع مهارکننده‌ی لگاریتمی در کاهش اثر تقلب است. یکی از رایج‌ترین روش‌های تقلب در سیستم‌های جستجوی اولیه، تکنیکی به نام **بمباران کلمه کلیدی (Keyword Stuffing)** بود. در این روش که شاخه‌ای از «سئو کلاه سیاه» (**Black Hat SEO**) محسوب می‌شود، تولیدکننده محتوا کلمات کلیدی پر جستجو را به صورت پنهان (مثلاً با فونت هم‌رنگ پس‌زمینه، کدهای **CSS** یا خارج از کادر صفحه) صدها بار تکرار می‌کند تا موتور جستجو را فریب دهد.

در این مسئله، عملکرد دو رویکرد متفاوت ریاضی (فرکانس خام در برابر فرکانس لگاریتمی) در مواجهه با چنین حمله‌ای ارزیابی می‌شود تا برتری معماری‌های مدرن اثبات گردد.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

الف) چرا فرمول **tf** خالص در دام این حمله می‌افتد؟

در مدل فرکانس خام (**Pure TF**)، فرض بر این است که ارزش یک کلمه مستقیماً و به صورت خطی با تعداد تکرار آن در متن تناسب دارد.

فرض کنید پرس‌وجوی کاربر "بلیط هواپیما" است:

سند سالم (معتبر): مقاله‌ای واقعی که 3 بار از این کلمه به صورت طبیعی استفاده کرده است ($tf_{normal} = 3$).

سند متخلف (اسپم): صفحه‌ای نامرتب که کلمه را 300 بار در پس‌زمینه پنهان کرده است ($tf_{spam} = 300$).

محاسبه نسبت ارزش:

$$\text{Ratio} = \frac{tf_{\text{spam}}}{tf_{\text{normal}}} = \frac{300}{3} = 100$$

تحلیل خطا: موتور جستجویی که از فرمول خطی استفاده می‌کند، به صورت کورکورانه نتیجه می‌گیرد که سند اسپم **100 برابر** مرتبط‌تر از سند سالم است! در نتیجه، سند متخلف به راحتی رتبه یک نتایج را تصاحب می‌کند. این آسیب‌پذیری بنیادین به دلیل نبود هیچ‌گونه سقفی برای رشد امتیاز به وجود می‌آید.

ب) خنثی‌سازی اسپم با استفاده از مقیاس‌دهی زیرخطی (Sublinear TF Scaling)

برای رفع این نقص سیستماتیک، فرمول وزن‌دهی به صورت $w_{t,d} = 1 + \log_{10}(tf_{t,d})$ اصلاح می‌شود. اکنون وزن کلمه را در هر دو سند با این فرمول محاسبه می‌کنیم:

$$w_{\text{normal}} = 1 + \log_{10}(3)$$

با توجه به اینکه $\log_{10}(3) \approx 0.477$ است:

$$w_{\text{normal}} = 1 + 0.477 = 1.477$$

گام دوم: محاسبه وزن در سند اسپم

$$w_{\text{spam}} = 1 + \log_{10}(300)$$

برای محاسبه بدون ماشین حساب، لگاریتم 300 را می توان به صورت $\log_{10}(3 \times 100)$ یا $\log_{10}(3) + \log_{10}(100)$ بسط داد:

$$w_{\text{spam}} = 1 + \log_{10}(3) + \log_{10}(100) = 1 + 0.477 + 2 = 3.477$$

گام سوم: محاسبه نسبت ارزش جدید

اکنون برتری سند اسپم نسبت به سند سالم را در سیستم جدید (مقیاس دهی لگاریتمی) می سنجیم:

$$\text{Ratio} = \frac{w_{\text{spam}}}{w_{\text{normal}}} = \frac{3.477}{1.477} \approx 2.35$$

نحوه خنثی سازی تقلب:

با اعمال تابع لگاریتمی، برتری سند متخلف از **100 برابر** به تنها **2.35 برابر** سقوط کرد! تابع لگاریتمی اصل «بازده نزولی» (**Diminishing Returns**) را در ریاضیات شبیه سازی می کند؛ به این معنا که تکرارهای اولیه ی کلمه ارزش بالایی دارند، اما با افزایش بی رویه ی تکرارها، ارزش افزوده ی هر تکرار جدید به شدت به سمت 0 میل می کند. در سیستم های واقعی، این اختلاف ناچیز (2.35 برابر) به راحتی توسط فاکتورهای دیگر موتور جستجو (مانند جریمه ی طول سند در نرمال سازی کسینوسی، یا بررسی اعتبار دامنه) جبران شده و سند اسپم به طور کامل از رقابت خارج می شود.

بخش 3) جمع بندی:

نکته اول: فرمول های خطی در محیط های کنترل شده و آکادمیک به خوبی کار می کنند، اما در محیطی متخاصم (**Adversarial**) مانند وب، جایی که افراد انگیزه ی مالی برای دستکاری الگوریتم دارند، استفاده از توابع مهارکننده (**Dampening Functions**) مانند لگاریتم نه تنها یک انتخاب بهینه سازی، بلکه یک لایه حفاظتی و امنیتی الزامی است.

نکته دوم: در سوالات تحلیلی مرتبط با سئو کلاه سیاه و معماری **VSM**، همواره به یاد داشته باشید که ایندکسر (**Indexer**) قادر به درک رنگ متن یا «مخفی بودن» کلمات در **CSS** نیست؛ سیستم در این لایه تنها با اعداد و فرکانس ها

سر و کار دارد. بنابراین، خط دفاعی سیستم در برابر تکرارهای نامتعارف، باید مستقیماً در قالب توابع ریاضی وزن‌دهی (TF Scaling) تعبیه شود.

پاسخنامه مسئله 6.23: چالش اسناد ناهمگون و ضعف نرمال‌سازی حداکثر فرکانس

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، تحلیل رفتار آماری توزیع کلمات در پیکره‌های متنی با طول‌های نامتوازن، درک حساسیت الگوریتم **Maximum TF** به قانون زیپ (**Zipf's Law**)، و انتخاب معماری وزن‌دهی بهینه برای سامانه‌های اطلاعاتی ناهمگون (**Heterogeneous**) است.

در طراحی سیستم‌های بازیابی اطلاعات، پایگاه‌های داده ناهمگون (مانند ترکیب ویکی‌پدیا و کاتالوگ‌های کوتاه) یکی از بزرگترین چالش‌های معماری را ایجاد می‌کنند. در چنین محیط‌هایی، واریانس طول اسناد به شدت بالاست.

فرمول نرمال‌سازی حداکثر فرکانس به صورت زیر تعریف می‌شود:

$$w_{t,d} = a + (1 - a) \frac{tf_{t,d}}{tf_{\max}(d)}$$

در این فرمول، مقدار $tf_{\max}(d)$ نشان‌دهنده‌ی فرکانس پرتکرارترین کلمه‌ی موجود در سند d است. بر اساس **قانون زیپ (Zipf's Law)** در پردازش زبان طبیعی، پرتکرارترین کلمات در هر زبان (مانند "است"، "در"، "و") کلمات توقف (**Stop Words**) هستند که فرکانس آن‌ها با افزایش طول متن، به صورت تصاعدی و لگاریتمی رشد می‌کند. این مسئله، نقطه آغاز شکست این معماری در متون ناهمگون است.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

الف) تحلیل دلیل شکست معماری **Maximum TF**

دلیل اصلی این شکست، «وابستگی مخرج کسر به کلمات توقف در متون طولانی» است. برای اثبات این موضوع، سناریوی صورت مسئله را با یک مثال ریاضی مدل‌سازی می‌کنیم. فرض کنید کاربر کلمه‌ی "میکروکنترلر" را جستجو کرده است و پارامتر هموارسازی $a = 0.4$ است:

1. مقاله ویکی‌پدیا (سند طولانی):

مقاله بسیار جامع است و کلمه "میکروکنترلر" در آن 20 بار تکرار شده است ($tf = 20$). اما به دلیل طولانی بودن متن، کلمه‌ی ربط "و" در این مقاله 5000 بار تکرار شده است ($tf_{\max} = 5000$).

$$w_{\text{long}} = 0.4 + 0.6 \times \left(\frac{20}{5000} \right) = 0.4 + 0.0024 = 0.4024$$

2. کاتالوگ مهندسی (سند کوتاه):

متن کاتالوگ تنها چند خط است. کلمه "میکروکنترلر" 2 بار تکرار شده است ($tf = 2$). پرتکرارترین کلمه در این کاتالوگ تنها 10 بار تکرار شده است ($tf_{max} = 10$).

$$w_{short} = 0.4 + 0.6 \times \left(\frac{2}{10} \right) = 0.4 + 0.12 = 0.52$$

استنتاج تحلیلی خطا: همان‌طور که مشاهده می‌شود، مقاله ویکی‌پدیای غنی که 20 بار کلمه تخصصی را در خود داشت، امتیاز 0.4024 را کسب کرد، در حالی که کاتالوگ کوتاه با تنها 2 بار تکرار، امتیاز 0.52 را گرفت! کلمات بی‌ارزشیِ پرکاربرد در مقاله ویکی‌پدیا، مخرج کسر را به قدری بزرگ کردند که ارزش کلمات کلیدی کاملاً لِه (**Squash**) شد و امتیاز مقاله به کفِ حداقلی (a) سقوط کرد.

ب) نسخه‌های مناسب‌تر Variant TF-IDF برای دیتابیس ناهمگون

برای دیتابیس‌هایی که واریانس طول اسناد در آن‌ها بالاست، استفاده از هر فرمولی که مستقیماً به tf_{max} متکی باشد، اشتباه است. بهترین استراتژی‌های جایگزین عبارتند از:

1. رویکرد کلاسیک (Sublinear TF + Cosine Normalization):

همان‌طور که در شمای **Inc.Itc** بررسی می‌شود، استفاده از **مقیاس‌دهی لگاریتمی** ($1 + \log_{10}(tf)$) بهترین جایگزین برای **Maximum TF** است. در این روش، وزن کلمه کلیدی به هیچ وجه به فرکانس سایر کلمات (کلمات توقف) وابسته نیست. پس از محاسبه لگاریتمی، نرمال‌سازی کسینوسی (تقسیم بر طول فیزیکی بردار سند) اعمال می‌شود تا اثر طول متن به صورت منطقی‌تری کنترل شود.

2. رویکرد پیشرفته و استاندارد صنعتی (Pivoted Length Normalization / Okapi BM25):

بهترین و بی‌نقص‌ترین رویکرد برای متون کاملاً ناهمگون، استفاده از توابعی است که به جای تکیه بر «حداکثر فرکانس یک سند»، بر اساس «**میانگین طول کل اسناد در مجموعه**» (**avdl**) عمل می‌کنند. در الگوریتم‌هایی نظیر **BM25**، سیستم با مقایسه‌ی طول سند جاری نسبت به میانگین طول تمام اسناد، ضریب جریمه را به صورت محوری (**Pivoted**) و پویا تنظیم می‌کند. این کار مانع از جریمه‌ی ناعادلانه‌ی اسناد غنی مانند ویکی‌پدیا می‌شود.

بخش 3) جمع‌بندی:

نکته اول: فرمول **Maximum TF** برای پایگاه‌های داده‌ی همگن (مانند مجموعه‌ای از چکیده مقالات با طول تقریباً برابر) طراحی شده بود تا نوسانات جزئی را کنترل کند. در دنیای واقعی وب، جایی که یک توییت 280 کاراکتری در کنار یک پایان‌نامه 200 صفحه‌ای قرار می‌گیرد، این روش به دلیل اثر قانون زیف بر tf_{max} به شدت آسیب‌پذیر است.

نکته دوم: اگر سوالی پیرامون ضعف توابع **TF** مطرح شد، به یاد داشته باشید که کلید پاسخ تحلیلی، اشاره به **مخرج کسر** است. هر تابعی که مخرج آن به یک نقطه‌ی نوسانی متصل باشد (مثل پرتکرارترین کلمه‌ی یک متن خاص)، در برابر داده‌های پرت (**Outliers**) و اسناد بسیار طولانی دچار فروپاشی محاسباتی خواهد شد. برای چنین سیستم‌هایی، عبور از محاسبات محلی (**Local Maximum**) و حرکت به سمت پارامترهای سراسری مانند **avdl** یا استفاده از لگاریتم، راه‌حل قطعی است.

بخش 1) تحلیل اولیه و مبانی تئوری

هدف از این مسئله، توانایی تحلیل نیازمندی‌های یک سیستم اطلاعاتی در دنیای واقعی (مانند **HIS**)، درک رفتار توزیع کلمات در پرس‌وجوهای طولانی (**Long Queries**)، و انتخاب معماری وزن‌دهی بهینه بر اساس نمادگذاری استاندارد **SMART** برای مقابله با واریانس طول است.

در طراحی سیستم‌های بازیابی اطلاعات، انتخاب شمای وزن‌دهی (**Weighting Scheme**) هرگز تصادفی نیست، بلکه مستقیماً به ماهیت و ساختار داده‌های ورودی بستگی دارد. بر اساس سناریوی مطرح شده در سیستم اطلاعات بیمارستانی (**HIS**)، با 2 چالش و ویژگی ساختاری مهم مواجه هستیم:

1. **ویژگی پرس‌وجوها (Queries):** برخلاف موتورهای جستجوی وب که میانگین طول کوئری در آن‌ها 2 الی 3 کلمه است، پزشکان در این سیستم یک پاراگراف کامل (شرح حال و علائم) را وارد می‌کنند. این یعنی کوئری در اینجا ماهیتی شبیه به یک «سند کوتاه» (**Pseudo-document**) دارد و دارای کلمات تکراری و حروف اضافه‌ی فراوان است.

2. **ویژگی اسناد (Documents):** پرونده‌های پزشکی واریانس طول بسیار بالایی دارند؛ یک سند ممکن است صرفاً نتیجه‌ی 1 خطی آزمایش خون باشد و سند دیگر، تاریخچه‌ی 50 صفحه‌ای بستری‌های یک بیمار.

با توجه به این 2 ویژگی، شمای **Inc.Itc** بی‌نقص‌ترین و استانداردترین انتخاب مهندسی برای این سیستم است. در ادامه، دلایل رد طرح‌هایی مانند **ntc.ntc** و اثبات برتری **Inc.Itc** کالبدشکافی می‌شود.

بخش 2) حل گام‌به‌گام و اثبات معماری سیستم

الف) طراحی معماری برای اسناد (**Document Scheme: Inc**)

برای پردازش پرونده‌های پزشکی، شمای **Inc** را به دلایل زیر انتخاب می‌کنیم:

حرف (Logarithmic tf): به دلیل متفاوت بودن طول اسناد، استفاده از فرکانس خام (n) فاجعه‌بار است. اگر کلمه‌ی «دیابت» در یک پرونده‌ی 50 صفحه‌ای 20 بار تکرار شده باشد، ارزش آن به صورت خطی 20 برابر یک پرونده‌ی 1 صفحه‌ای نیست. مقیاس‌دهی زیرخطی ($1 + \log(tf)$) اثر تکرارهای ناشی از طولانی بودن پرونده را مهار می‌کند.

حرف (No idf): در سیستم‌های استاندارد، فرکانس معکوس سند (**IDF**) تنها 1 بار در کل محاسبات ضرب داخلی اعمال می‌شود. اعمال مجدد آن روی سند (مانند آنچه در طرح **ntc** دیده می‌شود)، باعث مربع شدن اثر **IDF** (IDF^2) شده و به کلمات نادر، وزنی غیرطبیعی و به شدت مخرب می‌دهد.

حرف (Cosine Normalization): با توجه به وجود اسناد 1 خطی و 50 صفحه‌ای در **HIS**، نرمال‌سازی کسینوسی (تقسیم بر طول فیزیکی بردار) کاملاً الزامی است تا شانس بازیابی پرونده‌های کوتاه در برابر پرونده‌های حجیم، عادلانه و برابر باقی بماند.

ب) طراحی معماری برای پرس‌وجو (**Query Scheme: Itc**)

طراحی شمای پرس‌وجو در این مسئله، نقطه‌ی عطف معماری است. به دلیل پاراگرافی بودن کوئری‌ها، شمای **Itc** را انتخاب می‌کنیم:

حرف (Logarithmic tf): l در کوئری‌های کوتاه، مقدار **tf** معمولاً 1 است و استفاده از **l** یا **n** تفاوتی ایجاد نمی‌کند. اما در این سناریو، پزشک یک پاراگراف می‌نویسد. در یک متن پاراگرافی، کلماتی مانند «بیمار»، «دارد» یا «درد» ممکن است بارها تکرار شوند. اگر از فرکانس خام (n) استفاده کنیم، سیستم فریب این تکرارهای نگارشی را می‌خورد. تابع لگاریتمی (l) این تکرارها در کوئری را به درستی هموار (**Smooth**) می‌کند.

حرف t (idf): قلب تپنده‌ی این معماری است. پاراگراف نوشته شده توسط پزشک، پر از کلمات عمومی و افعال رایج پزشکی است. متغیر t (یعنی $\log(N/df)$) به عنوان یک فیلتر تشخیصی عمل می‌کند؛ کلمات رایج را به شدت جریمه کرده و در عوض، وزن علائم نادر و کلیدی (مانند "هموپتزی" یا "برادی‌کاردی") را در بردار کوئری به حداکثر می‌رساند تا سیستم رتبه‌بندی دقیقاً روی همان علائم قفل شود.

حرف c (Cosine Normalization): از آنجا که کوئری خود به یک بردار طولانی تبدیل شده است، نرمال‌سازی کسینوسی آن باعث می‌شود مقیاس محاسبات در ضرب داخلی (**Dot Product**) استاندارد باقی بماند و مقادیر خروجی همواره بین 0 و 1 باشند.

ج) چرا ترکیب ntc.ntc مردود است؟

1. استفاده از **n** (فرکانس خام) برای هر دو طرف، سیستم را به شدت در برابر پدیده‌ی «تکرار کلمات» در پرونده‌های طولانی و پاراگراف‌های توصیفی پزشکان آسیب‌پذیر می‌کند.

2. استفاده از **t** برای سند، همان‌طور که پیش‌تر اثبات شد، منجر به اعمال مضاعف **IDF** و ناپایداری ریاضی فرمول وزن‌دهی در مدل فضای برداری (**VSM**) می‌گردد.

بخش 3) جمع‌بندی:

نکته اول: همیشه به یاد داشته باشید که در سیستم‌های جستجوی استاندارد (مانند گوگل)، کوئری کاربر معمولاً به قدری کوتاه است که بردار آن کاملاً تُنک (**Sparse**) و مقادیر **tf** آن برابر با 1 است. در آن حالت‌های کلاسیک، حتی استفاده از شمای **Inc.ntc** (با فرکانس خام برای کوئری) نیز منطقی است. اما **قانون طلایی طراحی** این است: هرگاه پرس‌وجوی کاربر از حالت «چند کلمه کلیدی» خارج شد و فرمت «زبان طبیعی و پاراگرافی» (**Natural Language / Long-form**) به خود گرفت، شما به عنوان طراح موظف هستید با بردار پرس‌وجو دقیقاً مشابه با یک «سند» رفتار کنید و تابع مهارکننده‌ی تکرار (لگاریتمی یا **l**) را روی آن اعمال نمایید.

نکته دوم: در پاسخگویی به مسائل طراحی معماری **SMART**، همیشه تحلیل خود را به 3 بخش «تکرار کلمات (**TF**)»، «ارزش تفکیک‌کنندگی کلمات (**IDF**)» و «طول متن (**Norm**)» تفکیک کنید و تصمیم خود برای هر بخش را با داده‌های صورت مسئله پیوند دهید.

پاسخنامه مسئله 6.25: تنظیم پارامتر نرمال‌سازی محوری و تحلیل نموداری

(Tuning Pivot Parameter and Graphical Analysis)

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک عمیق مکانیزم جریمه‌ی طول سند در فضای برداری، تحلیل ریاضیاتی توابع خطی در نرمال‌سازی محوری (**Pivoted Normalization**)، و توانایی عیب‌یابی (**Troubleshooting**) الگوریتم‌های رتبه‌بندی بر اساس خروجی‌های آزمایش

است.

نرمال‌سازی محوری طول سند که توسط آمیت سینگال (**Amit Singhal**) معرفی شد، برای رفع یکی از بزرگترین نقص‌های نرمال‌سازی کسینوسی طراحی شده است. نرمال‌سازی کسینوسی اسناد طولانی را بیش از حد جریمه می‌کند. با این حال، اگر این مکانیزم به درستی تنظیم نشود، سیستم از آن سوی بام افتاده و اسناد کوتاه را بیش از حد پاداش می‌دهد (پدیده‌ای که در صورت مسئله رخ داده است).

فرمول استاندارد ضریب نرمال‌ساز در این روش به شکل زیر تعریف می‌شود (که در مخرج کسر امتیازدهی قرار می‌گیرد):

$$\text{Normalizer} = (1 - s) + s \frac{dl}{p}$$

در این معادله:

dl: طول فیزیکی سند جاری (**Document Length**).

p: نقطه‌ی محور (**Pivot**)، که معمولاً برابر با میانگین طول اسناد کلکسیون (*avdl*) در نظر گرفته می‌شود.

s: پارامتر شیب (**Slope**) که شدت اعمال جریمه‌ی طول را کنترل می‌کند ($0 \leq s \leq 1$).

از آنجا که این ضریب در مخرج کسر محاسبه‌ی امتیاز (**Score**) قرار دارد، هرچه مقدار **Normalizer** بزرگتر شود، امتیاز نهایی سند کمتر شده و رتبه‌ی آن افت می‌کند.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

اگر سیستم در مرحله‌ی آزمایش، اسناد کوتاه را بیش از حد رتبه‌بندی می‌کند (**Over-retrieving short docs**)، به این معناست که امتیاز اسناد کوتاه به صورت کاذب بالاست. از منظر ریاضی، این یعنی مخرج کسر (ضریب نرمال‌ساز) برای اسناد کوتاه بیش از حد کوچک است و باید آن را افزایش دهیم.

الف) تنظیم پارامتر شیب (**Slope - s**)

برای یک سند کوتاه، طول آن از نقطه‌ی محور کمتر است ($dl < p$). در نتیجه، کسر $\frac{dl}{p}$ مقداری کوچکتر از 1 خواهد بود (مثلاً 0.5).

معادله‌ی نرمال‌ساز را می‌توان به این شکل بازنویسی کرد:

$$\text{Normalizer} = 1 - s \left(1 - \frac{dl}{p} \right)$$

از آنجا که عبارت درون پرانتز ($1 - \frac{dl}{p}$) برای اسناد کوتاه همواره یک عدد مثبت است، برای اینکه کل مقدار **Normalizer** بزرگتر شود (تا امتیاز سند کاهش یابد و مشکل حل شود)، ما باید مقدار عبارتی که از 1 کم می‌شود را کاهش دهیم. این کار تنها با کاهش دادن مقدار شیب (**s**) امکان‌پذیر است.

استنتاج مهندسی: شیب (**s**) باید کاهش یابد (به سمت 0 میل کند) تا تابع نرمال‌سازی، تخت‌تر (**Flatter**) شود. این کار پاداش ناعادلانه‌ی اسناد کوتاه و جریمه‌ی شدید اسناد طولانی را متعادل می‌کند.

ب) تنظیم نقطه‌ی محور (Pivot - p)

اگر بخواهیم مشکل را از طریق نقطه‌ی محور حل کنیم، مجدداً باید به کسر $\frac{dl}{p}$ نگاه کنیم. ما نیاز داریم مقدار نهایی معادله برای اسناد کوتاه افزایش یابد.

اگر مقدار محور (p) را **کاهش** دهیم (آن را به سمت چپ نمودار منتقل کنیم)، مخرج کسر $\frac{dl}{p}$ کوچکتر شده و در نتیجه کل کسر بزرگتر می‌شود. بزرگتر شدن این بخش، باعث افزایش مقدار **Normalizer** و در نتیجه کاهش امتیاز نهایی اسناد کوتاه می‌گردد.

استنتاج مهندسی: نقطه‌ی محور (p) نیز در صورت نیاز باید **کاهش** یابد (به عددی کوچکتر از میانگین طول اسناد تنظیم شود).

بخش 3) تحلیل با رسم یک نمودار

برای تثبیت این مفهوم، نمودار دو بعدی زیر را در نظر بگیرید:

محور افقی (X-axis): طول سند (dl).

محور عمودی (Y-axis): مقدار ضریب نرمال‌ساز (**Normalizer**).

نقطه‌ی محور (Pivot Point): نقطه‌ای روی نمودار که در آن $dl = p$ و $Normalizer = 1$ است. تمامی خطوط با شیب‌های مختلف، از این نقطه عبور می‌کنند (مانند یک لولا).

سناریوی فعلی (خطای سیستم): خط نمودار دارای شیب تند است. این یعنی برای اسناد کوتاه (سمت چپ نقطه‌ی لولا)، خط به شدت به سمت پایین افت می‌کند (نرمال‌ساز کوچک امتیاز بزرگ کاذب).

راه حل (چرخش لولا): شما باید خط را حول نقطه‌ی لولا به صورت **پادساعت‌گرد (Counter-Clockwise)** بچرخانید تا شیب آن ملایم‌تر (**Flatter**) شود. با این چرخش خواهید دید که:

1. در سمت چپ محور (اسناد کوتاه)، خط به سمت بالا می‌آید (افزایش جریمه).

2. در سمت راست محور (اسناد طولانی)، خط به سمت پایین می‌رود (کاهش جریمه).

بخش 4) جمع‌بندی:

نکته اول: در الگوریتم‌های پیشرفته‌ی بازیابی مانند **BM25**، پارامتر معروف b دقیقاً همان نقش پارامتر شیب (s) را در نرمال‌سازی محوری ایفا می‌کند. اگر در یک سیستم مبتنی بر **BM25** اسناد کوتاه بیش از حد بالا آمدند، بلافاصله باید مقدار پارامتر b را کاهش دهید.

نکته دوم: در پاسخ به سوالات تحلیلی تیونینگ (**Tuning**)، به جای حفظ کردن جهت‌ها، همیشه از فرمول $1 - s(1 - x)$ استفاده کنید. اگر $x < 1$ (سند کوتاه) است، برای بزرگتر شدن نتیجه، باید ضریب منفی (s) کوچک شود. این تحلیل جبری ساده، شما را از هرگونه خطای مفهومی در جلسه‌ی امتحان مصون می‌دارد.

پاسخنامه مسئله 6.26: مدیریت مترادف‌ها در فضای برداری و مشکل بردارهای متعامد

(Synonyms and Null Vectors in VSM)

بخش 1) تحلیل اولیه و مبانی تئوری

هدف این مسئله، درک یکی از بزرگ‌ترین محدودیت‌های ساختاری در مدل فضای برداری (VSM) معروف به «عدم تطابق واژگان» (Vocabulary Mismatch)، شناخت مفهوم تعامد ابعاد (Orthogonality)، و فراگیری معماری‌های جبران‌کننده در دو سطح نمایه‌سازی و پردازش کوئری برای رسیدن به جستجوی معنایی (Semantic Search) است. در مدل پایه‌ای فضای برداری (VSM)، یک فرض ساده‌انگارانه و به‌شدت محدودکننده وجود دارد: «تمامی کلمات واژه‌نامه کاملاً از یکدیگر مستقل هستند».

از منظر هندسی، هر کلمه یکتای موجود در پایگاه داده، یک محور (Axis) مستقل در یک فضای V -بعدی تشکیل می‌دهد و تمامی این محورها بر یکدیگر عمود (Orthogonal) هستند. بنابراین، اگر زاویه بین محور کلمه "Car" و محور کلمه "Automobile" دقیقاً 90 درجه باشد، شباهت کسینوسی و ضرب داخلی بردارهای آن‌ها همواره 0 خواهد شد:

$$\vec{q} \cdot \vec{d} = 0$$

در این حالت، موتور جستجو کاملاً نسبت به روابط معنایی و مترادف بودن کلمات نابینا است. برای رفع این مشکل، مهندسان سیستم‌های بازیابی باید فرض استقلال کلمات را شکسته و معماری سیستم را تغییر دهند.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

برای غلبه بر این بحران، 2 رویکرد متفاوت در دو فاز مختلف از چرخه حیات موتور جستجو (Search Engine Pipeline) قابل پیاده‌سازی است:

راهکار اول: در زمان پردازش کوئری (Query Time) - تکنیک بسط پرس‌وجو

این تکنیک که به آن Query Expansion گفته می‌شود، در لایه‌ی کاربری و دقیقاً پس از دریافت کوئری از کاربر و پیش از ارسال آن به هسته‌ی VSM اعمال می‌شود.

نحوه عملکرد: سیستم از یک ساختار داده‌ی جانبی مانند اصطلاح‌نامه (Thesaurus - مانند WordNet) یا ماتریس‌های هم‌رخدادی کلمات استفاده می‌کند. زمانی که کاربر کوئری $\vec{q} = [\text{Car}]$ را وارد می‌کند، ماژول بسط پرس‌وجو، مترادف‌های آن را پیدا کرده و بردار کوئری را به شکل زیر بازنویسی (Rewrite) و گسترده می‌کند:

$$\vec{q}_{\text{expanded}} = [\text{Car}, \text{Automobile}, \text{Vehicle}]$$

نتیجه در VSM: اکنون بردار پرس‌وجو دارای ابعاد غیرصفر متعددی است. هنگامی که این بردار بسط‌یافته با بردار سند (که حاوی بُعد "Automobile" است) ضرب داخلی می‌شود، حاصل ضرب آن بُعد غیرصفر شده و سند با موفقیت بازیابی می‌گردد.

الگوریتم جایگزین در این فاز: استفاده از بازخورد شبه-مرتبط (Pseudo-Relevance Feedback) مانند الگوریتم Rocchio، که در آن سیستم نتایج اولیه را پردازش کرده و کلمات کلیدی آن‌ها را به صورت خودکار به کوئری کاربر اضافه می‌کند.

راهکار دوم: در زمان ساخت ایندکس (Index Time) - تکنیک نمایه‌سازی معنایی نهان (LSI)

این تکنیک بسیار بنیادی‌تر است و معماری هسته‌ی ایندکس (Inverted Index) را در زمان نمایه‌سازی تغییر می‌دهد تا به جای «کلمات»، «مفاهیم» (Concepts) را ذخیره کند. معروف‌ترین روش در این رویکرد، Latent Semantic Indexing (LSI) است.

نحوه عملکرد: در این روش، ماتریس عظیم ترم-سند (**Term-Document Matrix**) با استفاده از تکنیک‌های جبر خطی به نام **تجزیه مقادیر منفرد (Singular Value Decomposition - SVD)**، به یک فضای با ابعاد بسیار کمتر (**Low-dimensional Space**) تقلیل می‌یابد.

شکستن تعامد: در این فضای تقلیل‌یافته‌ی جدید (مثلاً با 300 بُعد به جای 1,000,000 بُعد)، کلمات "Car" و "Automobile" دیگر 2 محور عمود بر هم نیستند! از آنجا که این 2 کلمه همواره در متون مشابهی در کل پایگاه داده تکرار می‌شوند، الگوریتم **SVD** آن‌ها را روی یک **محور مفهومی مشترک (Concept Axis)** ادغام (**Map**) می‌کند.

نتیجه در VSM: هنگامی که کوئری "Car" وارد سیستم می‌شود، بردار کوئری به فضای مفاهیم منتقل می‌شود. در آنجا، فاصله‌ی هندسی بین مفهوم "Car" و سند حاوی "Automobile" بسیار کم است و ضرب داخلی آن‌ها به شدت بالا خواهد بود.

بخش 3) جمع‌بندی:

• تحلیل هزینه و فایده (Trade-offs):

- تکنیک **بسط پرس‌وجو (Query Time)** پیاده‌سازی ساده‌تری دارد و نیازی به تغییر دیتابیس عظیم اسناد نیست، اما باعث افزایش طول کوئری شده و در نتیجه سرعت پاسخ‌دهی در لحظه (**Latency**) را کند می‌کند.
- تکنیک **LSI (Index Time)** در زمان ایجاد نمایه، به توان پردازشی و محاسبات ریاضیاتی (**Matrix Decomposition**) بسیار سنگینی نیاز دارد، اما در زمان اجرای کوئری به شدت سریع است و قدرت تطابق معنایی شگفت‌انگیزی دارد.

نکته: در طراحی معماری‌های نوین، به خاطر داشته باشید که پیشرفت‌های اخیر در شبکه‌های عصبی عمیق (مانند **Word2Vec** و **BERT**) در واقع تکامل‌یافته‌ی همان ایده‌ی **LSI** هستند. در این مدل‌های مدرن (**Dense Retrieval**)، ما به طور کامل از بردارهای اسپارس متعامد عبور کرده‌ایم و کلمات را در فضایی متراکم (**Dense Embeddings**) نشان می‌دهیم که در آن، هر کلمه‌ی مترادف، برداری با زاویه بسیار نزدیک به کلمه‌ی هدف دارد.

پاسخنامه مسئله 6.27: پیاده‌سازی معماری فضای برداری و شمای وزن‌دهی **Inc.Itc** در پایتون

بخش 1) تحلیل اولیه و معماری کد

هدف از این مسئله، ترجمه‌ی مفاهیم انتزاعی و فرمول‌های ریاضیاتی به کدهای عملیاتی (**Production-level Code**)، درک مدیریت حافظه از طریق آرایه‌های انباشتگر (**Accumulators**) در پردازش **Term-at-a-Time (TAAT)**، و ساختاردهی به کدهایی است که قابلیت اجرای بهینه در سیستم‌های ارزیابی خودکار (**Autograders**) را داشته باشند.

برای پیاده‌سازی دقیق شمای **Inc.Itc**، باید فرمول‌های ریاضی را به 2 بخش کاملاً مجزا (یکی برای پردازش سند و دیگری برای کوئری) تقسیم کنیم. با توجه به ساختار استاندارد موتورهای جستجو، محاسبه‌ی طول بردار اسناد (برای نرمال‌سازی کسینوسی **c** در بخش **Inc**) نباید در زمان اجرای کوئری انجام شود، بلکه باید به عنوان یک پیش‌پردازش (**Pre-computation**) در زمان راه‌اندازی کلاس صورت گیرد تا سرعت سیستم حفظ شود.

در بخش اجرای جستجو، ما از ساختار دیکشنری به عنوان انباشتگر اسپارس (**Sparse Accumulator**) استفاده می‌کنیم تا حافظه تنها برای اسنادی اشغال شود که حداقل 1 کلمه مشترک با کوئری دارند.

بخش 2) پیاده‌سازی:

```

import math
from collections import defaultdict, Counter

class VectorRetrieval:
    def __init__(self, inverted_index, total_docs):
        # inverted_index format: {term: {doc_id: term_frequency, ...}, ...}
        self.inverted_index = inverted_index
        self.N = total_docs
        self.doc_lengths = self._precompute_doc_lengths()

    def _precompute_doc_lengths(self):
        # Calculates L2 norm (Euclidean length) for each document based on 'lnc' scheme
        # l: log tf, n: no idf, c: cosine normalization applied later
        lengths = defaultdict(float)

        for term, postings in self.inverted_index.items():
            for doc_id, tf in postings.items():
                if tf > 0:
                    w_td = 1.0 + math.log10(tf)
                    lengths[doc_id] += (w_td ** 2)

        for doc_id in lengths:
            lengths[doc_id] = math.sqrt(lengths[doc_id])

        return lengths

    def search(self, query_terms):
        query_tfs = Counter(query_terms)

        # Accumulator structure for TAAT processing (Memory efficient)
        accumulators = defaultdict(float)

        query_weights = {}
        query_length_sq = 0.0

        # Phase 1: Process Query using 'ltc' scheme
        for term, q_tf in query_tfs.items():
            if term not in self.inverted_index:
                continue

            df = len(self.inverted_index[term])
            if df == 0:
                continue

            # l: log tf for query
            w_tq_tf = 1.0 + math.log10(q_tf)

            # t: idf for query
            idf = math.log10(self.N / float(df))

            # Unnormalized query weight
            w_tq = w_tq_tf * idf
            query_weights[term] = w_tq

            # Add to query vector length squared
            query_length_sq += (w_tq ** 2)

        # Compute final query vector length
        query_length = math.sqrt(query_length_sq) if query_length_sq > 0 else 1.0

        # Phase 2: Term-at-a-Time (TAAT) Scoring using Accumulators
        for term, w_tq in query_weights.items():
            # c: Cosine normalization for query
            w_tq_norm = w_tq / query_length

            postings = self.inverted_index[term]

            for doc_id, d_tf in postings.items():
                # l: log tf for document
                w_td = 1.0 + math.log10(d_tf)

                # c: Cosine normalization for document
                w_td_norm = w_td / self.doc_lengths[doc_id]

                # Accumulate the dot product of normalized weights
                accumulators[doc_id] += (w_tq_norm * w_td_norm)

        # Phase 3: Sort and retrieve
        ranked_results = sorted(accumulators.items(), key=lambda x: x[1], reverse=True)
        return ranked_results

```

مدیریت بهینه حافظه (Accumulator Strategy): در متد **search**، متغیر **accumulators** نقش قلب تپنده سیستم را ایفا می‌کند. به جای تخصیص یک آرایه استاتیک به طول کل اسناد (که در دیتابیس‌های میلیونی باعث پر شدن سریع **RAM** می‌شود)، از یک دیکشنری پویا (**defaultdict**) استفاده شده است. این دیکشنری تنها در صورتی برای یک شناسه (**doc_id**) فضا اشغال می‌کند که پردازشگر در حلقه‌ی تو در تو به آن سند برسد؛ این دقیقاً همان تکنیکی است که پیچیدگی فضایی را از $O(N)$ به $O(\text{Documents containing query terms})$ تقلیل می‌دهد.

تفکیک منطق پیش‌پردازش: تابع **precompute_doc_lengths** نشان‌دهنده‌ی درک صحیح از چرخه‌ی حیات موتور جستجو است. از آنجا که شمای **Inc** نیازی به داده‌های کوئری ندارد، طول تمام اسناد می‌تواند در زمان نمایه‌سازی (**Indexing Time**) محاسبه و کش (**Cache**) شود. این کار از تکرار هزاران عملیات جذر و لگاریتم سنگین در زمان اجرای کوئری (**Query Time**) جلوگیری کرده و سیستم را قادر می‌سازد تا در سناریوهای پُرتراфик پاسخگویی سریعی داشته باشد.

نکته مهم: هنگام تحلیل کدهای **VSM**، همیشه مسیر اجرای نرمال‌سازی را ردیابی کنید. متغیر **w_td_norm** و **w_tq_norm** پیش از ضرب شدن در یکدیگر بر طول بردارهای خود تقسیم شده‌اند. این یعنی ما به جای محاسبه‌ی خام ضرب داخلی و تقسیم نهایی آن، عملوندها را در همان ابتدا نرمال‌سازی کرده‌ایم که از نظر ریاضی یکسان است:

$$\frac{A \cdot B}{|A||B|} = \frac{A}{|A|} \cdot \frac{B}{|B|}$$

اما از نظر برنامه‌نویسی و مدیریت انباشتگرها بسیار تمیزتر و ایمن‌تر است.

پاسخنامه مسئله 6.28: طراحی سیستم وزن‌دهی پویا با یادگیری ماشین برای تجارت الکترونیک

(Machine Learning-Based Dynamic Weighting)

بخش 1) تحلیل اولیه و مدل‌سازی ریاضی

هدف از این مسئله، کالبدشکافی مسیر تبدیل یک مسئله‌ی رتبه‌بندی ساده به یک مدل یادگیری ماشین (**Learning to Rank**)، فرمول‌بندی ریاضی تابع هزینه (**Loss Function**)، و پیاده‌سازی گام‌به‌گام الگوریتم «گرادیان کاهشی» برای بهینه‌سازی خودکار پارامترهای موتور جستجو است.

هدف ما بهینه‌سازی پارامتر $g \in [0, 1]$ است که اهمیت نسبی «عنوان محصول» در برابر «توضیحات» را تعیین می‌کند. ما مجموعه‌ای از داده‌های آموزشی T داریم که شامل سه‌تایی‌های (q_j, d_j, r_j) است؛ جایی که r_j امتیاز ارتباط ایده‌آل (**Ground Truth**) است که از تحلیل کلیک‌های کاربران و قضاوت‌های کارشناسان انسانی به دست آمده است.

امتیاز پیش‌بینی شده توسط سیستم (score_j) برای هر نمونه‌ی آموزشی به صورت زیر است:

$$\text{score}_j(g) = g \cdot s_{\text{title},j} + (1 - g) \cdot s_{\text{body},j}$$

بخش 2) طراحی الگوریتم یادگیری وزن‌ها (Learning Algorithm)

برای بهینه‌سازی، از الگوریتم **گرادیان کاهشی (Gradient Descent)** استفاده می‌کنیم که به صورت تکرارشونده مقدار g را به‌روزرسانی می‌کند تا خطا را به حداقل برساند.

گام اول: تعریف تابع هزینه (Cost Function)

ما از **مجموع مربعات خطا (SSE)** برای اندازه‌گیری فاصله بین امتیاز پیش‌بینی شده و امتیاز ایده‌آل استفاده می‌کنیم:

$$J(g) = \frac{1}{2} \sum_{j \in T} (\text{score}_j(g) - r_j)^2$$

(ضریب 0.5 صرفاً برای ساده‌تر شدن مشتق‌گیری در گام بعدی اضافه شده است).

گام دوم: محاسبه مشتق (Gradient Calculation)

برای پیدا کردن جهتی که باید در آن g را تغییر دهیم تا خطا کم شود، مشتق تابع هزینه نسبت به g را محاسبه می‌کنیم:

$$\frac{\partial J}{\partial g} = \sum_{j \in T} (\text{score}_j(g) - r_j) \frac{\partial}{\partial g} (\text{score}_j(g))$$

از آنجا که مشتق $\text{score}_j(g)$ نسبت به g برابر با $(s_{\text{title},j} - s_{\text{body},j})$ است، داریم:

$$\frac{\partial J}{\partial g} = \sum_{j \in T} (\text{score}_j(g) - r_j) (s_{\text{title},j} - s_{\text{body},j})$$

گام سوم: به‌روزرسانی پارامتر (Parameter Update)

در هر تکرار، وزن g را در جهت مخالف گرادیان به‌روزرسانی می‌کنیم تا به نقطه‌ی بهینه نزدیک شویم:

$$g_{\text{new}} = g_{\text{old}} - \alpha \frac{\partial J}{\partial g}$$

که در آن α (نرخ یادگیری یا **Learning Rate**) یک مقدار بسیار کوچک (مثلاً 0.01) برای کنترل سرعت تغییرات است.

بخش 3) مراحل اجرایی الگوریتم (الگوریتم عملیاتی)

1. Initialize: مقدار g را به صورت تصادفی (مثلاً 0.5) مقداردهی اولیه کنید.

2. Iterate: برای تعداد تکرار مشخص (یا تا زمانی که خطا تغییر چندانی نکند):

الف) امتیاز تمام اسناد موجود در داده‌های آموزشی T را با مقدار فعلی g محاسبه کنید.

ب) گرادیان (مشتق) را با استفاده از فرمول گام دوم محاسبه کنید.

ج) مقدار g را طبق فرمول گام سوم به‌روزرسانی کنید.

د) Constraint: اطمینان حاصل کنید که g همیشه در بازه $[0, 1]$ باقی بماند (اگر $g < 0$ شد آن را 0 و اگر $g > 1$ شد آن را 1 قرار دهید).

3. Result: مقدار نهایی g ، وزن بهینه‌ای است که سیستم را با رفتارهای واقعی کاربران هم‌سو می‌کند.

بخش 4) جمع‌بندی و تکنیک‌های استنتاج

نکته کلیدی: اگر متوجه شدید در لاگ کلیک‌ها، کاربرها برای کوئری‌های خاصی همیشه روی توضیحاتی کلیک می‌کنند که در منطقه **Body** است، الگوریتم به صورت خودکار مقدار g را (یعنی وزن **Title**) کاهش می‌دهد.

گسترش به مقیاس بزرگ: در یک فروشگاه اینترنتی واقعی، این مدل خطی ($score_j = g \cdot s_{title} + (1 - g) \cdot s_{body}$) تنها پایه است. برای عملکرد حرفه‌ای، این مدل را به یک **مدل غیرخطی (مانند LambdaMART)** ارتقا دهید که می‌تواند ده‌ها فاکتور مختلف (از جمله قیمت محصول، تاریخ انقضا و موجودی) را به عنوان ویژگی (**Feature**) وارد محاسبات کند تا رتبه‌بندی به حداکثر دقت برسد.

پاسخنامه مسئله 6.29: نرمال‌سازی محوری طول سند در پایگاه‌های داده ناهمگون (Pivoted Document Length Normalization in Heterogeneous Databases)

بخش 1) تحلیل اولیه و مبانی تئوری

هدف از این مسئله، درک پدیده «جریمه‌ی بیش از حد» (**Over-penalization**) در متون طولانی هنگام استفاده از نرمال‌سازی کسینوسی، و فراگیری نحوه‌ی پیاده‌سازی و تنظیم الگوریتم نرمال‌سازی محوری برای رفع اعوجاج در سیستم‌های ناهمگون است.

در پایگاه‌های داده ناهمگون (**Heterogeneous Corpora**) که شامل ترکیبی از اسناد بسیار کوتاه (مانند توییت‌ها) و اسناد بسیار طولانی (مانند گزارش‌های 100 صفحه‌ای) هستند، استفاده از **نرمال‌سازی کسینوسی خالص (Pure Cosine Normalization)** به یک بحران ساختاری منجر می‌شود.

اسناد طولانی به طور طبیعی دایره‌واژگان وسیع‌تری دارند و کلمات در آن‌ها بیشتر تکرار می‌شوند؛ در نتیجه، بردار این اسناد در مدل فضای برداری (**VSM**) دارای ابعاد غیرصفر بیشتر و طول هندسی ($|\vec{d}|$) بسیار بزرگی خواهد بود. از آنجا که در نرمال‌سازی کسینوسی، امتیاز تطابق بر این طول هندسی تقسیم می‌شود، مخرج کسر برای اسناد طولانی به شدت بزرگ شده و امتیاز آن‌ها را به ناحق سرکوب می‌کند (**Squashing**). در مقابل، اسناد کوتاه به دلیل داشتن مخرج کوچک، به صورت کاذب در صدر نتایج قرار می‌گیرند. تکنیک **نرمال‌سازی محوری (Pivoted Normalization)** برای جبران این خطای سیستماتیک معرفی شده است.

بخش 2) حل گام‌به‌گام و تشریح خواسته‌های مسئله

الف) نحوه‌ی پیاده‌سازی تکنیک Pivoted Normalization

برای پیاده‌سازی این تکنیک، ما باید مخرج کسر در فرمول شباهت کسینوسی را از طول هندسی خام به یک «ضریب نرمال‌ساز تعدیل‌شده» (**Adjusted Normalizer**) تغییر دهیم.

فرمول خطی و استاندارد این نرمال‌ساز به شکل زیر تعریف می‌شود:

$$\text{Normalizer} = (1 - s) + s \frac{|\vec{d}|}{p}$$

در این فرمول:

$|\vec{d}|$: طول هندسی (نرم اقلیدسی) بردار سند جاری.

p: نقطه‌ی محور (Pivot).

s: پارامتر شیب (Slope) که عددی بین 0 و 1 است و شدت مهار طول را کنترل می‌کند.

برای محاسبه‌ی امتیاز نهایی هر سند، سیستم به جای فرمول کلاسیک $\frac{\vec{q} \cdot \vec{d}}{|\vec{d}|}$ ، از فرمول $\frac{\vec{q} \cdot \vec{d}}{\text{Normalizer}}$ استفاده خواهد کرد.

ب) پارامتر Pivot بر چه اساسی تنظیم می‌شود؟

نقطه‌ی محور (p) در این الگوریتم، نماینده‌ی «متوسط طول اسناد در کل پایگاه داده» (Average Document Length - avdl) است.

سیستم در زمان ساخت نمایه (Index Time)، طول بردارهای تمامی اسناد را محاسبه کرده و میانگین آن‌ها را به عنوان نقطه‌ی p در نظر می‌گیرد.

منطق ریاضی: اگر طول سند جاری دقیقاً برابر با میانگین کلکسیون باشد ($|\vec{d}| = p$)، کسر $\frac{|\vec{d}|}{p}$ برابر با 1 خواهد شد. در این حالت، معادله به صورت $s(1) + (1 - s) = 1$ درمی‌آید و سند نه جریمه می‌شود و نه پاداش می‌گیرد. این دقیقاً همان نقطه‌ی تعادل یا لولا (Pivot Point) در نمودار است.

ج) نمودار نرمال‌سازی چگونه با این روش اصلاح می‌گردد؟

برای درک اصلاح نمودار، یک فضای دو بعدی را در نظر بگیرید که محور افقی آن «طول سند» و محور عمودی آن «احتمال بازیابی (ضریب نرمال‌ساز)» است.

1. نمودار کسینوسی خام: یک خط (یا منحنی) با شیب بسیار تند است که با افزایش طول سند، به شدت بالا می‌رود (یعنی مخرج کسر بزرگ شده و احتمال بازیابی به شدت افت می‌کند).

2. اعمال چرخش (Pivoting): الگوریتم محوری، این خط تند را حول نقطه‌ی p (میانگین طول) می‌گیرد و آن را در جهت **تخت‌تر شدن (Flatter)** می‌چرخاند (میزان این چرخش توسط پارامتر s کنترل می‌شود).

نتیجه‌ی اصلاح نمودار بر 2 گروه اسناد:

اسناد کوتاه (سمت چپ محور p): در نمودار جدید، خط به سمت بالا کشیده می‌شود. این یعنی مخرج کسر برای اسناد کوتاه کمی بزرگتر از حالت کسینوسی محاسبه می‌شود تا جلوی برتری بی‌دلیل و کاذب آن‌ها گرفته شود.

اسناد طولانی (سمت راست محور p): خط نمودار به سمت پایین کشیده می‌شود. این یعنی مخرج کسر برای گزارش‌های 100 صفحه‌ای، به اندازه‌ی حالت کسینوسی خام بزرگ نخواهد بود و جریمه‌ی ناعادلانه‌ی آن‌ها تضعیف (Dampen) می‌گردد تا شانس حضور در صدر نتایج را پیدا کنند.

بخش 3) جمع‌بندی:

نکته اول: تفاوت بنیادین میان ریاضیات محض و مهندسی بازیابی اطلاعات در همین فرمول نهفته است. ریاضیات محض می‌گوید طول بردار باید دقیقاً نرمال شود تا زوایا در فضای اقلیدسی معنا پیدا کنند؛ اما مهندسی بازیابی می‌پذیرد که زبان طبیعی از هندسه‌ی محض پیروی نمی‌کند و اسناد طولانی‌تر حق دارند بدون دریافت جریمه‌های سنگین، حاوی اطلاعات جامع‌تری باشند.

نکته دوم: در مسائل تحلیلی، اگر با متون به‌شدت ناهمگون (Heterogeneous) مواجه شدید، بدانید که توابعی که بر مقادیر محلی همان سند تکیه دارند (مانند Maximum TF) به سرعت شکست می‌خورند. راهکار قطعی در این سناریوها، عبور از

متغیرهای محلی و استفاده از متغیرهای سراسری کلکسیون (مانند میانگین طول یا avdl) به عنوان نقطه‌ی مرجع است. الگوریتم‌های پیشرفته‌ای مانند **Okapi BM25** نیز هسته‌ی نرمال‌سازی طول خود را دقیقاً بر پایه‌ی همین مفهوم «محور قرار دادن میانگین طول» بنا کرده‌اند.

پاسخنامه مسئله 6.30: طراحی و پیاده‌سازی ماشین موتور جستجو بر پایه فضای برداری

(Design and Implementation of a Vector Space Search Engine)

بخش 1) تحلیل اولیه و معماری کلاس

هدف از این مسئله، آشنایی با فرآیند ساخت یک سیستم بازیابی اطلاعات (**IR**) از مرحله‌ی پردازش متون خام تا استخراج بردارهای ویژگی، و پیاده‌سازی ریاضیاتی فرمول‌های وزن‌دهی (**TF-IDF**) و شباهت کسینوسی در قالب شیء‌گرایی (**OOP**) با زبان برنامه‌نویسی پایتون است.

برای ساخت یک موتور جستجوی پایه‌ای بر اساس **مدل فضای برداری (Vector Space Model)**، نیازمند طراحی کلاسی هستیم که بتواند فازهای اصلی سیستم بازیابی را به صورت کپسوله (**Encapsulated**) مدیریت کند:

1. **فاز نمایه‌سازی (Indexing):** دریافت اسناد خام (**Raw Documents**)، توکنایز کردن (**Tokenization**) کلمات، و محاسبه فرکانس کلمات (**TF**) و فرکانس معکوس سند (**IDF**).

2. **فاز تولید بردار (Vectorization):** ساخت بردارهای ریاضی برای هر سند و محاسبه پیشاپیش طول بردارها (جهت نرمال‌سازی).

3. **فاز پرس‌وجو (Query Processing):** تبدیل پرس‌وجوی کاربر به یک بردار در همان فضای ریاضی، محاسبه ضرب داخلی (**Dot Product**) و اعمال نرمال‌سازی کسینوسی برای یافتن مرتبط‌ترین اسناد.

بخش 2) پیاده‌سازی گام‌به‌گام الگوریتم در پایتون

```
vector_space_model.py

import math
from collections import defaultdict, Counter

class VectorSpaceModel:
    def __init__(self, raw_documents):
        self.documents = raw_documents
        self.vocab = set()
        self.document_frequencies = defaultdict(int)
        self.document_vectors = []
        self.document_lengths = []
        self.total_docs = len(raw_documents)

        self._build_vector_space()

    def _tokenize(self, text):
        return text.lower().split()

    def _build_vector_space(self):
        tokenized_documents = [self._tokenize(doc) for doc in self.documents]

        # Calculate Document Frequencies (DF)
        for tokens in tokenized_documents:
            unique_tokens = set(tokens)
            for token in unique_tokens:
                self.vocab.add(token)
                self.document_frequencies[token] += 1

        # Construct tf-idf vectors and compute vector lengths
        for tokens in tokenized_documents:
            term_frequencies = Counter(tokens)
            doc_vector = {}
```

```
length_squared = 0.0

for token, tf_raw in term_frequencies.items():
    tf_weight = 1.0 + math.log10(tf_raw)
    idf_weight = math.log10(self.total_docs / self.document_frequencies[token])

    final_weight = tf_weight * idf_weight
    doc_vector[token] = final_weight
    length_squared += (final_weight ** 2)

self.document_vectors.append(doc_vector)
self.document_lengths.append(math.sqrt(length_squared))

def search(self, query_string):
    query_tokens = self._tokenize(query_string)
    query_tf = Counter(query_tokens)

    query_vector = {}
    query_length_squared = 0.0

    # Construct query vector
    for token, tf_raw in query_tf.items():
        if token in self.vocab:
            tf_weight = 1.0 + math.log10(tf_raw)
            idf_weight = math.log10(self.total_docs / self.document_frequencies[token])

            final_weight = tf_weight * idf_weight
            query_vector[token] = final_weight
            query_length_squared += (final_weight ** 2)

    query_length = math.sqrt(query_length_squared)

    if query_length == 0.0:
        return []

    # Calculate Cosine Similarity
    search_results = []
    for doc_id, doc_vector in enumerate(self.document_vectors):
        dot_product = 0.0

        for token, query_weight in query_vector.items():
            if token in doc_vector:
                dot_product += query_weight * doc_vector[token]

        if self.document_lengths[doc_id] > 0.0:
            cosine_score = dot_product / (query_length * self.document_lengths[doc_id])
        else:
            cosine_score = 0.0

        if cosine_score > 0.0:
            search_results.append((doc_id, cosine_score))

    return sorted(search_results, key=lambda item: item[1], reverse=True)
```

بخش 3) جمع‌بندی:

تحلیل پیچیدگی فضا (Space Complexity): در این پیاده‌سازی، به جای ایجاد یک ماتریس دو بعدی متراکم (**Dense Matrix**) که شامل تعداد بی‌شماری صفر باشد، از ساختار داده‌ی **dictionary** در پایتون برای ذخیره بردارهای سند (**doc_vector**) استفاده شده است. این روش تضمین می‌کند که تنها ابعاد غیرصفر (کلمات موجود در متن) فضای حافظه را اشغال کنند، که برای پیاده‌سازی متون واقعی با واژه‌نامه‌های بزرگ بسیار حیاتی است.

پیش‌پردازش ریاضی (Mathematical Pre-computation): محاسبه‌ی طول بردار در زمان جستجو، یک گلوگاه پردازشی به شمار می‌رود. به همین دلیل، در متد **build_vector_space**، متغیر **length_squared** همزمان با تولید بردارها محاسبه شده و در آرایه‌ی **self.document_lengths** کش می‌شود. این تکنیک باعث می‌شود متد **search** در زمان اجرا تنها یک ضرب داخلی ساده انجام دهد. پیچیدگی فضایی در این حالت به جای $O(N)$ به مقدار بهینه $O(\text{Documents containing query terms})$ تقلیل می‌یابد.

نکته مهم: در مسائل کدنویسی مرتبط با بازیابی اطلاعات، همیشه به ساختار مدیریت خطاهای ریاضی (**Mathematical Exceptions**) دقت کنید. در کلاسی که طراحی شد، شرط **if query_length == 0.0** از بروز خطای معروف

ZeroDivisionError جلوگیری می‌کند؛ وضعیتی که زمانی رخ می‌دهد که کلمات جستجو شده توسط کاربر، به هیچوجه در واژه‌نامه (**Vocabulary**) پایگاه داده وجود نداشته باشند.